



MACQUARIE
University
SYDNEY · AUSTRALIA

COMP3850 Project Deliverable Certificate

Name of Deliverable	<i>D4 - User Manual, Team Retrospective and updated D3</i>
Date Submitted	<i>25 / 10 / 2025</i>
Project Group Number	<i>22</i>
Rubric stream being followed for this deliverable (highlight one) <i>Note: the feasibility study has the same rubric for all streams.</i>	SOFTWARE Rubric <i>GAMES Rubric</i> <i>CYBERSECURITY Rubric</i> <i>DATA SCIENCE Rubric</i>

We, the undersigned members of the above Project Group, collectively and individually certify that the above Project Deliverable, as submitted, **is entirely our own work**, other than where explicitly indicated in the deliverable documentation.

INITIAL S	SURNAME	GIVEN NAME	STUDENT NUMBER	SIGNATURE (IN-PERSON OR DIGITAL)
<i>KAA</i>	<i>Ashraf</i>	<i>Khawaja Abeer</i>	<i>47650249</i>	<i>Abeer</i>
<i>EP</i>	<i>Chairudin</i>	<i>Edward Pandiya</i>	<i>47977523</i>	<i>Edward</i>
<i>EK</i>	<i>Kok</i>	<i>Eric</i>	<i>46108572</i>	<i>eric</i>
<i>YD</i>	<i>Diab</i>	<i>Yousef</i>	<i>47359099</i>	<i>Yousef</i>
<i>AA</i>	<i>Abdelghaffar</i>	<i>Amr</i>	<i>47872411</i>	<i>Amr</i>
<i>AG</i>	<i>Gupta</i>	<i>Anirudh</i>	<i>47547081</i>	<i>Anirudh</i>

NB: please write all details clearly (if handwritten).

© Macquarie University, 2021

List of tasks completed for the deliverable and activities since last deliverable certificate with totals for each individual team member and whole team

(copy individual total row for each member and copy pages if more pages needed)

Performed by (Student Names)	Duration (hrs)	Complexity (L, M, H)	Name of task	Checked by (Initials)
Khawaja Abeer	2	M	User manual section 3,6,9	AG
Khawaja Abeer	1.5	M	Team retrospective section 2	AG
Khawaja Abeer	1.5	M	Srs section updated 4.1.2, context diagram 4.1.4, 4.3.1	AG
Khawaja Abeer	0.5	M	Project plan section 4.1 updated	AG
Khawaja Abeer	0.5	M	Project document section 3.1 updated	AG
Khawaja Abeer	1	M	MVP document Added new code	AG
Khawaja Abeer	0.5	L	MVP code snippet of ML visual studio code structure in section 3.2	AG
Khawaja Abeer	1	M	MVP Updated section 4 by adding sponsor feedback of meeting 3 and team action plan on it	AG
Khawaja Abeer	1.5	M	Updated Scrum, Kanban boards and backlog in Jira for project management	AG
Khawaja Abeer	3	H	Researched about new approaches and models like flask app, orm, minmax	AG
Khawaja Abeer	1	M	Incorporated feedback from sponsor into reports	AG
Khawaja Abeer	3	H	Did competitor section coding on visual studio code	AG
Khawaja Abeer	2	M	Updated bm25 ranking algorithm and main.py	AG
Khawaja Abeer	2.5	H	Started coding mongo structure for article retrieval and saving articles for history on visual studio code	AG
Khawaja Abeer	1	L	Reviewed and proofread team deliverables before submission	AG

<i>Khawaja Abeer</i>	0.5	L	Wrote weekly reflections for weekly team report	AG
<i>Eric Kok</i>	1	L	Updated section 2 of the SRS document based on the latest development	YD
<i>Eric Kok</i>	1	L	Updated and added new information to section 3.2 and 3.3 of the MVP document	YD
<i>Eric Kok</i>	0.5	L	Updated the use case diagram and added new use case description as per new requirement	YD
<i>Eric Kok</i>	0.5	L	Created a template for team retrospective document	YD
<i>Eric Kok</i>	1.5	M	Completed section 5 of the user manual document	YD
<i>Eric Kok</i>	1	L	Completed section 3 of the team retrospective document	YD
<i>Eric Kok</i>	1.5	M	Restructured fetcher.py to a modular structure for multiple source fetchers	YD
<i>Eric Kok</i>	4	H	Researched and designed database pipeline	YD
<i>Eric Kok</i>	3	H	Created and documented backend persist.py logic for history, raw insights and AI results collections with Edward	YD
<i>Eric Kok</i>	6	H	Integrated MongoDB Atlas database with backend and tested data persistence with Edward	YD
<i>Eric Kok</i>	0.5	L	Wrote weekly reflection for weekly team report and team meeting minute	YD
<i>Eric Kok</i>	0.5	L	Updated backend's README.md file	YD
<i>Yousef</i>	3	M	Updated previous documentations of deliverable 3 using feedback	AG
<i>Yousef</i>	3	M	Wrote section 4 and 8 in the user manual, and section 6 in the team retrospective.	AG
<i>Yousef</i>	3	M	Redesigned the filter card in the UI following requests made by sponsor	AG

<i>Yousef</i>	4	H	Designed and implemented a ranking system for the priority filter in the UI.	AG
<i>Yousef</i>	3.5	M	Updated the history page to use real stored data that updates as you use it	AG
<i>Yousef</i>	3	M	Update any noticed adjustments needed after backend updates	AG
<i>Yousef</i>	0.5	L	Wrote weekly reflection	AG
<i>Anirudh</i>	2	M	Wrote User Manual section 2 and 6	YD
<i>Anirudh</i>	1	M	Wrote Team retrospective section 4	YD
<i>Anirudh</i>	2	M	Updated section 1 and 2 of project plan	YD
<i>Anirudh</i>	1	L	Updated section 1 of SRS	YD
<i>Anirudh</i>	2	L	Updated MVP for latest code snippets	YD
<i>Anirudh</i>	3	H	Coded content analyser with a hybrid approach	YD
<i>Anirudh</i>	1.5	H	Coded the topic detection logic	YD
<i>Anirudh</i>	1.5	H	Coded the topic summariser logic	YD
<i>Anirudh</i>	3	H	Designed the first draft for database rework (later discarded for a better implementation)	YD
<i>Anirudh</i>	2	M	Reviewed the Deliverable 4 submission docs for consistency and coordinated with the team to fix the issues	YD
<i>Anirudh</i>	2	L	Formatted the document presentation for deliverable 4 submission	YD
<i>Anirudh</i>	0.5	L	Wrote weekly submission	YD
<i>Anirudh</i>	1	M	Reformatted the github repository to be more self-explanatory	YD
<i>Edward</i>	2	M	Design the database system for history implementation, etc	AG
<i>Edward</i>	5	H	Implement the Database design to the mongodb and code, need to	AG

			reconstruct the database schema for every persist function, and also need to create a new collection called history, AI_result, and raw_insight	
<i>Edward</i>	2	M	Reconstruct the function to delete the data in a range of time, for backend maintenance database	AG
<i>Edward</i>	2	M	Make a new endpoint(/search/status) function to check the AI_result field whether the data is processed or not by the AI	AG
<i>Edward</i>	4	H	Make a new endpoint function for trend analysis after search, including the analytics function	AG
<i>Edward</i>	2	M	Reconstruct the /result endpoint so it only returning the history id and preview of history schema (not the data itself)	AG
<i>Edward</i>	3	M	Make a new /result endpoint to show the result of the fetching and AI processed data	AG
<i>Edward</i>	1	M	Call the AI main function in the background every time a new data fetching occurs	AG
<i>Edward</i>	3	M	Making a function to get the top tags and connect it to the endpoint	AG
<i>Edward</i>	3	M	Making a new endpoint to call the history of past search, by making the function and connect it to /history endpoint	AG
<i>Edward</i>	2	M	Make a helper function for AI team to used for easier persist on database	AG
<i>Edward</i>	1	L	Reformatting the code, and delete some duplication on backend folder code	AG
<i>Edward</i>	1	L	Do part 1 and 7 for retrospective document	AG
<i>Edward</i>	1	L	Do part 7 on user_manual	AG
<i>Edward</i>	2	M	Adding some test case specification and do some additional testing on the front end	AG

Amr Abdelghaffar	0.5	M	Reviewing AI/NLP portion of code in GitHub	AG
Amr Abdelghaffar	1	L	Create draft of Section 1 of User Manual	AG
Amr Abdelghaffar	1.5	M	Create draft of Section 5 of Team Retrospective	AG
Amr Abdelghaffar	1	M	Complete Section 5 of Team Retrospective	AG
Amr Abdelghaffar	0.5	L	Improve and finalise Section 1 of User Manual	AG
Amr Abdelghaffar	1.5	M	Review Sections 2.4-2.7 of SRS and Section 2 of MVP document. Ensuring it aligns with new requirements and improving the formatting.	AG
Amr Abdelghaffar	1	L	Proofreading various sections of the documents, improving formatting and image sizes for readability.	AG
<i>Khawaja Abeer Total</i>	23			
<i>Yousef Diab</i>	20			
<i>Eric Kok</i>	21			
Edward Pandiya Chairudin	34			
<i>Anirudh Gupta</i>	22.5			
<i>Amr</i>	6			
Team Total	126.5			



<AI-Powered Web Application for Social Listening>

Version <03>, <25/10/2025>

Contents

1. Handover Requirements

2. Statement of Purpose

3. Risk Management

- 3.1. ACS Code of Ethics Overview
- 3.2. Team Values

4. Resource Management

- 4.1. People
- 5.2. Hardware, Software, Other

5. Change management

- 5.1. Managing requirements and scope change
- 5.2 Version Control

6. Quality Management

- 6.1. Managing Change
- 6.2. Conflict Resolution

7. Project Schedule

8. Assumptions

9. Standards/Templates/Appendices/Forms

Revision History

Name	Date	Reason For Changes	Version
Anirudh	30/09/25	Added handover requirements	2.0
Anirudh	1/10/25	Added the revision history table	2.0
Anirudh	1/10/25	Revised the index	2.0
Khawaja	1/10/25	Added new risk	2.0
Khawaja	1/10/25	Updated Jira task breakdown and timeline	2.0
Anirudh	1/10/25	Updated the Statement of purpose	2.0
Eric	2/10/25	Added agile approach in roles and responsibilities section	2.0
Eric	2/10/25	Added new information in section 4.2	2.0
Edward	2/10/2025	Adding the Software tools on 4.2	2.0
Khawaja	23/10/25	Updated Jira task breakdown and timeline	3.0
Anirudh	23/10/25	Updated Handover requirements	3.0
Anirudh	23/10/25	Updated the Scope of the project	3.0
Yousef	23/10/25	Updated the assumptions formatting	3.0
Eric	24/10/25	Reformatted the table for better visualisation	3.0

1. Handover Requirements

For the handover of this project, the team must provide a well-documented and reproducible package that allows any new developer to run, maintain and extend the system. The following things will be kept in mind during the handover:

LLM Configuration: The current pipeline uses the **Nemotron Nano 9B V2 model via OpenRouter** under a free-tier API key. The sponsor must **replace the existing openAI key** with their own API key if they wish to use the same model. Instructions for changing the existing key/model will also be provided in the readme file where applicable.

Github Repository: The github repository structure should be **self-explanatory** with the AI components, such as algorithms, in the AI folder, the MongoDB utilities in the Mongo folder and Django backend in backend folder. A detailed **README** must be included which outlines the steps to configure environment files, install dependencies and deploy the application.

Database Configuration: The **MongoDB schema**, as stored in Atlas, should be documented with its key fields such as url, title, body and engagement metrics. **Indexes, retention rules and deduplication strategies** must also be described.

API Configuration: The sponsor will need to **replace all the API keys** that are used by django for web scraping with their own. A **README** file will contain the instructions on how to replace all the API keys present in the project.

User Management: For **testing purposes**, we have implemented a **dummy user**. However, the sponsor has mentioned that the **user management is not in the scope** of this project and the **integration will be handled by their team** after the handover.

Algorithm Management: Each algorithm used should be explained with its configuration and formula and how they are fused into the final score.

Test Cases Documentation: The test cases should be **well documented** and also included in the handover report to document the application's performance with edge cases.

The sponsor will be demonstrated on the entire pipeline and will be provided with a user manual to ensure smooth operation in the future.

2. Statement of purpose [for plan] and scope [of project]

Statement of Purpose:

This project aims to design and develop an **AI-driven web-based application** that assists users in efficiently locating, evaluating, and prioritizing research articles sourced from a range of online platforms. By integrating **machine learning (ML)** and **natural language processing (NLP)** techniques, the system will assign each article a **relevancy score** based on its semantic alignment with the user's query. It will also take into account the engagement on an article for relevancy.

The purpose of the application is to **reduce information overload** and significantly decrease the time users spend manually searching and filtering through large volumes of content. This allows researchers, analysts, and organizations to focus their efforts on **interpretation, insight generation, and strategic decision-making** rather than data retrieval.

In addition to ranking results, the system will provide **basic source and competitor identification** to support market intelligence use cases. The platform therefore offers value both as a **research support tool** and as a **decision-support asset** for sponsor organization. The system will also provide the users with **potential opportunities** with the articles displayed.

Scope:

The project involves the development of a MVP with the following requirements:

1. A **web-based user interface** for submitting user queries/keywords.
2. **Automated data retrieval** with the help of API's.
3. **AI ranker** that ranks the retrieved data based on semantic similarity to the keyword.
4. A **MongoDB non relational** database layer for storing articles, history and related material.
5. **Filter** option to filter out the articles based on the user requirements.
6. A card for **backlinks** with the links to most engaging posts for easy access.
7. A **news feed** card that shows the latest news relevant to the user.

8. A **Trending topics** card that displays all the trending topics that are relevant to the user.
9. **Competitor analysis** will be conducted based on the company information to display what user's competitors are up to.
10. An **idea card** to provide ideas to a user based on the content they search.
11. **NLP algorithm** to detect the **topic** of an article and **summarise** it.

The following things are not in the scope of this project:

1. **User authentication** and **account management** (to be handled by the sponsor post-handover)
2. Integration with **paid or restricted-access content sources**.
3. **Advanced sentiment** or **misinformation classification features**.
4. Real-time **analytics dashboards** beyond the ranking view.
5. **Integration of the action-selection** feature with the sponsor's existing publishing or workflow systems (UI only is provided; backend automation will be handled by the sponsor post-handover).

This aims to cater to a variety of stakeholders, including research analysts, academic researchers, data scientists or strategy teams or research-led organizations.

3. Risk Management

Effective risk management is critical to ensuring the success of this project, given its reliance on external data sources, advanced machine learning (ML) and natural language processing (NLP) models, and compliance with strict ethical and regulatory standards. This section outlines the team's approach to identifying, categorising, and mitigating risks throughout the project lifecycle.

3.1 ACS Code of Ethics Overview

The project team commits to upholding the **ACS Code of Ethics**, which emphasises:

- **Honesty:** Transparent communication with stakeholders, accurate documentation, and responsible disclosure of project limitations.
- **Trustworthiness:** Protecting sponsor, user, and organisational data, ensuring outputs are reliable and unbiased.
- **Fairness:** Designing ML/NLP models that avoid demographic or contextual bias, validated against diverse datasets.
- **Professional Development:** Ensuring continuous learning within the team to address unfamiliar technologies and improve outcomes.

- **Public Interest:** Respecting privacy laws (GDPR, CCPA, Australian Privacy Act), API terms of service, and accessibility standards (WCAG 2.1 AA).

These principles guide the identification of risks, their treatment, and the ongoing monitoring process.

3.2 Team Values in Risk Management

The team's values; **openness, collaboration, respect, and trust**, play an active role in risk mitigation. Open communication ensures risks are flagged early, collaboration allows multiple perspectives in finding solutions, respect ensures that differences are constructively handled, and trust fosters accountability when risks require urgent action.

3.3 Risk Identification and Categories

The following risk categories have been identified:

- **Technical Risks:** Issues related to APIs, ML/NLP performance, data quality, and integration across system components.
- **Ethical and Legal Risks:** Risks of bias in algorithms, inadvertent data privacy violations, or breaches of API licensing terms.
- **Operational Risks:** Misalignment between backend, frontend, and AI teams; schedule compression near deliverables; sponsor feedback delays.
- **Resource Risks:** Limited student availability due to academic workload, reliance on free-tier APIs, or outages in cloud services.
- **Reputational Risks:** Incorrect or irrelevant outputs reducing user trust, or publishing a system that fails accessibility standards.

3.4 Risk Matrix

Key

- **Likelihood/Impact:** L / M / H / VH
- **Level:** computed qualitatively (Low/Med/High/Extreme)

Risk Register (Matrix)

ID	Risk	Category	Likelihood	Impact	Level	Early Signals / Triggers	Mitigation (Preventive)	Contingency (Reactive)
R1	API rate-limit volatility / quota cuts	Technical / External	M	H	High	Spike in 429s; vendor policy emails; slowed fetch	Multi-provider fallback, caching of recent queries; staggered calls; monitor quotas	Temporarily degrade to cached results; pause affected source; notify sponsor
R2	Platform ToS changes / breach	Legal / Compliance	M	H	High	ToS update notices; blocked endpoints	ToS review before integration; whitelist endpoints; log access patterns	Disable offending calls; contact vendor; switch provider
R3	Use of unauthorised APIs or methods	Legal / Compliance	L	H	Med	Missing API keys; scraping endpoints without permission	Use only public/approved APIs; keep a source registry	Remove offending integration; replace with compliant source
R4	Query ambiguity (Apple/Jaguar)	Functional / UX / ML	H	M	High	Irrelevant top results; user filters overused	Entity linking; user disambiguation prompts (industry/location); domain filters	Escalate to manual label rules; hotfix query-expansion blacklist
R5	Bias in ML/NLP models	Ethical / Quality	M	H	High	Skewed outcomes by group/topic; audit flags	Diverse training data; bias tests; fairness metrics in CI	Retrain with balanced sets; suppress biased features
R6	Irrelevant results / low precision	Quality / ML	M	H	High	Low CTR; sponsor feedback; poor validation metrics	Hybrid scoring (lexical+semantic+engagement); threshold tuning; AB tests	Roll back model; increase manual rules; fast patch weights

R7	Performance/latency (large datasets)	Technical	M	M	Med	P95 latency > target; timeouts	Lightweight models for MVP (MiniLM); batch scoring; async queues; DB indexes	Add caching; precompute hot queries; temporarily reduce result size
R8	Cloud/tool dependency outage (GitHub, MongoDB, CI/CD)	Operational	M	M	Med	Vendor status red; failed pipelines	Local dev fallbacks; weekly offline backups; infra runbook	Switch to local DB; commit offline; re-run jobs after recovery
R9	Data privacy non-compliance (PII)	Legal / Compliance	L	VH	Extreme	PII detected in logs; user complaints	Data minimisation; PII detectors in pipeline; suppression list; DPIA checklist	Purge data; rotate keys; breach notification per policy
R10	IP / copyright on republication	Legal	M	H	High	DMCA/takedown notices; blocked embeds	Store metadata/snippets; link out; respect robots/no-republish terms	Remove content; switch to summaries/links; legal review
R11	Toxic/unsafe content exposure	Safety / UX	M	H	High	Profanity/hate in results; user reports	Content filters (blocklists, toxicity model); safe search flags	Quarantine items; refine filters; add "report" flow
R12	Accessibility non-compliance (WCAG 2.1 AA)	Compliance / UX	L	H	Med	Failing axe tests; QA findings	Design reviews by QA; automated a11y tests; keyboard/contrast checks	Triage fixes; a11y hotfix sprint; defer features that block compliance
R13	Integration drift (schema/API contracts)	Technical / Process	M	H	High	4xx/5xx between services; failing contract tests	Shared OpenAPI schema; contract tests in CI; versioned endpoints	Freeze API; patch adapters; post-mortem & update contracts

R14	Scope creep	Delivery	M	H	High	New features mid-sprint; backlog ballooning	MoSCoW prioritisation; change-control board; sponsor sign-offs	De-scope to MVP; defer to later milestone
R15	Sponsor feedback delays	Stakeholder	M	M	Med	PRDs pending; blocked approvals	Set SLAs (3–5 days); provide decision packets; early demos	Proceed with conservative defaults; flag risks in status report
R16	Schedule compression near D3/D4	Delivery	H	H	Extreme	Testing tasks squeezed; rising open bugs	Feature freeze 7 days pre-deadline; test/handover buffer week	Slip non-critical scope; add all-hands test day
R17	Student capacity/time conflicts	Resource	M	M	Med	Exams week; missed stand-ups	Load-leveling in sprint planning; smaller tickets; backups per task	Reassign tasks; cut non-essentials; extend hours short-term
R18	Knowledge silos (single-point failure)	Team / Process	M	H	High	Only one approver on module; blocked reviews	Pair programming; module READMEs; cross-reviews	Swarm on module; rotate ownership
R19	Honesty/communication lapses	Governance / Ethics	L	H	Med	Undisclosed blockers; late surprises	Daily updates; blocker flag within 24h; retro culture	Escalate to lead; reset plan; transparent sponsor note
R20	Security breach (keys, secrets)	Security	L	VH	Extreme	Secrets in repo; unusual traffic	.env/secret vault; least privilege; secret scanning; HTTPS everywhere	Revoke/rotate; incident response; audit & patch

R21	Version control/merge regressions	Quality / Process	M	M	Med	Broken main; flaky builds	Branch protection; required reviews; CI test gates	Hotfix branch; revert; post-mortem
R22	Source data quality (spam/low-signal)	Data Quality	M	M	Med	Many low-quality posts; user complaints	Source whitelists; spam filters; engagement heuristics	Down-rank source; manual curation list
R23	LLM hallucinations	ML / Technical	M	H	High	Summaries/facts inconsistent with source	Use retrieval-augmented grounding, limit to extractive outputs, add explainability	Fallback to extractive summaries; flag & review outputs

Monitoring & Reporting

- **Risk Register Location:** GitHub/Confluence (single source of truth)
- **Review Cadence:** Weekly team meeting; escalation within 48h for High/Extreme risks
- **Status Codes:** Open / Monitoring / Mitigating / Closed (update each sprint)

3.5 Monitoring and Reporting

- **Weekly Team Meetings:** Progress and risks are reviewed; new risks logged.
- **Risk Register:** Maintained in GitHub/Confluence with updates to status, owners, and mitigation actions.
- **Sponsor Engagement:** Risks impacting scope, compliance, or deliverables are flagged in fortnightly meetings.
- **Escalation Policy:** Unresolved risks are escalated to the sponsor or unit convenor within 48 hours.

4. Resource Management

4.1. People

Team values

To work effectively as a team, we agreed on a set of shared values that guide how we collaborate and support each other. Here are the following values we uphold:

1) **Openness:**

Openness is essential for keeping both the team and the sponsor aligned throughout the project. Research in project management highlights that transparency reduces misunderstandings and strengthens accountability. Given that the project spans frontend, backend and AI components, there is a risk of one area becoming disconnected from the others. Therefore, by sharing progress, delays and obstacles openly, the team ensures that potential issues are surfaced early and addressed collaboratively. For example, if the backend team encounters delays with API integration, openness allows the AI developers to adjust their testing timelines rather than waiting idly.

2) **Collaboration:**

Collaboration is one of the foundations of team building as it allows us to share ideas, support one another and solve problems faster. Studies in software engineering show that practices such as pair programming and peer code reviews lead to higher quality outcomes and fewer errors. In a project like ours, where multiple layers of technology have to work together, collaboration ensures we do not build in isolation. In practice, this means working together on tasks that cut across roles, such as designing APIs jointly between backend and AI or reviewing frontend code as a group to ensure consistency.

3) **Respect:**

Respect underpins everything we do and is critical for building a positive and productive working culture. Findings from Google's Project Aristotle emphasise that psychological safety, which is the ability to contribute without fear of judgment, is a key driver of team effectiveness. Our group includes members with different levels of expertise, so showing respect means recognising these differences while valuing everyone's contribution. Rather than dismissing questions, we aim to answer them clearly and supportively. For instance, if someone without an AI background asks for clarification on a model's output, the AI developers will explain the reasoning behind it in a way that helps the whole team learn.

4) **Trust:**

Trust is what allows us to move forward without second-guessing each other. Organisational studies show that teams with high trust spend less time monitoring and more time producing meaningful work. For us, this means once a task is assigned, we assume it will be delivered unless otherwise communicated. This helps with planning and coordination, as other members can build their own timelines around those commitments. For example, if the backend team commits to delivering endpoints by Friday, the AI team can plan to start integration immediately afterward, confident that the work will be ready. Trust, in this sense, allows us to work efficiently under tight deadlines.

Team structure

The project team is organised into clearly defined roles to ensure accountability, balanced workload and effective collaboration. Each member has been assigned responsibilities that align with their strengths and contribute to the overall success of the project.

Name	Role	Responsibility Highlight
Anirudh Gupta	Group leader	Oversees project direction and coordination
Khawaja Abeer	Technical lead	Guides technical decisions and architecture
Eric Kok	Documentation lead	Main meeting minutes and documentation
Yousef Diab	Quality assurance lead	Ensures deliverables meet quality standards
Amr Abdelghaffar	Quality assurance lead	Support quality checks and validation
Edward Chairudin	Testing lead	Designs and manages testing processes

Roles and responsibilities

To ensure clarity and accountability, each role in the team has been matched with specific responsibilities based on expertise and project needs, as shown below.

While these roles guide our work, the team adopts an agile approach. If workloads change or unexpected challenges arise, responsibilities can be redistributed to ensure balance and timely delivery. This flexibility ensures that no individual is overburdened and that critical tasks are always prioritised.

Role	Responsibilities	Assigned To
Web and app developer	Frontend development, Django templating and UX/UI design	Yousef
Software tech developer	Backend logic, Django setup, API integration, database management, deployment	Eric, Edward
ML/AI developer	NLP model development, data processing, data scraping, topic relevance scoring	Anirudh, Khawaja, Amr

Note: Roles are not rigid. For example, if the AI team faces increased workload near integration, backend developers may temporarily assist with data handling or testing tasks. Likewise, frontend contributors may review documentation or QA tasks if capacity allows.

Stakeholders

Our project involves several stakeholders, each playing a different role in guiding, supporting and ultimately benefiting from the system. These can be grouped into internal and external stakeholders:

Category	Stakeholder	Role / interest
Internal	Project team members	Responsible for design, coding, testing and documentation to deliver the final product.
	Sponsor (Robotic Marketer)	Defines requirements, reviews progress and ensures the system supports business objectives.
External	End users	Use the system to access ranked and summarised articles for research/insight.
	Unit convenor & teaching staff	Ensure academic alignment, assess deliverables and provide guidance if conflicts arise.
	Compliance / legal advisors	Ensure ethical AI use, data privacy compliance and adherence to API licensing terms.

Communication pattern

1) Day to day communication

- Quick updates, reminders and coordination take place on WhatsApp and Discord. These channels are used for informal and immediate interactions among team members.

2) Weekly team communication

- The team holds weekly Discord meetings to review progress, assign new tasks and address any ongoing issues or obstacles.

3) Weekly sponsor check-ins

- Formal meetings with the sponsor are conducted on Microsoft Teams every week. These sessions are used to share updates, discuss potential challenges and gather feedback for alignment.

4) Documentation collaboration

- Google Docs is used for drafting, collaborative writing and maintaining technical documentation.

5) **Code collaboration**

- GitHub is also used for pull requests, code reviews and collaborative development, which ensures that version control and quality checks are consistently applied.

6) **Service-level expectations for communication**

- Internal team chats (WhatsApp/Discord): responses expected within 4 hours on weekdays.
- Sponsor communication (via email): responses expected within 24–48 hours.
- Critical blockers: flagged within 24 hours during stand-ups or directly in the group.

Conflict resolution strategies

Strategy 1 - reactive resolution

This approach focuses on resolving issues in progressive stages:

- 1) **Direct conflict:** If there is a conflict between team members, the first attempt is to resolve disagreements privately and respectfully.
- 2) **Team review:** If the conflict is still unresolved, the issue will be raised in the weekly team meeting for open discussion.
- 3) **Mediation:** If there is no majority vote, the team leader will step in to mediate and suggest a compromise.
- 4) **Sponsor intervention:** If the issue is still unresolved, the matter will be escalated to the sponsor for ultimate decision.
- 5) **Prevention step:** The team will reflect on the conflict and come up with an improved conflict resolution strategy for potential conflict in order to reduce the chance of a conflict escalating further.

Strategy 2 - proactive measures

This approach emphasises proactive steps to avoid conflicts before they escalate:

- 1) **Clear role definition:** Ensure all members understand their roles and responsibilities to reduce overlap or misunderstandings.
- 2) **Shared values:** Reiterate team values (openness, collaboration, respect, trust) at the start of meetings to set the tone.
- 3) **Check-ins:** Hold quick mid-week check-ins to catch frustrations early.
- 4) **Documentation:** Keep transparent records of decisions and expectations in shared platforms (Google Docs, GitHub) so no one feels left out.
- 5) **Feedback culture:** Encourage constructive feedback to surface disagreements as “ideas to improve” rather than personal disputes.

4.2. Hardware, Software and Other

To deliver this project effectively, the team relies on a combination of hardware, software and collaborative tools. These resources support development, testing, version control and communication which ensures that both technical and organisational needs are met. Below is a breakdown of the key resources we will be using, grouped by category.

Hardware

- **Servers for AI model training:** Cloud-based servers capable of handling model training and inference workloads. These provide the required compute resources (CPU/GPU) for running NLP models.
- **Local development machines (Fallback strategy):** Each team member's laptop/PC serves as a backup in case cloud services become unavailable.

Software

- **GitHub:** Version control for both code and documentation, enabling branching, pull requests and collaborative reviews.
- **GitHub models:** Provides access to hosted pre-trained AL/ML models for quick prototyping and integration. As it reduces the need to train from scratch and accelerates development.
- **Hugging Face Transformers:** Open-source summarisation models (T5 via Hugging Face) are used to generate concise article summaries.
- **MongoDB Atlas:** Cloud-hosted NoSQL database cluster (free shared tier) for storing articles, metadata and processed outputs.
- **Django REST Framework:** Backend framework for API development and orchestration between data sources and AI components.
- **HTML/CSS/JavaScript:** Core web technologies for building and styling the frontend interface.
- **Large Language Model:** Supports semantic analysis and generates concise article summaries.
- **Scikit-learn:** Python library used for TF-IDF, lexical analysis and feature extraction for ranking.
- **Figma:** Collaborative design tool for creating and iterating on UI wireframes.
- **Playwright:** End-to-end testing framework ensuring frontend reliability and consistent behaviour across browsers.
- **Jira:** Project management tool for sprint planning, backlog tracking and Scrum practices.
- **Confluence or GitHub Wiki:** Used to document technical decisions, processes and knowledge sharing.
- **Feedparser:** Python library for parsing RSS feeds (used to extract articles from Guardian, BBC, TechCrunch, etc.)
- **Requests:** HTTP library used for fetching external data and APIs.

- **python-dotenv:** For managing environment variables securely (e.g., API keys, DB connections).

Other resources

- **API access to external sources:** Access keys for platforms such as Reddit, and Guardian API and other news RSS feeds are self provided. It is used within rate-limit constraints (Reddit 60 requests/minute). At this stage, free public APIs or with less restrictions are preferred.
- **Communication tools:** WhatsApp for day-to-day updates, Microsoft Teams for sponsor meetings and Discord for weekly check-ins.

5. Change Management

5.1. Managing requirements and scope change

Documentation of Changes:

Given the ever-changing nature of the project, the team must maintain a clear procedure to manage changes in the plan. Any changes to the requirements or project scope will first be documented clearly, ensuring that no sponsor requests are overlooked, and to also allow all team members to have access to the information. The team will then ask for any needed clarifications on the details regarding the sponsor request, ensuring there is no ambiguity or misunderstanding while the plan is being updated. Once confirmed, the project plan will be updated to reflect the changes.

Assessment of Timeline Impact:

Each change will be assessed for its impact on the project timeline while the plan is being updated. If necessary, the existing timeline will be updated to accommodate for the changes, ensuring the plan continues to be realistic and achievable within the required deadline. A change request log will then be made, recording the date, description of change, the approval status, and the owner responsible for managing the change. A two-day approval policy will be applied for sponsor approval of the updated plans made, ensuring no unnecessary delays are created.

Prioritisation of Change Requests:

To appropriately prioritise the change requests, the MoSCoW method will be used, requiring all changes are classified as “Must Have”, “Should Have”, “Could Have”, or “Won’t Have”, with advice from the sponsor. This ensures both the team and sponsor have a shared understanding of the relative importance of each change request. Additionally, for change requests that involve affecting the application’s interface, cross browser compatibility must be reviewed before the change is implemented, to ensure alignment with existing standards.

Integration into Agile Workflow:

After finalising the change planning and approval process, the change should be mapped to a Jira issue, with linked pull requests. Then the Jira issue should be assigned to either the current sprint week, or deferred to the next sprint if the approval process had been completed after day 3 of the 1-week sprint. This ensures that the changes made do not potentially lead to delays in delivering the current sprint's goals.

By following this structured process, the project team can ensure that changes are managed transparently, whilst meeting sponsor expectations, reducing the risk of miscommunication, overlooked changes, or delays to the project.

5.2. Version Control

Version control will be applied to documents and data alongside the code. GitHub will be used as the central repository for managing all files, providing transparency and accountability across the entire project. The team will adopt consistent branch naming conventions and require all pull requests to be reviewed before any changes are merged into the main branch. Each version of the project will also include release tags, using semantic versioning aligned to the deliverable each file is for. This ensures that all updates are peer-reviewed, reducing the chance of errors being introduced to the project, improving the overall quality.

To maintain accuracy and professionalism in both code and documentation, quality leads will review code and documents prior to submission. This process provides an additional opportunity for inaccuracies or errors to be found prior to submission. By using GitHub, the team will minimise the risk of version conflicts, maintain clear records of all updates and their author, improving the team's ability to work collaboratively.

6. Quality Management

Maintaining the platform's quality is essential to ensure it delivers relevant and accurate information. Since the application sources data from external data sources, quality management will focus on: Data integrity, ML model performance and system reliability. A structured quality control plan will be adopted to monitor, evaluate and continuously improve the project deliverables.

- **Document Quality:** All project documents (Team manuals, feasibility report, project plan, etc) will be reviewed by the Quality Assurance Leads before submission
- **Progress Monitoring:** Weekly team meetings are scheduled to ensure deliverables are met promptly and risks are detected early.
- **Test Planning:** The Testing Lead will design test cases covering data extraction, ML relevance scoring, and UI output to validate end-to-end functionality.
- **Sponsor Engagement:** Fortnightly meetings with the sponsor to ensure sponsor feedback on progress and alignment of expectations.
- **Code Assurance:** GitHub branch protection and mandatory code reviews will safeguard code quality, version control, and maintainability
- **Team Communication:** Open discussions will be encouraged to raise issues early and collaborate on solutions.
- **Agile Review:** Sprint reviews at the end of each cycle enable the team to evaluate progress, identify areas for improvement, and refine upcoming tasks.

- **Automated Testing:** Continuous Integration pipelines will include automated unit, integration, and end-to-end testing to ensure robustness of keyword search, data retrieval, and ML scoring components.

6.1. Managing Change

Given the evolving nature of external data sources (Reddit, linkedin, X, etc.) and machine learning models, changes to the project are expected. To manage change effectively:

- A change Log will be maintained on GitHub, which contains all the change requests.
- Each change will be evaluated based on its impact on scope, timeline and quality.
- Changes to a new ML Model will require testing and validation before adoption.
- Approval of major changes will require discussion in weekly meetings and confirmation from the sponsor.
- Minor changes (e.g., bug fixes, UI tweaks) may be approved directly by the Quality Lead.
- Any accepted changes will be documented and reflected in the sprint backlog, ensuring traceability.

6.2. Conflict Resolution

Team-based projects often involve differing opinions, particularly around technical decisions such as ML model selection, feature prioritisation, or testing approaches. To prevent conflicts from affecting progress:

- **Early identification:** Team members are encouraged to raise concerns immediately during stand-ups or weekly meetings.
- **Open discussion:** Conflicts will first be addressed collaboratively within the team to find a compromise
- **Escalation:** If no agreement is reached, the issue will be escalated to the Project Lead for resolution.
- **Sponsor involvement:** For decisions impacting project scope or deliverables, the sponsor's feedback will be sought.
- **Documentation:** Outcomes of conflict resolution will be recorded in meeting notes to ensure clarity and accountability.
- **Convenor involvement:** For major disruptions, a dialogue will be opened with the convenor to seek a solution.

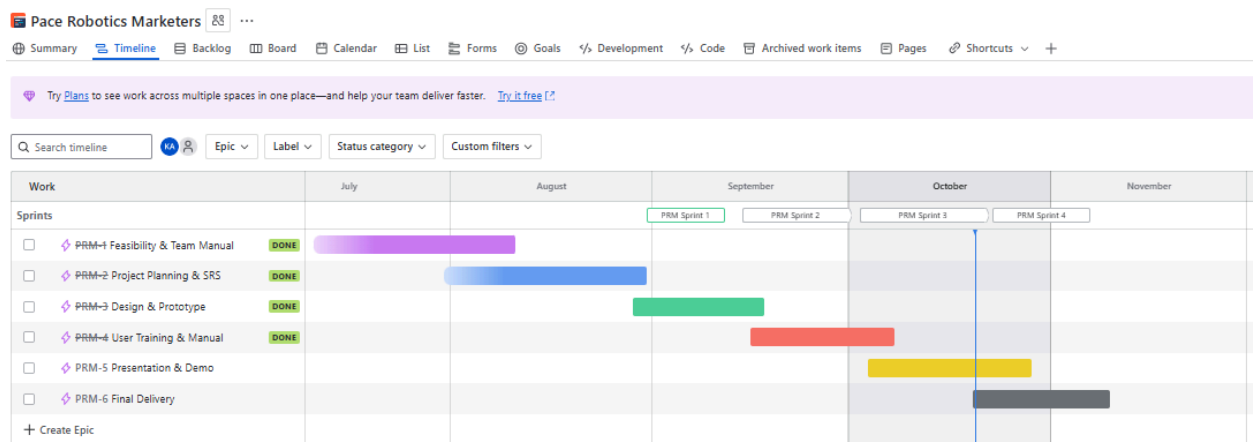
Any decision requires a vote from the entire team to be approved. If a resolution is not reached, the final say lies with the Team Leader.

7. Project Schedule

Every week we try to work in sprints. Before a deadline we stop adding new things, test, fix, and polish. The checklist includes testing if every automated test possibly passes, if there any blocking bugs may be around, and if it runs on Chrome, Edge, Safari. Changed docs updated, demo runs end-to-end, and approvals recorded.

7.1. Jira for Project Management

Project Timeline:



Task breakdown:

Weeks 7–8 – Scoring MVP	Implement lexical + semantic scoring; build server-rendered Search & History pages; add pagination + loading states; simple keyboard navigation	Working results list; basic scoring; screenshots; test cases (happy path)	Anirudh, Amr, Khawaja (scoring), Yousef (pages)	Open-source NLP libs, Django templates, PyMongo, Python 3.12
Week 9 – Prototype	End-to-end path input → ranking → output; error/empty states; simple logs for failures; prepare demo	Demo-ready build; demo script; short bug list	All	Same stack; slide outline; screen recorder
Week 10 – Hardening	Fix bugs; cross-browser checks (Chrome/Edge/Safari); keyboard-only path (skip link, visible focus); small speed improvements	Test summary; updated docs; fixed UI issues	Yousef (UI/a11y), Eric, Edward (API), AI team (scoring fixes)	Browsers on desktop/mobile, checklists, GitHub Issues
Week 11 – Docs & Training	User/training notes; operations runbook (start/stop, env vars); improve tests; light refactor for clarity	User/training doc; runbook; updated tests	Eric (docs/runbook), Yousef (UI guide), All (reviews)	Repo docs, README/runbook templates
Week 12 – Presentation	Polish demo; deck + script; rehearse; apply feedback	Polished demo; slide deck; rehearsal notes	All (presenters), Yousef (UI walkthrough)	Slides, demo dataset, meeting room/Teams

Final – Handover	Package code, docs, runbook; credentials checklist; sponsor sign-off	Final handover pack; sign-off record	Leads (handover), All (checks)	GitHub release/export, checklist template
-------------------------	--	--------------------------------------	--------------------------------	---

8. Assumptions

APIs and data

- /api/search/ and /api/history/ contracts stay stable. No JSON field changes.
- Data-retrieval APIs and datasets remain available at current levels and cost.
- Terms of service do not change in a material way.
- API changes are announced at least one sprint in advance.
- Predictable seed data exists for non-empty, empty, and error cases.
- Returned content is licensed for snippet display and deep links.

Environment and infrastructure

- Two servers run in development at the same time.
- A consistent API_BASE is used.
- Result links are absolute http or https URLs.
- Backend allows the frontend origin via CORS.
- Services used by the project remain accessible: Fly.io, GitHub Actions, MongoDB Atlas.
- Reliable internet and cloud access are available.
- Production hosting provides TLS and domain routing.
- Network errors and timeouts surface clear UI messages.

Tooling and compatibility

- Django deployment is compatible with Playwright and Figma for testing and design only.
- Target browsers are the latest Chrome and one contemporary secondary browser.
- No legacy browser support.

Security, privacy, accessibility

- Keyboard navigation and sufficient contrast are maintained.
- No PII is stored client-side beyond sessionStorage.
- Logs avoid sensitive data.
- Non expiring development credentials are available when needed and stored securely.

Process and team

- Weekly and ad hoc check-ins occur on Friday at 3 PM.
- Work is managed as small, testable tasks in GitHub Projects.

- Sponsor feedback arrives within 3 to 5 days.
- The client supplies enough competitor context to build and update the competitors list.
- Equal participation is expected, with reasonable adjustments during tests and assignments.

9. Standards/Templates/Appendices/Forms

9.1 Coding & configuration standards.

- Python 3.12+, Django 5.2+; follow PEP 8. Use semantic HTML and plain CSS (no framework).
- Configuration via environment variables; no secrets in source control.

9.2 Version control & reviews.

- GitHub with branch protection; all changes via Pull Request.
- Conventional Commits for clear history; at least one reviewer approval before merge.

9.3 Documentation standards.

- SRS, Project Plan, Team Manual, and Feasibility kept in version control and updated per release.
- Architecture Decision Records (ADRs) for significant technical/process choices.
- Meeting minutes captured for all client/team meetings and stored with the project docs.

9.4 Testing & quality gates.

- Unit/integration tests for backend functions; basic E2E coverage for main user flows.
- Accessibility basics: keyboard-only operation, skip link, visible focus, sensible tab order.
- Cross-browser smoke: Chrome, Edge, Safari (desktop/mobile).
- CI must pass before merge; no open P1 defects at release time.

9.5 Security & privacy practices.

- HTTPS for all traffic; input/output validation and sanitization.
- API keys/credentials remain server-side (MONGODB_URI); logs minimized and exclude PII.
- Respect external platform Terms of Service and applicable privacy obligations.

9.6 Accessibility & UX standards.

- Aim for WCAG 2.1 AA: clear headings/labels, focus visible, link purpose clear, skip-to-content, logical focus order.
- Consistent layout and terminology across Search and History screens; clear empty/error states.

9.7 Templates (provided with the repo/docs).

- Meeting minutes, weekly report, change request, risk register, test case sheet, bug report, release notes, ADR template, handover checklist.

9.8 Appendices.

- API & Data Source Register (endpoints, limits, ToS).
- Glossary (project terms and acronyms).
- License & Attribution Register (libraries/datasets).
- Operations Runbook (start/stop, environment variables, backups).

9.9 Forms.

- Change Request Form (scope, rationale, impact, approval).
- Issue/Bug Form (steps to reproduce, expected/actual, severity).
- Risk Log Entry (risk, likelihood/impact, owner, mitigation).
- Release Sign-off (checklist, approvers, date/tag).

NOTE: This template is shareware downloaded from **HYPERLINK** "<http://www.processimpact.com>" www.processimpact.com. All shareware payments are donated to the Norm Kerth Benefit Fund to help a consultant who is disabled with a brain injury. Please visit **HYPERLINK** "http://www.processimpact.com/norm_kerth.html" http://www.processimpact.com/norm_kerth.html to make a shareware payment (\$10 suggested). Thank you!

Software Requirements Specification

for

AI-Powered Web Application for Social Listening

Version 3.0 approved

Prepared by Team 22

Robotics Marketers Pty Ltd

25/10/2025

eLearning versions of several popular Process Impact training seminars are available at
HYPERLINK "<http://www.processimpact.com/elearning.shtml>"
www.processimpact.com/elearning.shtml, including "In Search of Excellent Requirements,"
"Exploring User Requirements with Use Cases," "Writing High-Quality Requirements,"
"Software Inspections and Peer Reviews," and "Project Management Best Practices".
Single-user and corporate-wide site licenses are both available.

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	5
1.4 Project Scope	5
1.5 References	6
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Features	7
2.3 User Classes and Characteristics	9
2.4 Operating Environment	10
2.5 Design and Implementation Constraints	10
2.6 User Documentation	10
2.7 Assumptions and Dependencies	11
3. System Features	11
3.1 System Feature 1 - Article retrieval	11
3.2 System Feature 2 - Relevance Scoring & Ranking	12
3.2.1 Description and Priority	12
3.2.2 Stimulus/Response Sequences	12
3.2.3 Functional Requirements	12
3.3 System Feature 3 - Filtering, Disambiguation & Facets - elaborate	12
3.3.1 Description and Priority	12
3.3.2 Stimulus/Response Sequences	12
3.3.3 Functional Requirements	13
3.4 System Feature 4 - Summarization (Optional/Configurable)	13
3.4.1 Description and Priority	13
3.5 System Feature 5 - Caching & Resilience	13
3.5.1 Description and Priority	13
3.5.2 Stimulus/Response Sequences	13
3.5.3 Functional Requirements	13
4. External Interface Requirements	14
4.1 User Interfaces	14
4.1.1 UI Principles & Compliance	14
4.1.2 Core Screens	14
4.1.3 Interaction & Usability	15
4.1.4 Context Diagram	15
4.1.5 User Interface Design	15
4.2 Hardware Interfaces	16
4.3 Software Interfaces	16
4.3.1 Internal Components	16

4.3.2 External APIs & Data Sources	16
4.3.3 Data Contracts	16
4.4 Communications Interfaces	17
4.4.1 Transport & Formats	17
4.4.2 Authentication & Authorization	17
4.4.3 Reliability, Errors, and Rate-Limiting	17
4.4.4 Performance Targets	17
4.4.5 Security & Privacy	17
4.4.6 Observability	18
4.4.7 Traceability Considerations	18
5. Non Functional	
5.1 Performance Requirements	18
5.2 Safety Requirements	19
5.3 Security Requirements	19
5.4 Software Quality Attributes	19
Other Requirements	20

Revision History

Name	Date	Reason For Changes	Version
Eric	28/08/25	Added ideas in bullet points as a draft for sections 1to 4	1.0
Anirudh	28/08/25	Added bullet points as draft for all sections	1.0
Amr	28/08/25	Added bullet points as draft in sections 1 and 2	1.0
Khawaja	29/08/25	Added bullets in all sections for draft	1.0
Yousef	29/08/25	Added draft bullet points throughout the document	1.0
Eric	30/08/25	Added a sequence diagram in section 2.1	1.0
Eric	30/08/25	Added a data flow diagram in section 2.2	1.0
Yousef	30/08/25	Created UI designs screens	1.0
Khawaja	30/08/25	Added 4 functional requirements	1.0
Edward	30/08/25	Add some bulletpoint	1.0
Anirudh	30/08/25	Completed section 1.1 of the report	1.0
Edward	1/09/25	Do the 5.3 section	1.0
Anirudh	1/09/25	Completed section 1.2 of the report	1.0
Anirudh	1/09/25	Completed section 1.3 of the report	1.0
Khawaja	01/09/25	Added context diagram and UI designs	1.0
Eric	02/09/25	Finalised and refined section 2,1, 2.2 and 2.3	1.0
Khawaja	02/09/25	Finalised my assigned sections	1.0

Yousef	02/09/25	Completed section 5.1 and 5.2	1.0
Anirudh	02/09/25	Completed section 1.4 of the report	1.0
Edward	02/09/25	Do the 5.4 and 6 section	1.0
Amr	02/09/25	Completed sections 2.4 and 2.5	1.0
Edward	04/09/25	Fix the formatting of my section	1.0
Yousef	04/09/25	Revised the overall document before submission	1.0
Anirudh	04/09/25	Incorporated sponsor feedbacks in the report	1.0
Anirudh	04/09/25	Formatted section 1 for submission	1.0
Amr	04/09/25	Completed Section 2.6 and 2.7	1.0
Amr	04/09/25	Adjusted Section 2.4, 2.5, 2.6, and 2.7 to account for sponsor feedback	1.0
Anirudh	1/10/25	Amended section 1 to include new requirements	2.0
Eric	2/10/25	Reformatted section 2.1 to bullet points structure	2.0
Eric	2/10/25	Resized diagrams for better visibility	2.0
Anirudh	2/10/25	Updated the system architecture in section 2	2.0
Eric	2/10/25	Appended a new product feature as per new requirements in section 2.2	2.0
Khawaja	2/10/25	Added new UI pictures in section 4.1.5	2.0
Anirudh	23/10/25	Updated the project scope	3.0
Khawaja	23/10/25	Updated section 4.1.2	3.0
Khawaja	23/10/25	Updated the context diagram in section 4.1.4	3.0
Khawaja	23/10/25	Updated section 4.3.1	3.0
Eric	25/10/25	Updated section 2.1 as per new requirements	3.0
Eric	25/10/25	Updated section 2.2 as per new requirements	3.0
Eric	25/10/25	Updated diagrams and descriptions based on the latest development	3.0
yousef	25/10/25	Updated UI design	3.0

1. Introduction

1.1 Purpose

This software requirements specification document(SRS) defines the requirements for the development of an AI-powered research application, sponsored by Robotic Marketer Pty Ltd. This document specifies the requirements for version 3.0 (MVP release).

The product proposed by Robotic Marketer is designed to assist users in retrieving, filtering and ranking articles from multiple online sources. The sources range from platforms like LinkedIn

and reddit to news sites and forums. This will be through the integration of third party APIs while keeping intellectual property rights and other licensing concerns in mind. The articles thus retrieved will be filtered and ranked by implementing ML techniques. By automating search and relevance scoring, the application addresses inefficiencies in manual research processes and provides faster access to high-value insights.

The system should also have capabilities to analyze the retrieved article to identify if its sourced from a pre-identified industry rival or a competitor who was not yet identified. The articles should also have a tiny description of how the information can be used or uploaded to benefit the user. This will help the product to stand out in the area.

The scope of this SRS covers the following core features for Version 2.0:

- Automated retrieval of articles from approved sources via APIs.
- Relevance scoring and ranking of search results using ML algorithms.
- Analyze the source and classify it as competitors or not competitors.
- Provide recommendations on how to use the article information.
- Be able to access history logs.
- Summarization of retrieved articles for users to review the articles.
- A web-based interface that enables users to search, filter, and view ranked results.

This SRS applies to the full system and not a subsystem. It is intended to guide developers, project managers, testers, and stakeholders in aligning on system scope, functionality, and constraints for the initial release.

1.2 Document Conventions

This document follows the IEEE SRS standard format and uses the following conventions for clarity and consistency:

- Numbering - Sections and subsections are numbered hierarchically (e.g., 1.1, 1.2, 2.1.1) for ease of reference.
- Requirement Labels - Each requirement is assigned a unique label (e.g., REQ-1, REQ-2) to support traceability during development and testing.
- Priority Keywords - The following terms are used consistently:
 - Must - Mandatory requirement
 - Should - Recommended but not required
 - May - Optional
- Formatting Rules -
 - **Bold** text is used for requirement identifiers and key terms.
 - *Italics* are used for priority labels (*must*, *should*, *may*).
 - Merriweather font is used for code snippets or API references.
- Data and User References - References to requirements, data elements or figures use consistent labels (e.g., D1 = Search, D2 = Articles, D3 = Scores).
- Cross References - Internal references use section numbers and requirement IDs, while external references are listed in Section 1.5 (References).

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

- Project Sponsor (Robotic Marketer Pty Ltd) : Project Sponsors confirm that the system requirements align with business objectives and expected outcomes.
- Project Manager : To plan development activities, allocate resources, and track progress against defined requirements.
- Developers : To design and implement system functionality in accordance with the specified requirements.
- Testing and Quality Leads : To derive test cases and ensure that functional and non-functional requirements are met.
- End Users : To understand the core functionality of the system and validate whether it meets their needs.
- Sponsor's technical staff: To evaluate system compatibility with hosting environments, databases, and security policies.

Here is how the document is intended to be read:

- Stakeholders seeking a high-level overview should review Section 1 (Introduction) and Section 2 (Overall Description).
- Developers and technical teams should focus on Section 3 (System Features) and Section 4 (Interfaces).
- Testers should prioritize Section 3 (Functional Requirements) and Section 5 (Quality Attributes).
- Project managers and sponsors may benefit from reviewing all sections, with emphasis on scope, assumptions, and dependencies.

1.4 Project Scope

Research analysts and knowledge workers spend significant time manually gathering, filtering, and ranking information. This inefficiency leads to lost productivity, inconsistent results, and slower decision-making. Robotics Marketer Pty Ltd is developing this AI-powered research application to automate these processes and deliver faster, more reliable insights. The key benefits of this project are:

- Efficiency: Reduce time spent on manual searching and filtering by automating relevance scoring and summarization.
- Consistency: Provide standardized ranking of sources, removing variability in human judgment.
- Scalability: Enable research processes to expand across industries, topics, and geographies without proportional increases in effort.
- Innovation: Position Robotics Marketer as a leader in AI-driven productivity tools, strengthening its competitive advantage.

Here is how this benefits for Businesses and Users:

- Businesses: Gain measurable productivity improvements, cost savings from reduced research time, and better decision-making through access to timely insights.
- Users: Save hours of daily effort, access relevant information quickly, and focus more on analysis rather than information gathering.

In Version 3.0, the MVP release, we have the following points in scope:

- Retrieval of articles from approved sources via APIs (e.g., Reddit, X, LinkedIn).

- Relevance scoring and ranking of results using ML.
- Summarization of selected articles.
- Web-based interface with search, filter, and browsing features.
- Secure data handling and compliance with privacy regulations.
- Backlinks with urls to access posts with highest engagements easily.
- History page to access the history of a user.
- Filter option to filter out the result according to user preference.
- Idea card to display ideas tailored to a user.
- Competitor analysis for a user.
- Trending topic analysis relevant to a user.
- News feed with the latest news relevant to the user.

1.5 References

- *IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, 1998.*
- *WCAG 2.1 AA Accessibility Guidelines W3C Recommendation, 06 May 2025.*
<https://www.w3.org/TR/WCAG21/>
- *Django REST Framework Documentation. Version 3.15, Django REST Framework Documentation. Version 3.15, <https://www.django-rest-framework.org/>*
- *GDPR Regulations (EU) <https://gdpr-info.eu/>*
- *Australian Privacy Act 1988 (Cth)*

2. Overall Description

2.1 Product Perspective

Project Context

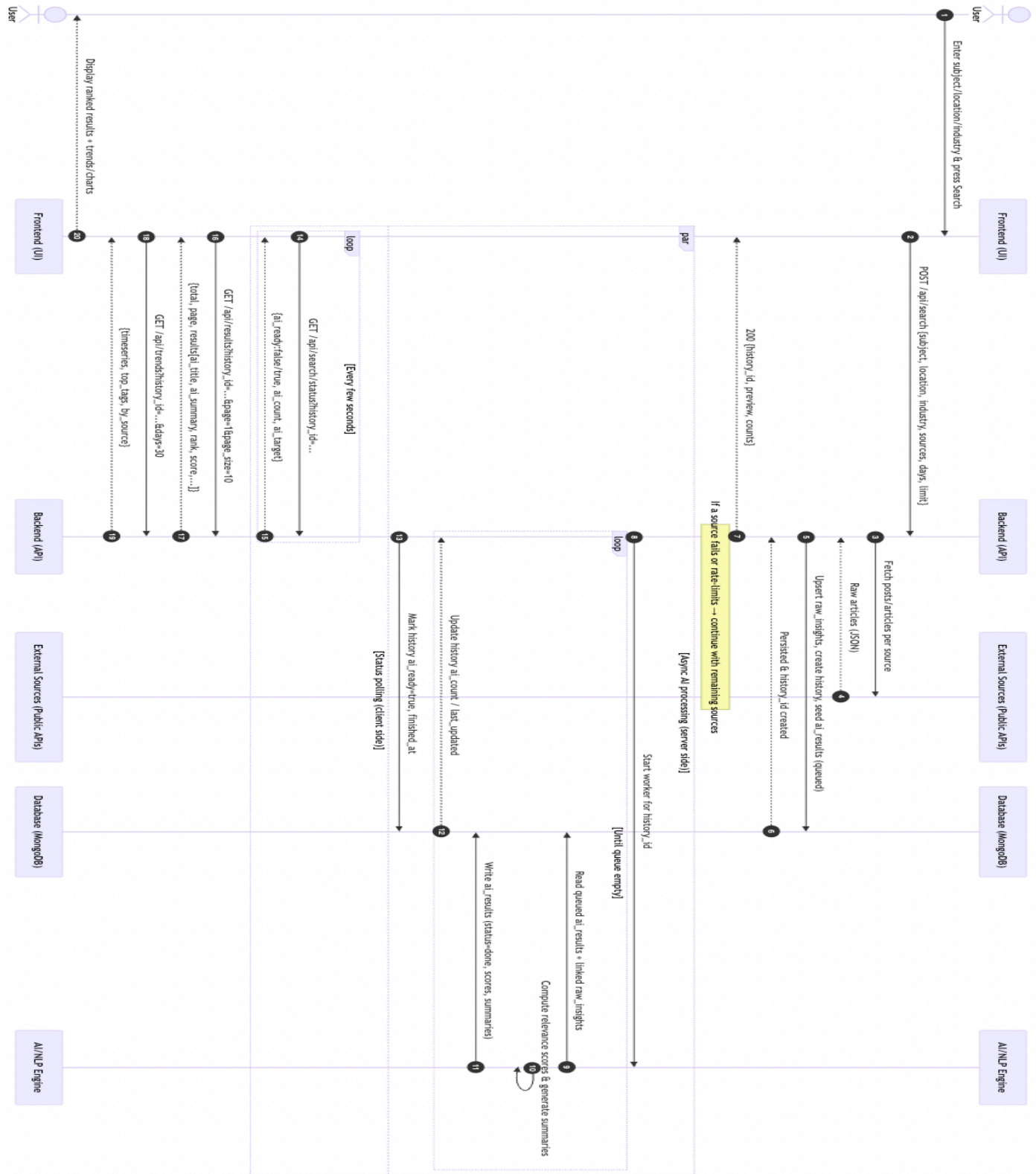
- This product is developed as part of the PACE collaboration with Robotic Marketer.
- It is not a standalone tool but a new module integrated into the Robotic Marketer dashboard.
- The goal is to enhance market intelligence and social listening by automatically retrieving, ranking and summarising content from multiple platforms.
- It complements existing dashboard modules (analytics and reporting) instead of replacing them.

System Architecture

- The system follows a three-layer architecture:
 - Frontend Interface: Web-based UI for keyword search and displaying ranked results.
 - Backend API (Django REST): Handles requests, performs multi-source retrieval and connects to external APIs.
 - NLP/ML Engine: Executes relevance scoring, semantic similarity checks and automated summarisation. It also performs analysis on the text source and content to identify competitors and potential use cases of the content.
- Centralised Database: Stores fetched articles and metadata (title, URL, source, engagement metrics) for efficient querying, ranking, caching and re-use.

External Dependencies

- Relies on APIs from platforms such as Reddit and News Feeds.
- Requires compliance with platform usage policies and rate-limit restrictions.



Sequence Diagram 1: Data flow from user search to results.

System Workflow (see Sequence Diagram 1)

- The user enters a subject, location or industry through the frontend search interface.
- The backend API receives the request and queries multiple external sources (Reddit, news APIs) for relevant articles.
- Retrieved data is cleaned, deduplicated and stored in MongoDB collections (raw_insights, history) for tracking and persistence.
- The backend triggers an asynchronous AI process, which:
 - Retrieves stored data from the database.
 - Performs ranking and summarisation using NLP algorithms.
 - Saves processed results into the ai_results collection.
- The frontend polls the backend periodically to update the user on AI progress and display results once ready.
- Users can explore dashboard insights showing trending topics, tags, influencers and source statistics.
- If any external API fails or is rate-limited, the system continues processing with available sources to maintain consistent output.

2.2 Product Features

This product is a feature within the Robotic Marketer dashboard that provides intelligent article retrieval and analysis. These are the main features the system will offer:

- **Keyword search interface**
Users can enter a keyword into the search field to begin their query. This acts as the starting point of the system which allows users to access multiple connected data sources with a single input.
- **Multi-source article retrieval**
The system collects articles and posts from multiple platforms such as Reddit and online news sites. By pulling data from different sources, users receive a more diverse perspective rather than relying on a single platform.
- **Relevance ranking**
Articles are automatically ranked using a hybrid scoring model that considers lexical similarity, semantic similarity and engagement signals. This ensures the most relevant and useful results are prioritised at the top of the list.
- **AI-powered summarisation**
Each retrieved article is paired with a short AI-generated summary. These summaries give users a quick overview of the content which helps them to decide at glance whether an article is worth reading in full.
- **Resilience to API failures**
If one or more external data sources encounter errors or rate limits, the system continues to return results from the remaining sources. This feature ensures consistent access to information without disruption.

- **Dashboard and analytics**

The system includes a dashboard view within the UI where users can track trending topics, competitor mentions, history logs, influencers, curated news feeds and suggested ideas. This feature provides a broader market intelligence layer beyond individual searches which enables users to monitor ongoing trends and insights over time.

The data flow diagram figure 2 below illustrates how each major feature of the system works together to deliver results to the user.

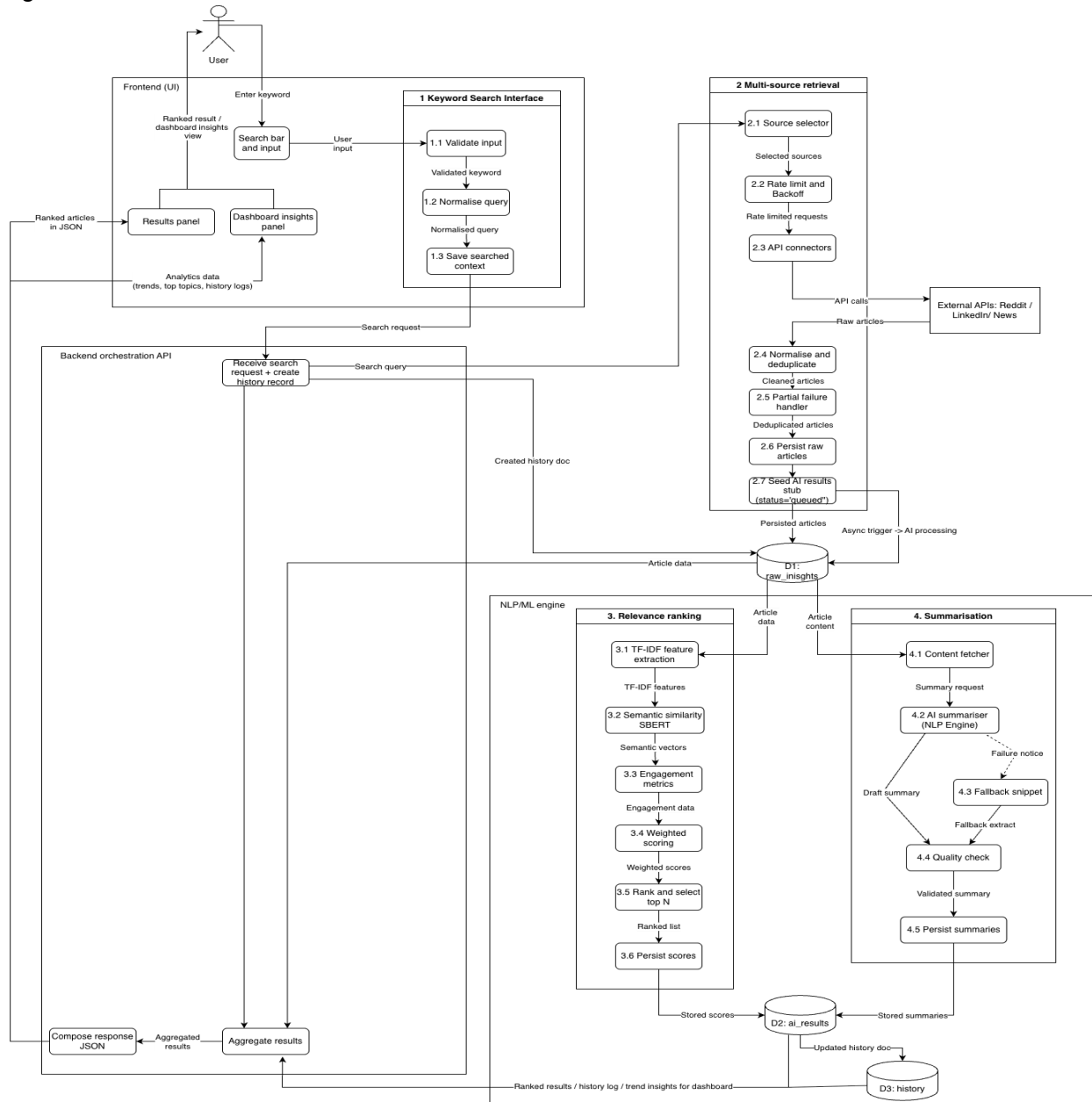


Figure 2: The relationship of each major feature.

- The user enters a subject, location or industry through the **frontend search interface**.

- The **backend API** creates a history record, fetches data from multiple sources (Reddit, LinkedIn, News) and stores cleaned results in **raw_insights**.
- An **AI process asynchronously** performs ranking (TF-IDF, semantic similarity, engagement metrics) and generates summaries stored in **ai_results**.
- The summarised and ranked outputs are linked to the user's search **history** for tracking and progress updates.
- The **frontend** displays ranked articles, trends and insights retrieved from the backend and database.

2.3 User Classes and Characteristics

The system is designed to support a range of user groups with different needs, levels of expertise and usage frequency. The following categories outline the primary, secondary and less critical user classes, along with their key characteristics and requirements.

Primary user classes (High priority - Favoured)

- Research analyst/ Market analyst
 - Frequency of use: Daily.
 - Technical expertise: Moderate to high, but prefers efficiency over technical detail.
 - Characteristics: Require accurate ranking, summarisation and freshness of data.
 - Requirement: The system must emphasise speed, clarity and reliability.
- Academic researchers
 - Frequency of use: Frequent (for literature reviews, case studies, coursework).
 - Technical expertise: Mixed, ranging from novice to advanced.
 - Characteristics: Need simple, intuitive UI to accommodate varying skills.
 - Requirement: Fast access to relevant and summarised content to reduce research time.
- Business decision makers / Sponsors
 - Frequency of use: Occasional.
 - Technical expertise: Low to high, may vary.
 - Characteristics: Expect polished presentation, reliability and minimal training curve
 - Requirement: Prioritise clarity, concise summaries and professional layout over technical depth.

Secondary user classes (Mid priority - Important)

- Data scientists / AI engineers
 - Frequency of use: Occasional to moderate.
 - Technical expertise: High.
 - Characteristics: Seek transparency in scoring models, fairness metrics and ranking logic.
 - Requirement: Explanations of weighting and relevance scoring to validate outputs.
- Knowledge workers (journalists, consultants)
 - Frequency of use: Occasional.
 - Technical expertise: Moderate.
 - Characteristics: Use tools for quick topic exploration and insights.

- *Requirement: Need fast, simple results without heavy configuration.*

Less critical user classes (Low priority - Supplementary)

- Casual users (general public, content creator, bloggers)
 - *Frequency of use: Rare, exploratory.*
 - *Technical expertise: Low.*
 - *Characteristics: Curious, non-professional usage.*
 - *Requirement: Lightweight interface with minimal learning curve.*

2.4 Operating Environment

The system will be deployed as a cloud-hosted web application, as a part of Robotic Marketer's existing platform. It will operate across all standard client devices and modern browsers, while the backend will utilise the latest versions of various technologies, to ensure compatibility with Robotic Marketer's existing systems. The following outlines the specific components of the operating system:

- **Client-Side:**
 - Accessible via any modern browser, such as Chrome, Firefox, or Edge.
 - Must support mobile browsers, with a fully responsive interface.
- **Server-Side:**
 - Developed in Python 3.12 or higher.
 - Uses the Django 5.2+ REST Framework for API development.
 - Hosted on Robotic Marketer's cloud architecture.
- **Database:**
 - Uses MongoDB Atlas, version 8 or higher, at the M10+ tier with TLS.
 - Region of the database will be selected based on the location of the majority of users, to minimise latency.

2.5 Design and Implementation Constraints

The design and implementation of the system is subject to several constraints that guide the team's technology choices, in order to ensure compliance with legal and ethical standards, and sponsor requirements. These constraints determine the boundaries within which the project must be developed, helping maintain consistency and long-term sustainability. The following outlines the key limitations and requirements:

- **Accessibility Compliance:** The application must comply with WCAG 2.1 AA accessibility standards.
- **Technology Used:** Development must use Python 3.12+ and the Django 5.2+ REST Framework.
- **Data Privacy:** Must comply with GDPR and Australian Privacy Act. This can potentially limit what data can be stored.
- **API Usage:** Only publicly available datasets or APIs can be used. Proprietary or paid APIs are excluded unless the sponsor provides access.
- **LLM Usage:** No paid LLMs to be used for inference unless approved by the sponsor. Summarisation tasks must default to extraction/compactive methods using small local models.

- **Open-Source Licenses:** Any library or pretrained model used must be on an open-source license which allows for commercial use, such as the Apache 2.0 or MIT license.

2.6 User Documentation

Clear and accessible documentation will be delivered along with the software to support end users in effectively utilising the application and all its features. The documentation will include both written guides and embedded help features to ensure users have multiple ways to access the information. The following documentation will be used:

- **User Guide:** A concise guide available in PDF and HTML formats, covering:
 - How to search and filter for specific industries/locations
 - How to interpret the scores used
 - Known system limitations
- **Contextual Tooltips:** The application should contain inline tooltips for the following:
 - How the relevance score is calculated
 - What the engagement numbers mean
 - What semantic similarity is, and how it's calculated
- **FAQ:** The application should contain a FAQ section covering the most common errors, and how to address them.

2.7 Assumptions and Dependencies

The successful development of the system relies on several assumptions about resources, external services, and team member participation. These factors represent the conditions outside the direct control of the project team. Any changes to these assumptions may impact the scope or schedule of the project. The following are the key assumptions being made:

- **Availability of Public Resources:** The public APIs and datasets required for the system will remain available for the entire duration of the project.
- **Stability of Development Platforms:** Services used for the development such as GitHub, MongoDB Atlas, and Fly.io will remain stable and accessible.
- **Policy Stability:** No major changes to the API ToS, or data privacy regulations, such as the Australian Privacy Act, will occur during the development of the project.
- **External API Usage Limits:** APIs used, such as for Reddit and LinkedIn, will continue to support the needed query volumes within their free tier limits.
- **Team Availability:** Project team members will be consistently available to attend meetings and complete assigned tasks before the required deadlines.
- **Technical Resources:** Team devices will meet the baseline requirements required to run the needed NLP tasks, such as TF-IDF locally during development and testing.
- **Sponsor Feedback:** Sponsor feedback will be provided within 3 business days of a demonstration, to avoid blocking sprint progression.
- **Branding Assets:** Sponsor will supply the necessary branding assets, such as logos, to ensure the application's user interface is visually consistent with the rest of Robotic Marketer's platform.

3. System Features

3.1 System Feature 1 - Article retrieval

3.1.1 Description and Priority

- *This feature provides the ability to fetch articles and posts from multiple online platforms, including Reddit and LinkedIn. The retrieved content forms the foundation of the system, which enables further processing such as ranking and summarisation.*
- *Priority: High, article retrieval is the core functionality of the system and is essential for the minimum viable product (MVP).*

3.1.2 Stimulus/Response Sequences

- A user enters a keyword into the search bar.
- The system validates the input and queries connected external APIs.
- Retrieved results are consolidated and displayed to the user through the frontend.
- If one API fails, results from the remaining sources are still returned.
- If API rate limits are exceeded, the system notifies the user but continues to display partial results from available sources.

3.1.3 Functional Requirements

REQ-1.1: *The search bar must display a default placeholder "Keyword."*

REQ-1.2: *If one API fails, results from other APIs must still appear.*

REQ-1.3: *When API limits are exceeded, display a clear message identifying the missing source.*

REQ-1.4: *Empty or invalid queries must return an error message, not crash the system.*

3.2 System Feature 2 - Relevance Scoring & Ranking

3.2.1 Description and Priority

- *Hybrid scoring that combines lexical (TF-IDF), semantic (SBERT-MiniLM), and engagement (likes/comments) into a single normalized relevance score. **Priority: High.***

3.2.2 Stimulus/Response Sequences

- *User submits a query → system computes per-document sub-scores → weighted fusion → returns ranked list with score breakdown per item.*
- *User adjusts filters (date range, domain, industry) → list re-ranks without a full refetch when possible (use cached vectors).*

3.2.3 Functional Requirements

- **REQ-2.1:** *The system shall compute TF-IDF cosine similarity between query and document text.*

- **REQ-2.2:** The system shall compute sentence-embedding cosine similarity using a small SBERT model (e.g., *MiniLM*).
 - **REQ-2.3:** The system shall ingest platform engagement signals (counts if available) and scale them to [0,1] before fusion.
 - **REQ-2.4:** The UI shall display a hoverable breakdown of the score components (lex/sem/eng).
 - **REQ-2.5:** The system shall return ranked results within **≤1.5s** median for a page of 20 items on staging data volume.
-

3.3 System Feature 3 - Filtering, Disambiguation & Facets - elaborate

3.3.1 Description and Priority

- Disambiguate ambiguous queries (e.g., “Apple”, “Jaguar”) via lightweight clarifying facets (industry, location, content type). **Priority: High.**

3.3.2 Stimulus/Response Sequences

- User searches ambiguous term → system suggests facet chips (e.g., “Technology”, “Automotive”) → user selects → results re-ranked.

3.3.3 Functional Requirements

- **REQ-3.1:** The system shall propose at least 2 facet suggestions when the query matches known ambiguous entities.
- **REQ-3.2:** The system shall allow filtering by date range, domain allow-list, industry, and language.
- **REQ-3.3:** The system shall persist the selected filters in the URL (query string) for shareable searches.

3.4 System Feature 4 - Summarization (Optional/Configurable)

3.4.1 Description and Priority: Provide short extractive summaries for quick scanning; optional and toggleable to control cost/performance. **Priority: Medium.**

3.4.2 Stimulus/Response Sequences

- *User opens result card → system displays a 2–3 sentence summary and highlighted matched phrases.*

3.4.3 Functional Requirements

- **REQ-4.1:** *The system shall generate a summary ≤ 400 characters using extractive methods by default.*
 - **REQ-4.2:** *Summaries must not include PII scraped from comments unless the source explicitly allows it; if present, redact.*
 - **REQ-4.3:** *Summarization can be disabled via an environment variable without a code change.*
-

3.5 System Feature 5 - Caching & Resilience

3.5.1 Description and Priority

- *Reduce latency and protect against rate limits by caching queries/results and degrading gracefully on upstream failure. **Priority: High.***

3.5.2 Stimulus/Response Sequences

- *Repeat query within cache TTL → serve cached results; stale-while-revalidate refresh runs asynchronously.*

3.5.3 Functional Requirements

- **REQ-5.1:** *The system shall cache normalized queries and their top-K results for at least 30 minutes (configurable).*
- **REQ-5.2:** *On third-party API outage, the system shall return partial results with a source status banner*
- **REQ-5.3:** *All outbound calls shall implement exponential backoff and jitter.*

4. External Interface Requirements

This section specifies all interfaces between the AI-powered web application and its environment, including user-facing UI, supported hardware, software dependencies, and communication/transport protocols. All requirements are written to be **clear, testable, and verifiable**.

4.1 User Interfaces

4.1.1 UI Principles & Compliance

- The system shall comply with **WCAG 2.1 AA** (keyboard operability, focus order, color contrast $\geq 4.5:1$, alt text, ARIA where applicable).
- The system shall implement **responsive design** for desktop ($\geq 1280\text{px}$), tablet ($\geq 768\text{px}$), and mobile ($\leq 480\text{px}$) breakpoints.
- The system shall follow sponsor brand guidelines for typography, color palette, and logo placement.

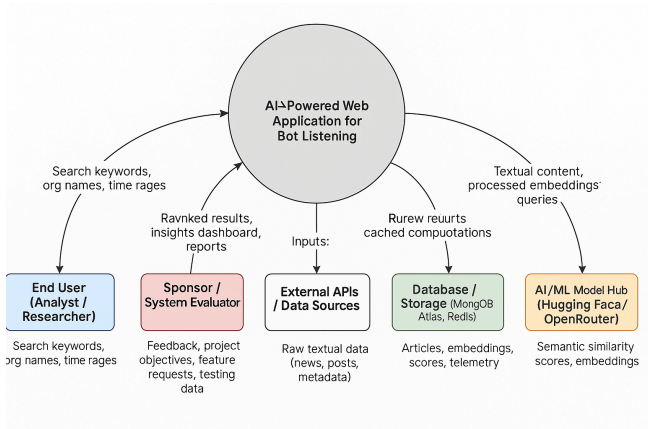
4.1.2 Core Screens

- **Home/Search**
 - The system shall display a keyword search input with placeholder text “Search topics, companies or keywords...”.
 - The system shall provide optional filters (date range, source, language, location/industry if available).
 - The system shall provide a “Search” action and show a loading indicator during retrieval.
- **Results Listing**
 - The system shall display ranked results with: title (clickable link), source, timestamp, snippet/summary, **lexical/semantic/engagement scores** (or a combined score), and badges for data quality (e.g., “Low engagement”).
 - The system shall provide sorting by **Relevance (default)**, **Newest**, and **Engagement**; and filtering by source.
 - The system shall support pagination or infinite scroll with deterministic ordering.
- **Detail View**
 - The system shall show full metadata, cached summary, and outbound link.
 - The system shall display an audit note when a source is unavailable/rate-limited.
- **Saved Items**
 - The system shall allow saving/unsaving results for later review (if in scope).
- **Global Elements**
 - The system shall provide a persistent header (Home, Saved, Settings, Help/Feedback).
 - The system shall present consistent, human-readable error messages (e.g., “LinkedIn data temporarily unavailable; showing results from other sources”).

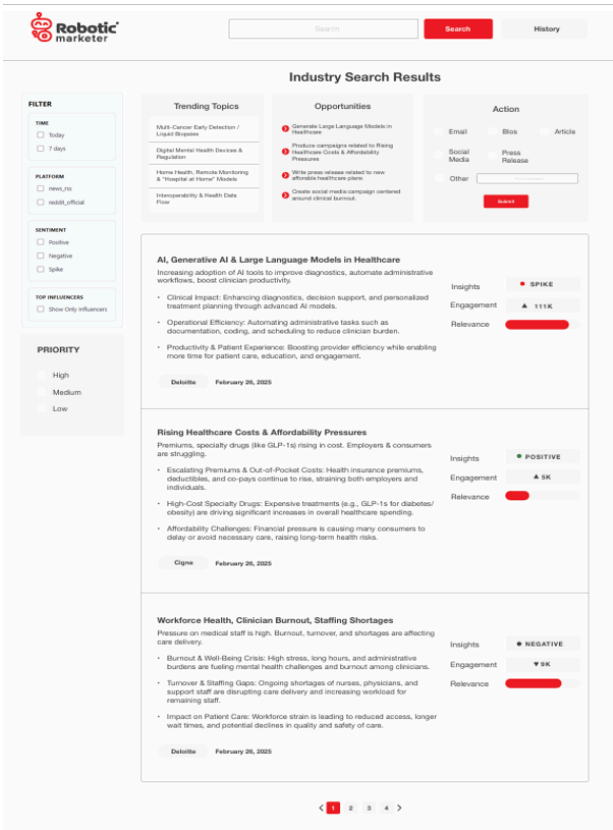
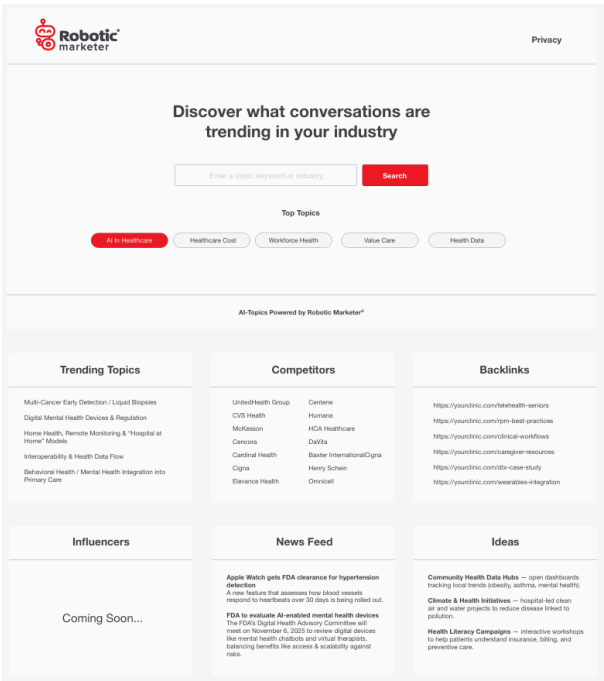
4.1.3 Interaction & Usability

- The system shall debounce keystrokes ($\geq 300\text{ms}$) to avoid excessive API calls for suggest/autocomplete (if implemented).
- The system shall persist user filters during navigation within a session.
- The system shall support i18n/l10n hooks for UI strings (future-ready).

4.1.4 Context Diagram



4.1.5 User Interface Design



Search History		
Date Searched	Topic	Engagement
February 26, 2025	AI, Generative AI & Large Language Models in Healthcare	▲ 111K
February 26, 2025		▼ 55K
February 26, 2025	Workforce Health, Clinician Burnout, Staffing Shortages	▲ 576K
February 26, 2025	Value-based Care, Digital Quality, and Consumer-Centered	▲ 176K
February 26, 2025	Health Data, Interoperability & Privacy/Security	▲ 876K

AI-Topics Powered by Robotic Marketer®

4.2 Hardware Interfaces

Client Devices

- *The system shall support modern browsers (latest two versions of Chrome, Edge, Safari, Firefox) on desktop and mobile devices with stable internet connectivity.*

Server/Compute

- *The system shall run on cloud infrastructure supporting autoscaling for API and NLP workloads.*
- *The system shall provide **≥10 GB** storage for model artifacts and cached results (expandable).*
- *(Optional) If a GPU is provisioned, the system should utilize it for transformer inference to reduce latency.*

4.3 Software Interfaces

4.3.1 Internal Components

- **Frontend:** HTML5/CSS3/JavaScript (framework optional), responsive UI.
- **Backend:** Python 3.12+, **Django REST Framework** for RESTful APIs.
- **NLP/ML:**
 - **Frameworks:** scikit-learn, PyTorch, and sentence-transformers (SBERT MiniLM).
 - **Lexical Scoring:** Implements BM25 (using `rank_bm25`) to supplement TF-IDF for more robust keyword relevance.
 - **Semantic Similarity:** Uses SBERT MiniLM for contextual embeddings and sentence-level semantic matching.
 - **Hybrid Pipeline:** Combines lexical, semantic, and engagement-based scoring to produce a composite relevance score.

- **LLM Integration:** Supports optional Large Language Model scoring via Hugging Face and OpenRouter APIs for enhanced context understanding when permitted by API quotas.
 - **Extensibility:** The pipeline is pluggable, future models or ranking algorithms (e.g., re-rankers or neural combiners) can be added with minimal refactoring.
- **Database/Cache:**
 - Stores articles, metadata, semantic embeddings, and ranking scores.
 - Includes telemetry collections for performance monitoring and usage analytics.
 - Uses unique indexes and optimized schemas for fast document retrieval.
 - Optional Redis for caching requests/results (if needed for performance).
- **Version Control/CI:** GitHub with protected branches and CI for lint/tests.

4.3.2 External APIs & Data Sources

- The system shall integrate with approved public APIs (e.g., Reddit, news APIs, others permitted by sponsor) and **shall not** use restricted/paid LLM APIs unless explicitly approved.
- The system shall store API credentials only in secure environment variables/secret manager (never client-side).
- The system shall honor each provider's **ToS, licensing, and rate limits**. If a provider changes terms or limits, the system shall degrade gracefully (Section 4.4).

4.3.3 Data Contracts

- The backend shall expose REST endpoints that return **JSON** with stable, versioned schemas, e.g.:

```
{
  "query": "renewable energy",
  "results": [{
    "id": "src:reddit:abc123",
    "title": "...",
    "url": "...",
    "source": "reddit",
    "published_at": "2025-08-10T12:05:00Z",
    "summary": "...",
    "scores": {"lexical": 0.62, "semantic": 0.79, "engagement": 0.43, "composite": 0.71}
  }],
  "meta": {"page": 1, "page_size": 20, "total": 124}
}
```

Backwards-incompatible API changes shall be introduced under a new versioned path (e.g., /api/v2/...).

4.4 Communications Interfaces

4.4.1 Transport & Formats

- All client-server communications shall use **HTTPS** with **TLS 1.2+**.
- The system shall use **REST** over HTTP(S) with **JSON** payloads (UTF-8).
- The system shall set appropriate cache headers for static assets and disable caching for sensitive API responses.

4.4.2 Authentication & Authorization

- External APIs: The system shall use API keys or OAuth2 as required by each provider.
- Internal APIs: If authentication is later required for users, the system shall support token-based auth (e.g., JWT) with expiration and refresh flows.

4.4.3 Reliability, Errors, and Rate-Limiting

- The system shall implement **timeouts** (default 10s) and **exponential backoff** retries (e.g., 2, 4, 8 seconds; max 3 attempts) for outbound API calls.
- On partial failures (e.g., one provider down), the system shall:
 - Return available results from other sources,
 - Include a machine-readable warning in response metadata,
 - Surface a human-readable notice in the UI.
- The system shall implement **circuit breaker** semantics to avoid hammering failing providers.
- The system shall log all outbound request status codes, durations, and failures for observability (PII-safe).

4.4.4 Performance Targets

- P50 search response $\leq 2.5s$, P95 $\leq 5s$ (assuming at least one provider available and cache warm).
- API throughput target: ≥ 20 requests/sec burst without degradation for MVP scope (tunable with infra).
- Maximum payload for a single page of results: ≤ 512 KB compressed.

4.4.5 Security & Privacy

- The system shall never expose secrets in client code or responses.
- The system shall sanitize all inbound parameters to prevent injection, and escape content from external sources to prevent XSS.
- The system shall process and store only **publicly available** data unless otherwise approved, and shall comply with **Australian Privacy Act 1988** and applicable regulations (GDPR/CCPA where relevant to use).
- The system shall support data retention policies (e.g., auto-purge cached content after N days configurable).

4.4.6 Observability

- The system shall provide structured logs (JSON) with trace IDs for request correlation.
- The system shall expose basic health endpoints (e.g., /healthz, /readiness) for deployment orchestration.
- The system shall capture metrics (request count/latency, error rates, cache hit ratio) for performance tuning.

4.4.7 Traceability Considerations

To ensure consistency from specification through verification, all external interface requirements are traceable to functional and non-functional specifications and to corresponding test artefacts.

Specifically, user-interface requirements (e.g., search, ranking, and summarisation views) are mapped to the relevant System Features to prevent orphaned UI behaviour and to guarantee end-to-end coverage. Software and communications interface definitions (e.g., external APIs, protocols, payload schemas) are cross-referenced to the Non-Functional Requirements for performance, security, privacy, and availability to confirm that interface contracts align with system quality attributes and regulatory constraints.

Each requirement expressed as a “shall” statement is written to be objectively verifiable and is linked to one or more test categories, UI tests (functional and accessibility), API contract tests (schema/semantic validation against providers), and integration tests with stubbed/mockd external services, to enable measurable acceptance criteria and regression safety over time.

5. Performance Requirements

Under typical circumstances, the first page of ranked results on Search should appear. Client-side pagination is used for results, with 10 items per page. A visible loading state must show during the fetching process. If it is successful, the list takes the loader's place. If it is unsuccessful, an inline error appears. When paginating locally, the frontend avoids unnecessary re-fetching by making a single search call for each filter change. For fast back-and-forth navigation, a subject's most recent successful response may be reused. Filters or a new subject initiate a new call. There is a hard timeout (e.g., 20s) for every request. When scrolling, opening links, and paging on standard laptop and mobile networks, the user interface must stay responsive.

5.1 Safety Requirements

The application doesn't scrape or get around paywalls. It just shows outgoing links and uses data from the configured backend API. Links to external websites open in a new tab. To stop injection (such as XSS), inputs and rendered outputs are cleaned up. Any keys stay server-side, and no secrets are revealed client-side. The page displays a clear error without blocking previously loaded content if the API is unreliable or slow. Users are free to try again. If enabled, the saved history is brief, clear, and simple to delete. Network calls adhere to basic rate-limit/backoff behavior and provider terms.

5.2 Security Requirements

HTTPS is used for all production traffic. The frontend origin must be explicitly permitted by CORS. Clean up and get away from user-controlled content; think about using a CSP. Credentials and API keys remain server-side only. End-user authentication is not included in the MVP; if customized features are added later, use Django auth with session protection and salted hashing. Observe privacy regulations that apply to data that is stored; refrain from gathering personally identifiable information in the browser beyond temporary session storage. Follow the OWASP Top 10 mitigations and think about doing a security scan or review before launching.

5.3 Software Quality Attributes

To ensure the application meets user expectations and functions reliably across different environments, several software quality attributes must be addressed. These attributes define

the standards for usability, performance, maintainability, and overall system behavior, providing a foundation for a robust and user-friendly application. The following quality attributes outline the key requirements for the system:

Usability:

- *Clear headings and labels for all UI components.*
- *Visible empty states and error messages to guide users.*
- *Consistent layout across all pages and devices.*

Compatibility/Portability:

- *Application must run seamlessly on major browsers: Chrome, Edge, and Safari.*
- *Must be accessible on laptops, desktops, and mobile devices.*

Reliability:

- *Handle API errors and timeouts gracefully.*
- *UI should never crash; provide retry buttons or fallback messages when data retrieval fails.*

Maintainability:

- *Use semantic HTML and modular CSS for easy updates.*
- *Code should be structured in short, well-defined components for easier modification.*

Performance:

- *The accuracy that its related to the search words must above 50%*
- *The Home-page when the website start must load within 5 seconds not more*
- *The search result must take no longer than 3-5 seconds*

Testability:

- *Unit tests must cover backend AI scoring logic and API interactions.*
- *Integration tests must validate end-to-end data flow from user input to displayed results.*

5.4 Other Requirements

In addition to functional and quality requirements, the system must satisfy several other considerations that support long-term usability, compliance, and flexibility. These requirements cover aspects such as database design, legal obligations, and code reuse, ensuring the application is robust, adaptable, and maintainable over time.

Database Requirements:

- *Use a non-relational database*
- *Showing max 10 article*

Legal Requirements:

- *Ensure only publicly available data is collected. Avoid violating third-party API terms of service.*

- Any agreements signed with partner organizations (IP, NDA) must be adhered to.

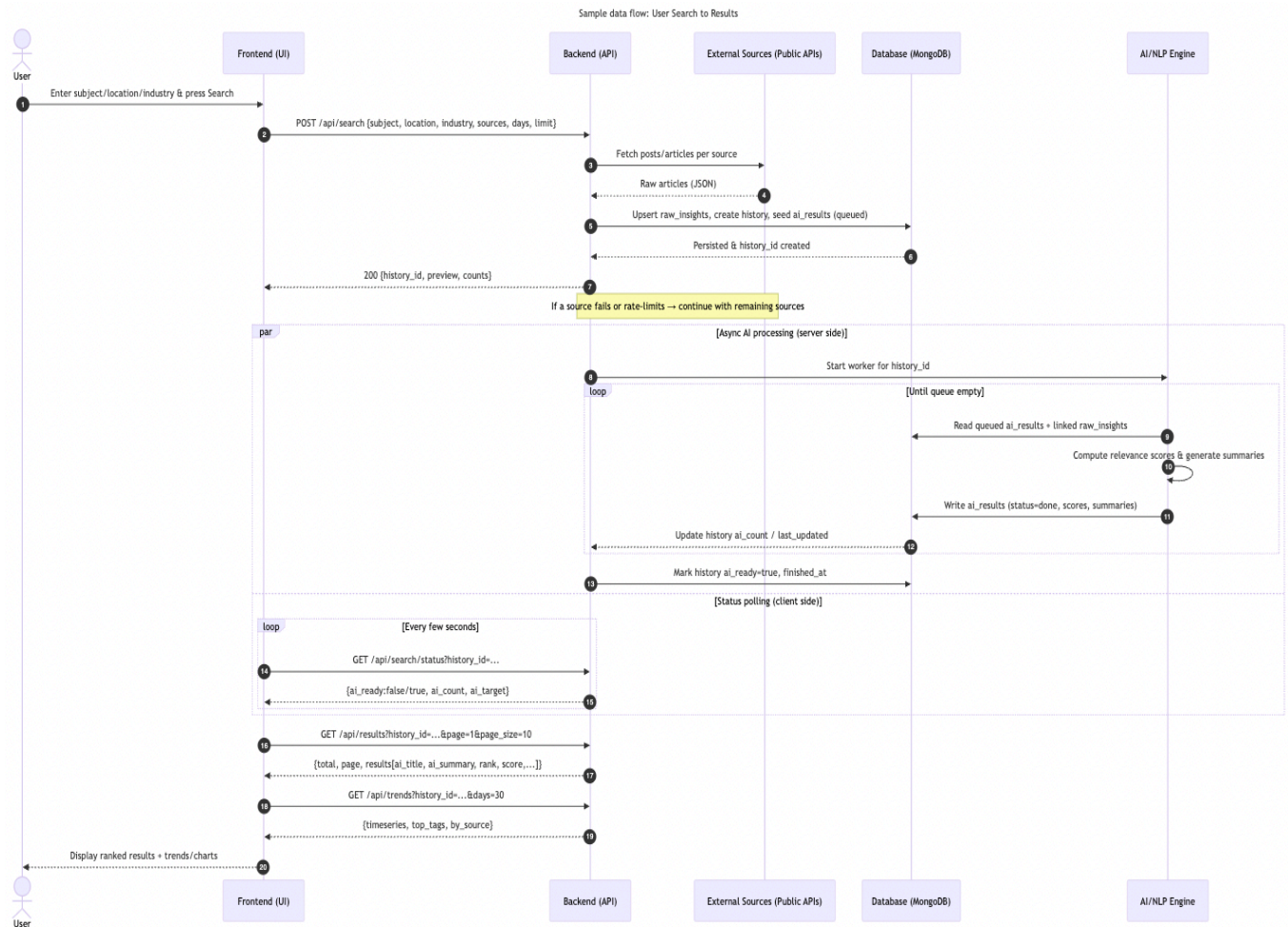
Reuse Objectives:

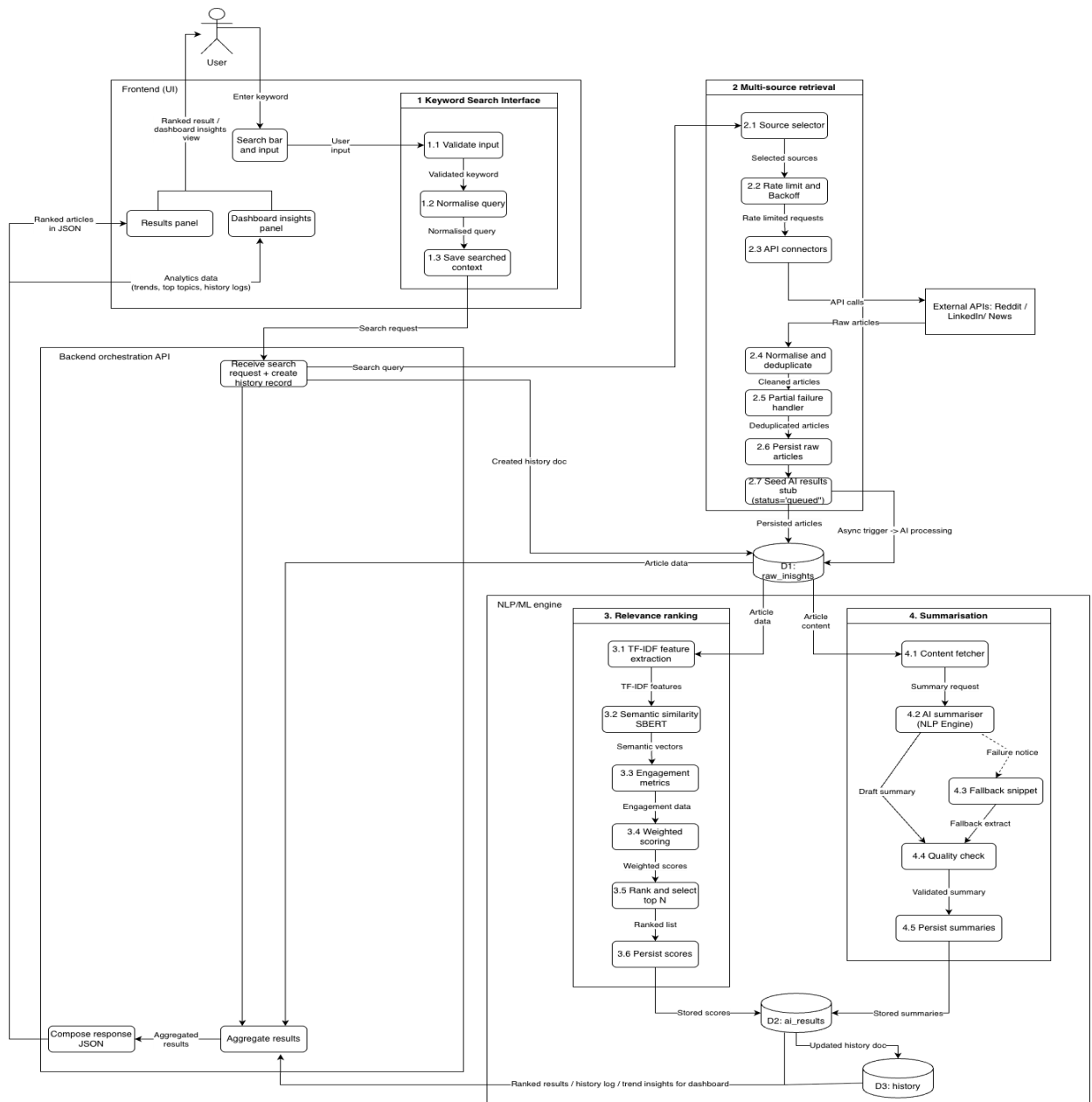
- Codebase should be modular to allow reuse in future AI-powered web applications or data mining tools.

Appendix A: Glossary

- Lexical Score:** Similarity based on word frequency features (e.g., TF-IDF cosine)
- Semantic Score:** Similarity based on sentence embeddings (e.g., SBERT MiniLM).
- Engagement Score:** Normalized metric from platform interactions (likes, comments) when available.
- Facet:** A clarifying filter (industry, location, date) used to disambiguate queries.

Appendix B: Analysis Models





Appendix C: Issues List

Client/Sponsor feedback

Meeting date and time

- Date: Every Thursday
- Time: 5pm, via Microsoft Teams
- Sponsor: Robotic Marketer
- Team: Group 22

Document shared with sponsors:

- 1st of September 2025
- 4th of September 2025

Feedback received (most recent meeting)

- Meeting date/time: Thursday, 04/09/2025, 5pm, via Microsoft Teams
- Attendees: Anirudh, Khawaja, Eric, Yousef, Amr, Edward, Miroslav (Sponsor), Humayun (Sponsor)

Summary of feedback:

- User interface design should include location and industry fields on the search Homepage.
- For every heading, begin with a brief introduction/description before listing details.
- Provide ideas in paragraph format, with bullet points only used as a supplementary.
- Elaborate further on the product features section to add depth and clarity.
- Maximum data to display is 10 articles per search.
- Must get the license if not for commercial use for any external resources like hugging face, etc
- The loading page must load a maximum of 5 minutes, loading page no more than 3-5 minutes, and accuracy above 50%.
- Place the explanation of diagrams after the diagrams rather than before.

Team response/action plan

- **UI adjustments:** We will update the homepage design to include location and industry fields in the main search interface for the next iteration.
- **Section structure:** All headings will begin with a short introductory paragraph in the latest documents and future drafts to improve readability and alignment with expectations.
- **Formatting approach:** We will shift the majority of ideas into paragraph form, reserving bullet points only for supporting or supplementary details.
- **Product features:** The product features section will be expanded with more detail, covering functionality, use cases, and benefits to add depth and clarity.
- **Article limit:** The system will be configured to display a maximum of 10 articles per search to keep results concise and relevant.
- **Licensing compliance:** We will confirm licensing requirements for external tools (e.g., Hugging Face models) and document their usage appropriately to ensure compliance.

- **Performance criteria:** *We will set system benchmarks to ensure results load within 3–5 minutes, with accuracy maintained above 50%. These will be tracked as part of testing and quality assurance.*
- **Diagram placement:** *Diagrams will be presented first, followed by explanatory text, to make the document flow more intuitive and visually clear.*



<AI-Powered Web Application for Social Listening>

Version <02>, <25/10/2025>

Table of contents:

1. Analysis Documentation.....	1
1.1 Use Case Diagram:.....	2
1.2 Use Case Descriptions.....	3
1.3 User Stories.....	4
2. Design Documentation:.....	5
2.1 System Design Document:.....	6
2.2 User Interface Layouts:.....	7
2.3 Report Layouts:.....	8
2.4 Window Navigation Diagram:.....	9
2.5 Data Definitions:.....	10
2.6 Activity Diagrams:.....	11
3. Testing Documentation:.....	12
3.1 Test Plan:.....	13
3.2 Test Case Specifications:.....	14

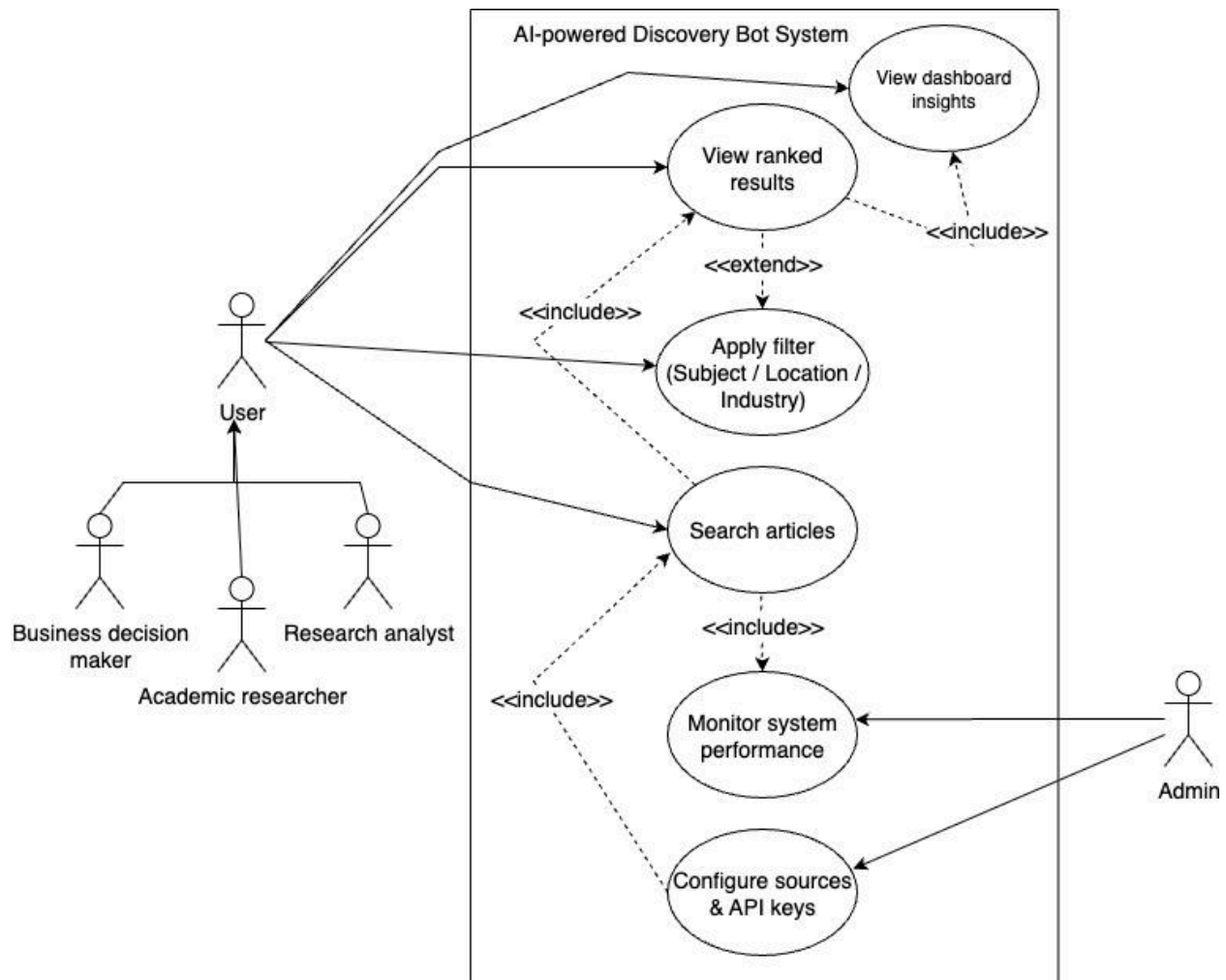
Revision History

Name	Date	Reason For Changes	Version
Anirudh	1/10/25	Added user stories	1.0
Anirudh	1/10/25	Added Data definitions	1.0
Anirudh	2/10/25	Formatted the document to structure it	1.0
Eric	30/09/25	Added use case diagram with a short explanation	1.0
Eric	30/09/25	Completed five use case descriptions in table format	1.0
Eric	30/09/25	Added user stories	1.0

Khawaja	29/09/25	Completed the system design document section 2.1	1.0
Khawaja	29/09/25	Added package diagram, Deployment view and a sequence diagram	1.0
Khawaja	1/10/25	Completed a test plan section 3.1	1.0
Khawaja	2/10/25	Structure a document	1.0
Yousef	1/10/25	Completed section 2.2	1.0
Yousef	1/10/25	Completed section 2.4	1.0
Yousef	2/10/25	Helped with structuring the document	1,0
Edward	30/9/25	Doing report layout	1.0
Edward	1/10/25	Make Test specification table	1.0
Edward	2/10/25	Doing the testing, and fill out some test cases based on the output	1.0
Amr	2/10/25	Added Activity Diagrams	1.0
Eric	22/10/25	Updated use case diagram as per new requirement	2.0
Eric	22/10/25	Added a new use case description	2.0
Khawaja	24/10/25	Updated section 3.1	2.0
Yousef	24/10/25	Updated UI screenshots and context texts	2.0
Edward	25/10/25	Update the test case specification	2.0

1. Analysis Documentation

1.1 Use Case Diagram:



The diagram above illustrates the interactions between users and the AI-powered Discovery Bot System. The primary users, research analysts, academic researchers and business decision makers perform key actions such as searching articles, applying filters (by subject, location or industry) and viewing ranked results. Additionally, there is a view dashboard insights feature which enables users to explore trends, recommendations, backlinks, influencers and history logs, offering a broader analytical overview of retrieved data. Administrators (admin) are responsible for configuring data sources and API keys as well as monitoring system

performance, in order to ensure the system remains reliable and aligned with the expected requirements.

1.2 Use Case Descriptions

Use Case – Number & Name	UC_001: Search Articles	
Goal	- To allow users to submit a keyword query that retrieves relevant articles from multiple external sources (Reddit, news APIs) and consolidates them, and prepares them for ranking and summarisation.	
Preconditions	- The user has access to the system frontend. - APIs are configured and accessible. - Internet connection is available.	
Success End Condition	- A consolidated set of articles is retrieved, normalised, stored and forwarded for ranking/summarisation.	
Failed End Condition	- No articles are retrieved and the user is informed of the failure (invalid input).	
Primary Actors; Secondary Actors:	- Primary actors = Research analyst, academic researcher and business decision maker. - Secondary actors = External content APIs	
Trigger	- The user enters a keyword and initiates a search request.	
Description / Main Success Scenario	Step	Action
	1	The user enters a keyword in the search bar.
	2	System validates the input (non-empty and properly formatted).
	3	Validated query is forwarded to the backend.
	4	Backend sends requests to multiple external APIs.

	5	External APIs return raw article data (titles, URLs, metadata).
	6	Backend normalises and deduplicates results.
	7	Articles are stored in the database for reuse.
	8	Retrieved articles are forwarded for ranking and summarisation.
	9	The system confirms completion of the retrieval process.
Alternative Flows <a: condition causing branching> <a1: action or name of sub use case>	Step	Branching Action
	2.1	Invalid Input: If the user submits an empty or malformed keyword, the system displays an error message.
	4.1	API authentication error: If API keys are invalid, the error is logged and flagged to the Admin.
	5.1	Partial API Failure: If one or more APIs fail or rate-limit, the system continues with available sources and informs the user.
	5.2	Timeout: If APIs take too long, the system cancels the request and shows partial results with a timeout notice.

Use Case – Number & Name	UC_002: View Ranked Results	
Goal	- To enable users to view a ranked list of articles based on relevance, semantic similarity and engagement metrics. The goal is to give users a prioritised and digestible view of search results.	
Preconditions	- A search query has already been executed (UC_001: Search Articles). - Ranking and summarisation processes have completed successfully. - Articles, scores and summaries exist in the database.	
Success End Condition	- The user sees a ranked list of articles with summaries displayed in the frontend.	
Failed End Condition	- No results available to display (if all retrieval/ranking failed). A no result found message is shown.	
Primary Actors; Secondary Actors:	- Primary actors = Research analyst, academic researcher and business decision maker. - Secondary actors = AI/NLP Engine (responsible for ranking and summarisation).	
Trigger	- The user selects “View Results” after completing a search or the frontend auto-refreshes once results are ready.	
Description / Main Success Scenario	Step	Action
	1	User initiates a search (from UC_001).
	2	Backend triggers ranking and summarisation modules.
	3	AI/NLP engine computes relevance scores and generates summaries.
	4	Ranked articles and summaries are stored in the database.

	5	Backend aggregates ranked results and packages them into JSON.
	6	Frontend fetches and displays ranked articles with summaries.
	7	The user views the ranked list of results and may interact with filters (UC_003).
Alternative Flows	Step	Branching Action
	3.1	Ranking failure: If AI/NLP fails, the system falls back to lexical (keyword-based) ordering.
	3.2	Summarisation failure: If summarisation fails, fallback snippets (first 2–3 lines of the article) are displayed.
	5.1	Partial results: If only some results are ranked, the system still displays available ones with a notice.
	6.1	Frontend timeout/error: If results cannot be fetched, the user is shown an error message and prompted to retry.

Use Case – Number & Name	UC_003: View Dashboard Insights
-------------------------------------	---------------------------------

Goal	- To allow users to access an interactive dashboard that displays key insights such as industry trends, recommendations, backlinks, influencers and history logs derived from previously retrieved and ranked articles.	
Preconditions	- The user has previously performed searches and the system has stored ranked and summarised article data.	
Success End Condition	- The user successfully views and interprets dashboard analytics, gaining insights from visualised data trends and recommendations.	
Failed End Condition	- The dashboard fails to load insights due to missing data or connection issues with external APIs or the database.	
Primary Actors; Secondary Actors:	- Primary actors = Research analyst, academic researcher and business decision maker. - Secondary actors = Admin	
Trigger	- The user enters a keyword and provides any of: subject, location, industry, then submits the search.	
Description / Main Success Scenario	Step	Action
	1	The user navigates to the Dashboard Insights section within the system.
	2	The system retrieves summarised data and ranked results from the database.
	3	The system aggregates information such as trending topics, top influencers, backlinks and historical activity logs.
	4	Data visualisations and summaries are generated dynamically and displayed to the user.
	5	The user reviews the dashboard insights to identify key industry or topic trends.

	6	The user may further refine displayed insights by selecting filters such as location, industry or location.
	7	The system updates and re-renders the dashboard with refined insights.
	8	The user interprets insights to support analysis, decision-making or reporting activities.
Alternative Flows	Step	Branching Action
	2.1	If no data is available, the system notifies the user that insights cannot be generated and prompts them to perform a new search.
	3.1	If an external API connection fails, the system retrieves and displays partial insights from available data sources.

Use Case – Number & Name	UC_004: Apply Filters (Subject / Location / Industry)
Goal	- To allow users to refine a search by supplying subject, location and industry along with the keyword, so retrieval and ranking operate only on context-relevant content.
Preconditions	- The user is on the search screen. - Keyword entry is available (see UC_001). - Valid lists (example: industry codes, location names) are available to validate/normalise inputs.
Success End Condition	- Filters are validated and normalised. A search context (keyword + filters) is saved and sent to the backend, where the results reflect the chosen filters.
Failed End Condition	- One or more filters are invalid/unsupported. The system prompts for correction or gracefully omits the invalid filter and proceeds (clearly informing the user).

Primary Actors; Secondary Actors:	- Primary actors = Research analyst, academic researcher and business decision maker. - Secondary actors = None	
Trigger	- The user enters a keyword and provides any of: subject, location, industry, then submits the search.	
Description / Main Success Scenario	Step	Action
	1	The user types a keyword and specifies subject / location / industry in the search form.
	2	The system validates each field (allowed values, length, characters).
	3	The system normalises filters (example: "AUS" means "Australia")
	4	The system builds a search context = {subject, location, industry, timestamp, user}.
	5	Search Context is stored for analytics..
	6	Backend receives the context and maps filters to source-specific parameters (query strings, categories).
	7	Backend queries external APIs with filters applied where supported. However, for sources without native support, filtering is applied post-fetch.
	8	Results are normalised/deduplicated and passed to ranking/summarisation.
	9	User later views filtered ranked results (UC_002).
Alternative Flows	Step	Branching Action
	2.1	Invalid filter value: System highlights the field and shows an inline message ("Unknown industry"). The user can correct and resubmit.
	5.1	Context store failure: Proceed without analytics storage; continue the search and log the incident.

	6.1	Source lacks filter support: Backend fetches broader results and performs server-side filtering before ranking.
	7.1	Partial API failure / rate limit: Proceed with available sources and flag the missing source in the UI.

Use Case – Number & Name	UC_005: Configure Sources & API Keys	
Goal	- To allow the admin to add, edit, enable/disable, and securely store API credentials and source settings so the system can retrieve content from approved providers reliably and compliantly.	
Preconditions	- Admin is authenticated with elevated privileges. - The system supports secure secret storage (encrypted DB). - Network access to provider endpoints is available.	
Success End Condition	- Source configurations (endpoint, parameters) and API keys are saved securely and a validation ping confirms connectivity.	
Failed End Condition	- Invalid credentials or unreachable endpoints which can prevent activation.	
Primary Actors; Secondary Actors:	- Primary actors = Admin - Secondary actors = External Content APIs (for connectivity checks)	
Trigger	- Admin opens the “Sources & API Keys” configurations and chooses to add or modify a source.	
Description / Main Success Scenario	Step	Action
	1	Admin opens the configurations and goes to Sources & API Keys.

	2	Admin selects a provider (Reddit, News API) or adds a new source.
	3	Admin enters credentials (API key/secret) and configuration (endpoint, request limits, default params, enable/disable).
	4	System validates fields (format, required values, rate-limit bounds).
	5	The system encrypts and stores secrets. Non-secret config saved in config store.
	6	The system performs a connectivity & permission check (test call with safe parameters).
	7	On success, the system activates the source and updates the active-source registry.
	8	System records an audit event (who, what, when, before/after without exposing secrets).
	9	Admin sees a confirmation and the source appears as active with health status.
Alternative Flows	Step	Branching Action
	3.1	Rotate API keys: Admin can generate/enter new keys; system preserves version history and retires old keys safely.
	4.1	Validation error: Missing/invalid fields, the system highlights errors and prevents save.
	5.1	Secret store unavailable: System aborts save, warns admin to retry and nothing is persisted.
	6.1	Connectivity failure: Endpoint unreachable/401, system shows error, logs details, keeps source Inactive.

Use Case – Number & Name	UC_006: Review System Performance
-------------------------------------	-----------------------------------

Goal	- To enable the admin to monitor system health and effectiveness. For example, API connectivity, fetch volumes, error rates, latency, ranking quality proxies and summary generation timing, so issues are detected early and SLAs are met.	
Preconditions	- Admin is authenticated with monitoring access. - Metrics and logs are being collected.	
Success End Condition	- The admin views up-to-date dashboards and/or exports a report. To any anomalies have clear precautions and next steps	
Failed End Condition	- Metrics unavailable or stale. Dashboards fail to render and the admin cannot assess health	
Primary Actors; Secondary Actors:	- Primary actors = Admin - Secondary actors = None	
Trigger	- Admin opens monitoring and the system performance alerts link from email.	
Description / Main Success Scenario	Step	Action
	1	Admin selects a time range (last 24h, last 7d) and environment (dev/prod).
	2	System loads overview KPIs: request count, success rate, summarisation time, ranking pipeline time and error rate.
	3	Admin checks into source health. For example: per provider success/timeout/rate-limit errors and average response time.
	4	Admin views ranking quality proxies (example: click through on top-N)
	5	Admin reviews summarisation metrics: avg tokens/time per article, fallback rate and model errors.

Alternative Flows	Step	Branching Action
	3.1	Provider outage detected: System highlights affected sources and suggests temporarily disabled or reduced concurrency.
	5.1	High fallback rate: System correlates to model/API changes and recommends model switch or context window adjustment.

1.3 User Stories

- As a **User**, I want to enter a **Keyword** and **Fetch** articles from different platforms (X, Reddit, News articles), So that I don't have to search up each site specifically
- As a **user**, I want the system to **handle API limits gracefully**, so that I still see results even if a source is **temporarily unavailable**.
- As a **user**, I want results ranked by relevance to my query, so that the most useful articles appear first.
- As a **sponsor**, I want ranking to include **freshness and engagement signals**, so that results feel current and meaningful.
- As a **user**, I want each article to include a short AI-generated summary, so that I can quickly decide if it's worth reading.

- As a **user**, I want a simple web UI where I can enter keywords and view ranked results, so that I can use the tool easily.
- As a **business decision maker**, I want to **refine my search** by location and industry so that I only see information relevant to my market context.
- As an **admin**, I want to **manage API keys and source settings** so that the system remains functional, secure and compliant with provider limits.
- As an **admin**, I want to **monitor system metrics** such as API availability and summarisation quality so that I can ensure the system remains reliable and useful.
- As a **Testing Lead**, I want automated test cases, so that system behavior is consistent across sprints.
- As a **sponsor**, I want a clear user manual, so that non-technical users can operate the system without developer help.

2. Design Documentation:

2.1 System Design Document:

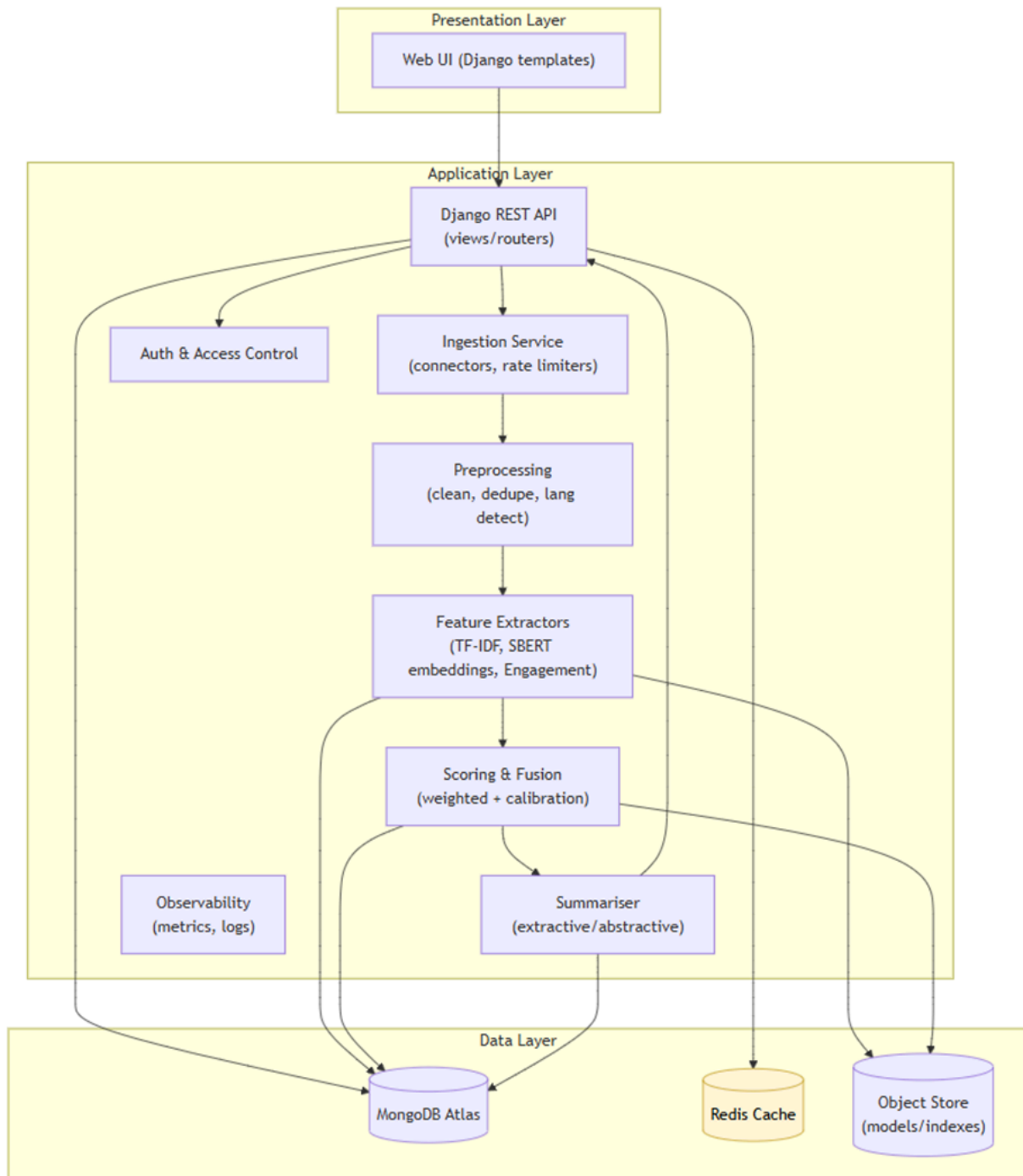
a) System Architecture

Overview

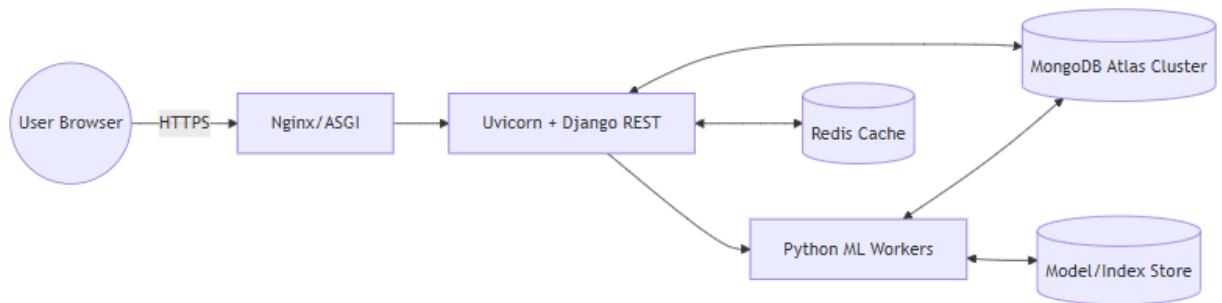
The system is a web application that ingests public content (e.g., Reddit, news), normalises and stores it, computes multi-factor relevance scores (lexical, semantic, engagement), and serves ranked results and summaries to users. We follow a service-oriented, three-tier architecture:

- **Presentation layer (Web UI)** - Django templates/React-lite front end for search, filters, and results.
- **Application layer (Django REST API + ML services)** - Request handling, orchestration, scoring pipeline, summaries.
- **Data layer (MongoDB Atlas + cache)** - Persistent storage for documents, features, scores, users, and logs; Redis (optional) for caching.

Package Diagram (logical)



Deployment View (high level)



Key Services/Packages

- api/ - REST endpoints (search, facets, health, feedback).
 - connectors/ - Data connectors (Reddit API, news feeds), backoff & rate-limit.
 - preprocess/ - Cleaning, language detection, normalisation, deduplication.
 - features/ - TF-IDF vectoriser, SBERT embeddings, engagement extractors.
 - ranking/ - Weighted score fusion, min-max/robust scaling, calibration.
 - summarise/ - Extractive (TextRank) now; abstractive later if approved.
 - persistence/ - MongoDB repositories, schema validators, TTL indexes.
 - security/ - API key vault loading, request sanitisation, output escaping.
 - observability/ - Structured logging, request/latency metrics, error tracing.
-

b) Storage / Persistent Data Strategy

Datastore Choice

- **MongoDB Atlas** for flexible, schemalite storage of heterogeneous documents from multiple sources; supports text indexes, compound indexes, and TTL for cacheable collections.
- **Redis (optional)** for short-lived caching of hot queries and pre-computed result sets.
- **Object store (repo or bucket)** for serialised TF-IDF models, FAISS indexes, and embedding caches.

Collections (proposed)

- documents
 - Fields: _id, source, source_id, url, title, author, body, lang, created_at, ingested_at, entities[], engagement{ups,comments,shares}, raw
 - Indexes: {source, source_id} (unique), created_at (desc), text index on title+body
- features
 - Fields: doc_id, tfidf_vec_ref, embedding_ref, engagement_norm, meta
 - Indexes: doc_id (unique)
- scores
 - Fields: doc_id, q_hash, lexical, semantic, engagement, final_score, weights, version, created_at
 - Indexes: {q_hash, final_score} (compound), TTL on created_at for cache expiry
- summaries
 - Fields: doc_id, q_hash, summary, method, created_at (TTL)
- queries (analytics/feedback)
 - Fields: q_text, filters, user_id?, latency_ms, clicked_doc_ids[], accepted@topk%
- models
 - Fields: name, version, type, path, checksum, licence, created_at

Persistence Policies

- **Write path:** idempotent upsert on documents to avoid duplicates; separate features and scores for modular upgrades.
 - **Retention:** recent content cached; old/low-value items pruned with TTL; permanent retention only for audit meta (no PII).
 - **Backups:** Daily Atlas snapshot; weekly integrity checks; recovery runbook.
 - **Data governance:** record source licence and terms per document; “right-to-erasure” process with tombstones.
-

c) Concurrency & Data Handling

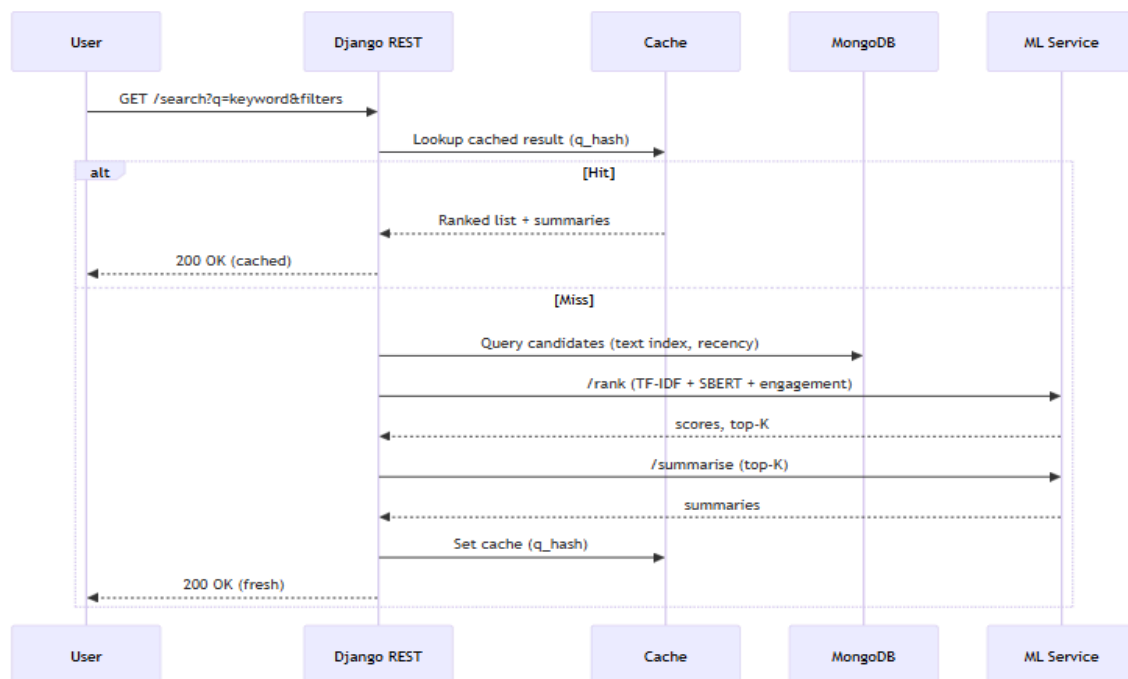
Workload Types

1. **Interactive requests (low-latency)** - search + rank + (cached) summary within target $P95 \leq 2.0s$.
2. **Background jobs (throughput-optimised)** - ingestion from sources, batch feature extraction, index rebuilds.

Concurrency Model

- **ASGI** (Uvicorn) for concurrent request handling.
- **Task queue (Celery/RQ)** for offloading: ingestion, heavy embedding computation, index building, and periodic cache warm-ups.
- **Rate-limit & backoff** in connectors to respect third-party APIs.
- **Locking strategy**: application-level idempotency keys for ingestion; FAISS/index writes guarded by file locks or single-writer policy.
- **Consistency**: eventual consistency between documents $\rightarrow$ features $\rightarrow$ scores; API returns best available data with freshness metadata.

Sequence (search happy path)



d) User Interface / User Experience Strategy

Principles

- **Fast first result** (TTFR $\leq 2s$) with progressive enhancement (lazy-load summaries).
- **Clarity & explainability** - show score breakdown on hover (lexical/semantic/engagement), source badges, and timestamps.
- **Accessibility** - WCAG 2.1 AA: semantic HTML, focus order, ARIA roles, colour-contrast, keyboard navigation, skip links.
- **Trust & safety** - display source, author (if public), and “report content” option; sanitize all rendered text.

Core Screens

1. **Search/Home** - query box, saved queries, recent searches.
2. **Results** - filters (time, source, industry/location), sort (Relevance, Recent), cards with snippet, highlights, score chip.
3. **Detail Drawer** - full text preview, entity chips, engagement metrics, “Why this result?” score explainer.
4. **Feedback** - “Relevant / Not Relevant” and “Improve results” (collects lightweight signals).

Interaction & Navigation

- Form/menu-based GUI (desktop-first, responsive for mobile).
- Global keyboard shortcuts: / focus search, $\uparrow/\downarrow$ navigate, Enter open, Esc close.
- Error states: explicit, human-readable (e.g., “Reddit API rate-limited; showing results from other sources”).

e) Design Decisions & Trade-offs

Tech Choices

- **Django REST + Python ML**: cohesive stack, rapid iteration, strong community and libs (NLTK, spaCy, sentence-transformers).
 - *Trade-off*: Python GIL limits CPU parallelism; mitigated via async I/O, task queues, vector ops, and precomputation.
- **MongoDB Atlas** for flexible, heterogeneous content and easy indexing;
 - *Trade-off*: No joins; mitigated by embedding denormalised fields and using aggregation pipelines.
- **SBERT (MiniLM) embeddings** for semantic recall + **TF-IDF** for precision;

- *Trade-off*: Embeddings add compute cost; mitigated by caching vectors and limiting candidate set before semantic scoring.
- **Weighted score fusion** (manual weights with calibration), not full learning-to-rank (LTR) in MVP;
 - *Trade-off*: LTR could yield better ranking but needs labelled data; we start with interpretable weights, add LTR once feedback accrues.
- **Extractive summaries** first (TextRank);
 - *Trade-off*: Abstractive needs more compute and may hallucinate; can be added later if budget permits.

Performance-Driven Design

- **Two-stage retrieval**: fast candidate fetch (text index + recency) → re-rank with embeddings.
- **Top-K only**: compute embeddings for K candidates (e.g., 200) to bound latency.
- **Cache & warm-ups**: cache popular queries; nightly warm-ups for common topics.
- **Indexes**: compound and TTL indexes aligned with query patterns; FAISS for similarity if needed.

Compliance & Security

- **Data minimisation**: store only public, licence-compatible content; record licence meta per doc.
- **Secrets**: environment-injected, never in client; strict CORS; HTTPS only.
- **Sanitisation**: strip HTML, block scripts, escape output to prevent XSS.
- **Auditability**: log model versions, weights, and time of scoring; reproducible outputs.

Reliability & Ops

- **Graceful degradation**: if a provider fails/rate-limits, still return partial results and disclose provenance.
- **Observability**: request traces, error budgets, per-stage latency histograms, daily SLA review.

2.2 User Interface Layouts:

Homepage:



Privacy

Discover what conversations are trending in your industry

Search

Top Topics

AI In Healthcare

Healthcare Cost

Workforce Health

Value Care

Health Data

AI-Topics Powered by Robotic Marketer®

Trending Topics

Multi-Cancer Early Detection / Liquid Biopsies
Digital Mental Health Devices & Regulation
Home Health, Remote Monitoring & "Hospital at Home" Models
Interoperability & Health Data Flow
Behavioral Health / Mental Health Integration into Primary Care

Competitors

UnitedHealth Group
CVS Health
McKesson
Cencora
Cardinal Health
Cigna
Elevance Health

Centene
Humana
HCA Healthcare
DaVita
Baxter International/Cigna
Henry Schein
Omnicell

Backlinks

<https://yourclinic.com/telehealth-seniors>
<https://yourclinic.com/rpm-best-practices>
<https://yourclinic.com/clinical-workflows>
<https://yourclinic.com/caregiver-resources>
<https://yourclinic.com/dbx-case-study>
<https://yourclinic.com/wearables-integration>

Influencers

Coming Soon...

News Feed

Apple Watch gets FDA clearance for hypertension detection

A new feature that assesses how blood vessels respond to heartbeats over 30 days is being rolled out.

FDA to evaluate AI-enabled mental health devices

The FDA's Digital Health Advisory Committee will meet on November 6, 2025 to review digital devices like mental health chatbots and virtual therapists, balancing benefits like access & scalability against risks.

Ideas

Community Health Data Hubs — open dashboards tracking local trends (obesity, asthma, mental health).

Climate & Health Initiatives — hospital-led clean air and water projects to reduce disease linked to pollution.

Health Literacy Campaigns — interactive workshops to help patients understand insurance, billing, and preventive care.

Results page:



AI in Healthcare

Search

History

Industry Search Results

FILTER

TIME

☐ Today

☐ 7 days

PLATFORM

☐ news_rss

☐ reddit_official

SENTIMENT

☐ Positive

☐ Negative

☐ Spike

TOP INFLUENCERS

☐ Show Only Influencers

PRIORITY

☐ High

☐ Medium

☐ Low

Trending Topics

Multi-Cancer Early Detection / Liquid Biopsies

Digital Mental Health Devices & Regulation

Home Health, Remote Monitoring & "Hospital at Home" Models

Interoperability & Health Data Flow

Opportunities

>> Generate Large Language Models in Healthcare

>> Produce campaigns related to Rising Healthcare Costs & Affordability Pressures

>> Write press release related to new alternate healthcare plans

>> Create social media campaign centered around clinical burnout

Action

☐ Email

☐ Blog

☐ Website Content

☐ Social Media

☐ Press Release

Submit

Payers, Providers, and Patients – Oh My!: AI in Healthcare – Key Policy Developments and State Laws - Crowell & Moring LLP

Payers, Providers, and Patients – Oh My!: AI in Healthcare – Key Policy Developments and State Laws Crowell & Moring LLP

news_rss

October 24, 2025

Insights: **• SPIKE**

Engagement: **▲ 0**

Relevance:

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business - CNBC TV18

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business CNBC TV18

news_rss

October 24, 2025

Insights: **• SPIKE**

Engagement: **▲ 0**

Relevance:

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer - AZoRobotics

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer AZoRobotics

news_rss

October 24, 2025

Insights: **• SPIKE**

Engagement: **▲ 0**

Relevance:

<

1

2

3

4

>

History Page:



Search

Search

Search History

Refine by Date

☐ Today

☐ Yesterday

☐ Last 7 days

☐ Last 30 Days

Date Searched	Topic	Engagement
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0

<

1

2

3

4

5

>

AI-Topics Powered by Robotic Marketer®

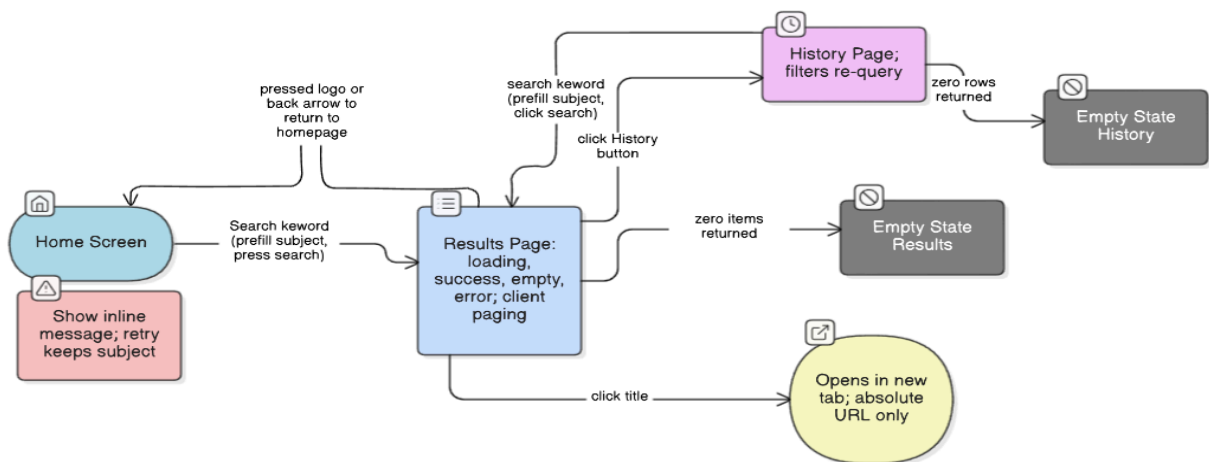
2.3 Report Layouts:

The primary output of the system is the dynamic Search Results Page, which is designed and fully detailed in the User Interface Layouts section. This page presents data in an interactive format where users can click, sort, and paginate through results. It's not generate any document format like PDF or CSV.

Therefore, separate static "report layouts" are not required for this project.

2.4 Window Navigation Diagram:

The flow of this prototype is straightforward and predictable. After selecting Home, users type in a topic and click Search. The application routes to Results, which handles client-side paging for quick page changes without requiring additional API calls and controls four UI states: loading, success, empty, and error. Users can jump to History or open an item in a new tab (absolute URL only) from Results. When filters or date ranges change, History re-queries the server; if no rows are returned, the user is presented with a clear Empty state. Navigation through the logo or back button returns to Home in a clean manner, and errors appear inline on Home with a retry that maintains the subject. Clear states, few round-trips, and seamless transitions between discovery (Results) and audit trail (History) are highlighted in the diagram.

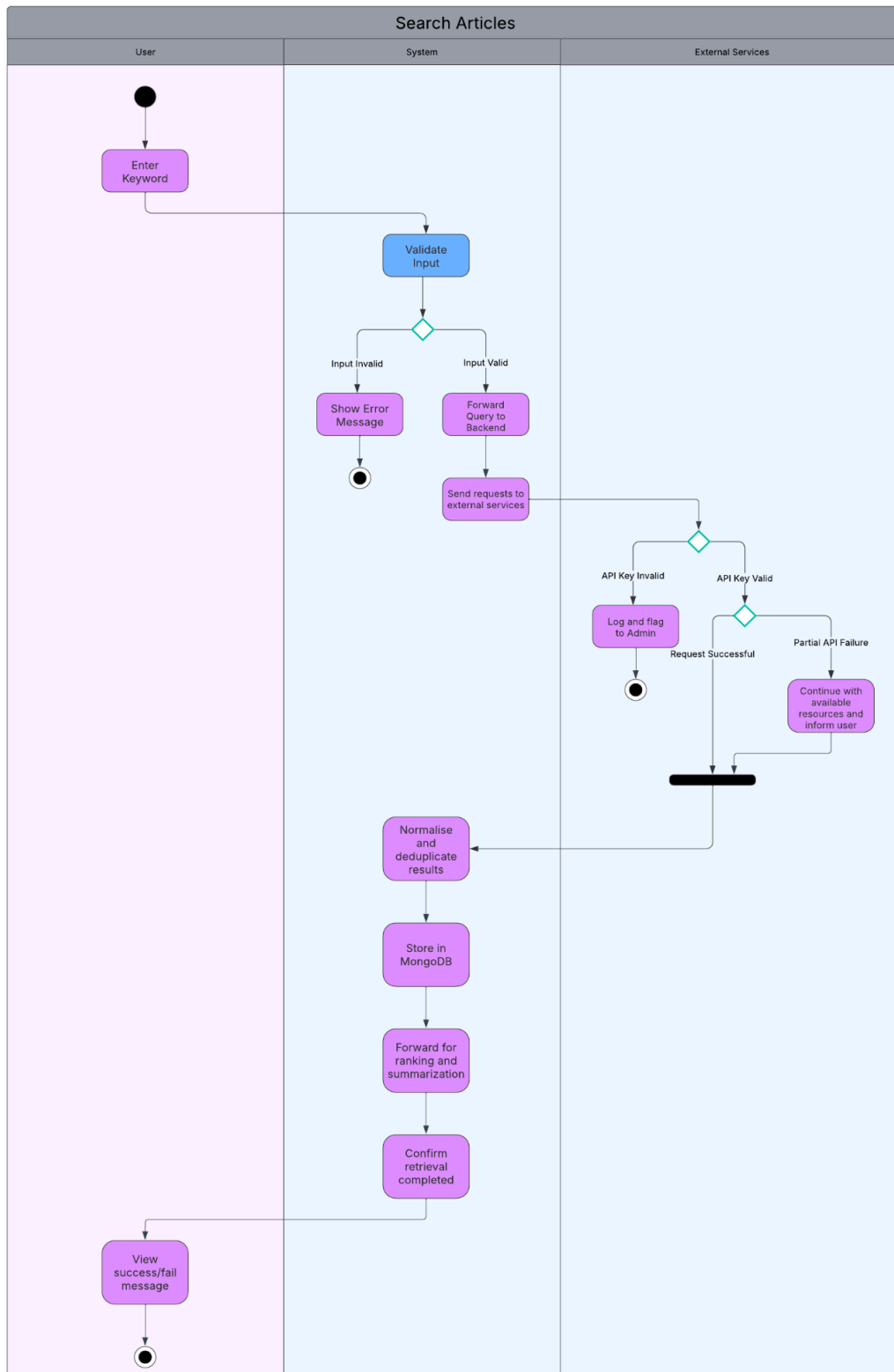


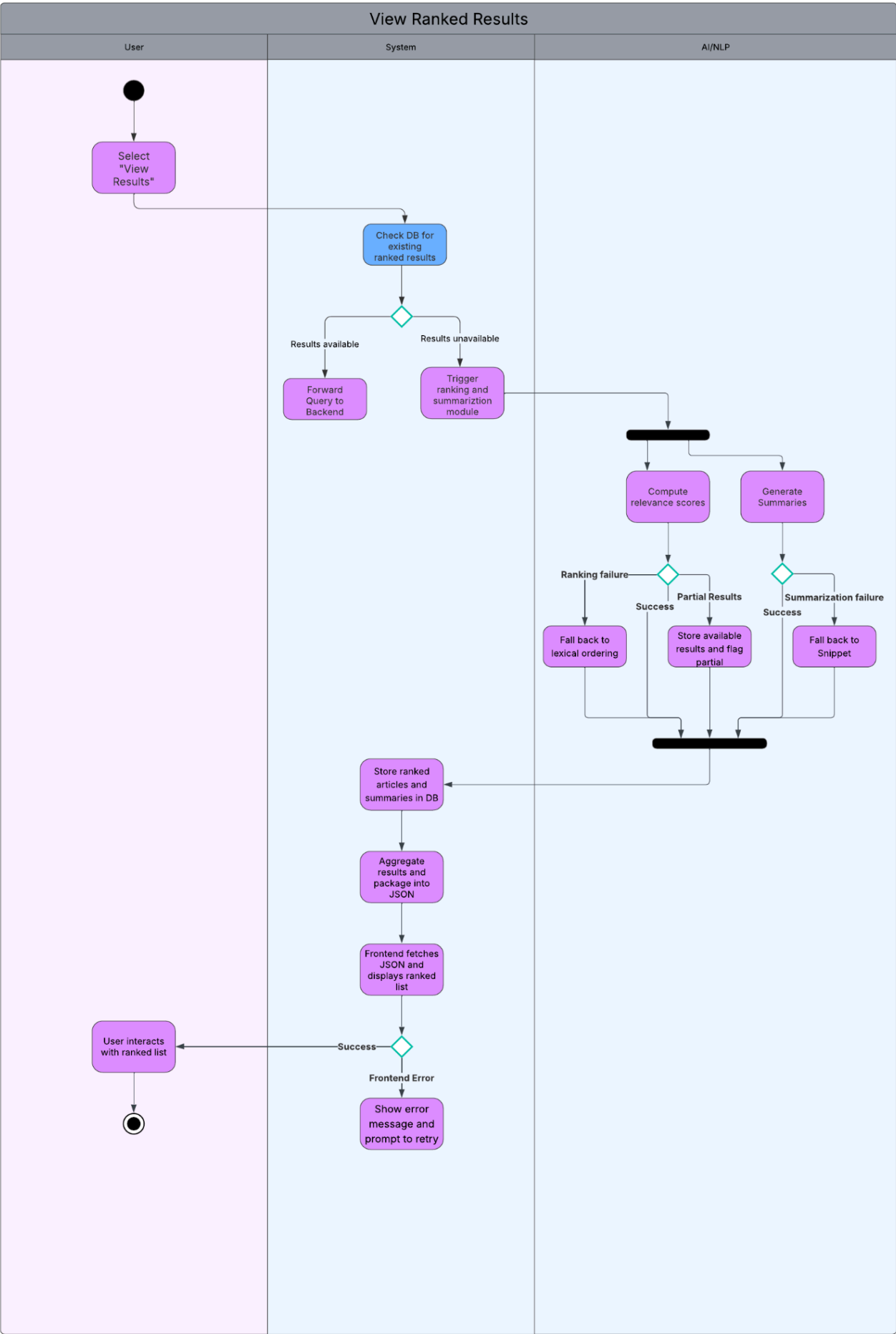
2.5 Data Definitions:

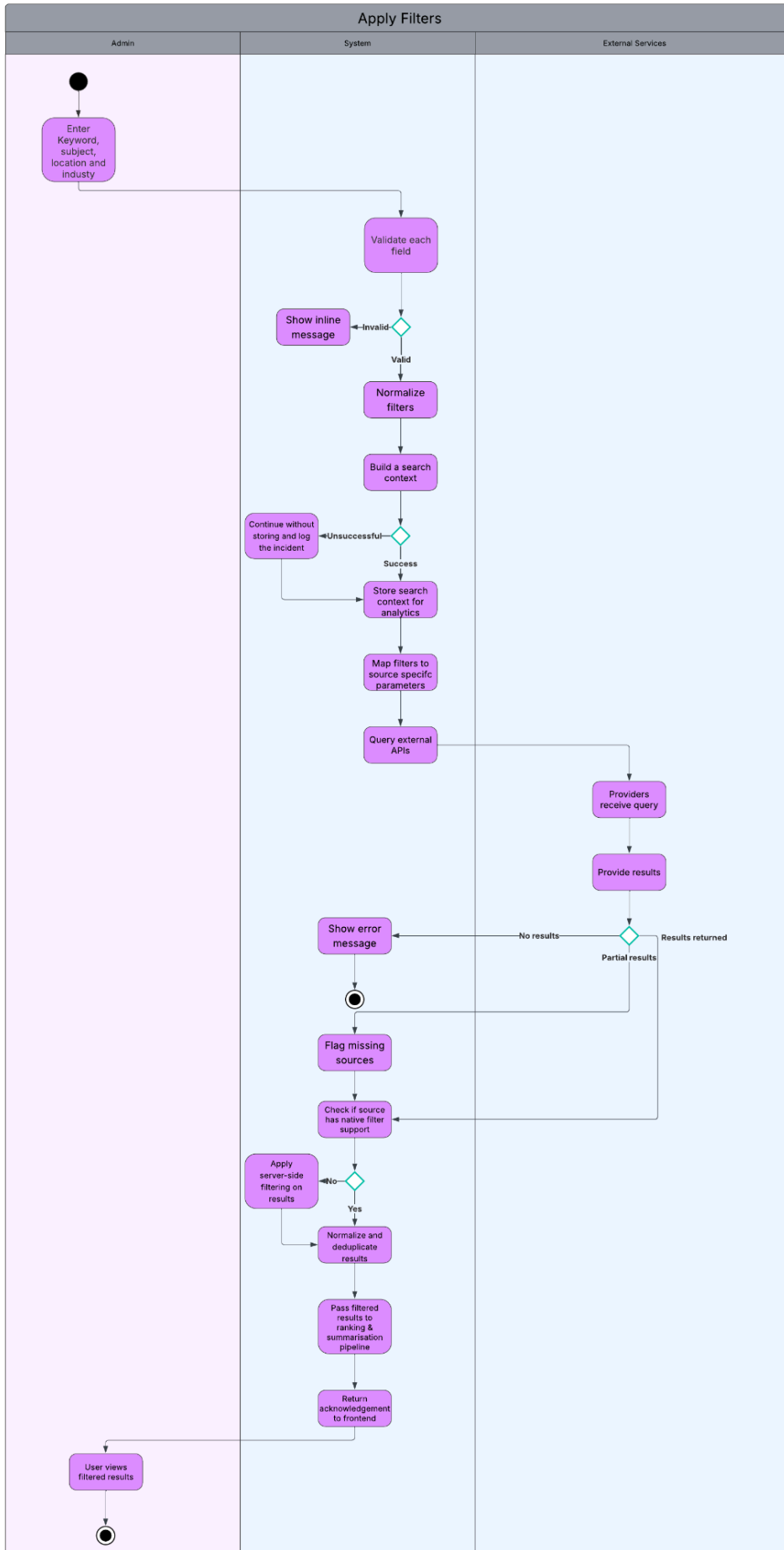
Collection: Articles

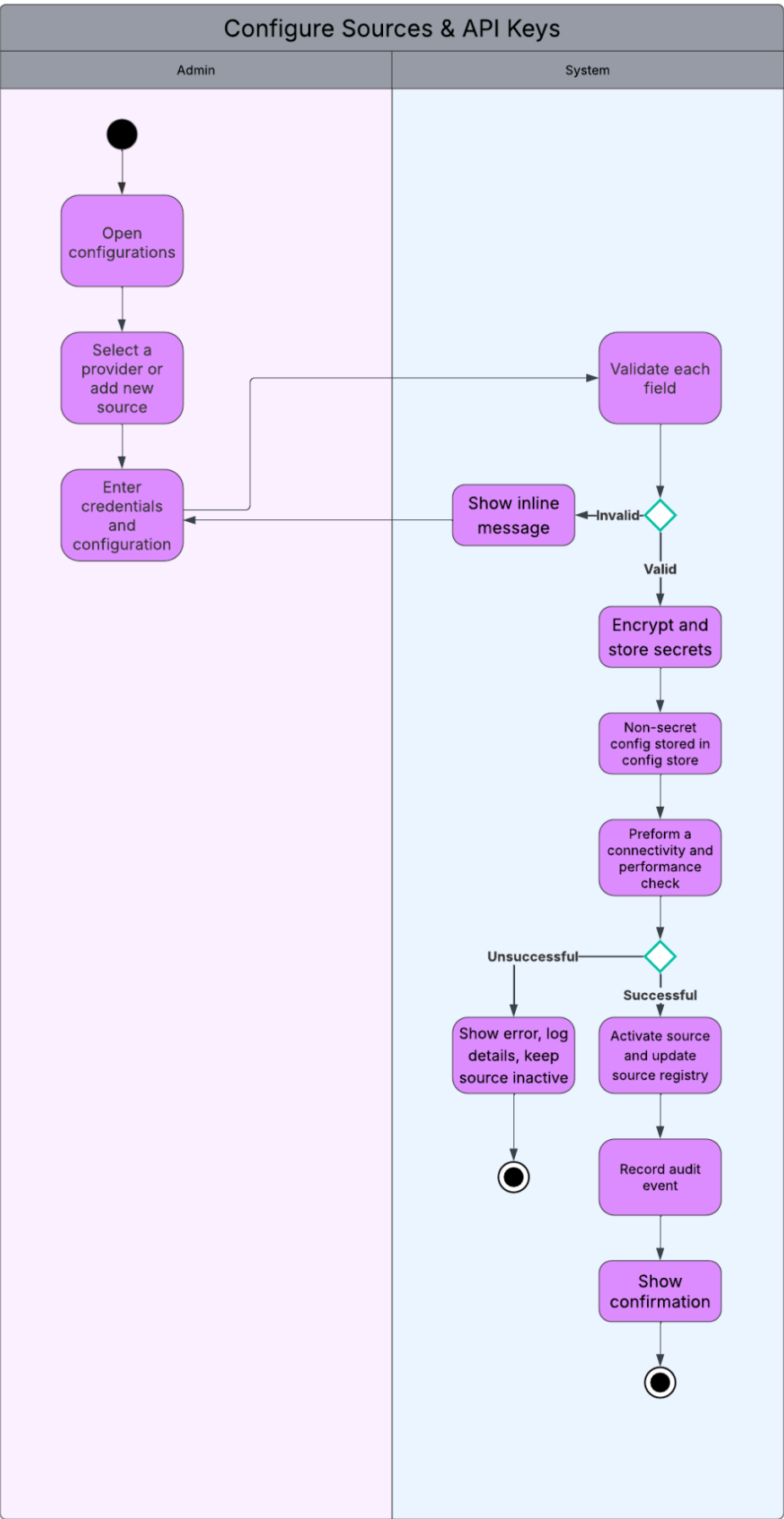
Field	Type	Example
url	String	"https://www.reddit.com/r/buildapc/comments/..
author	String	"Endlessly Careful"
engagement.score	number	1
engagement.num_comments	number	0
engagement.upvote_ratio	number	1
engagement.total_awards	number	0
post_id	string	"reddit_official:lnhmuoq"
published_ts	number(epoch)	1757945277
source_key	string	"reddit_official"
text	string	"Example"
text_html	string	"example"
title	string	"example"

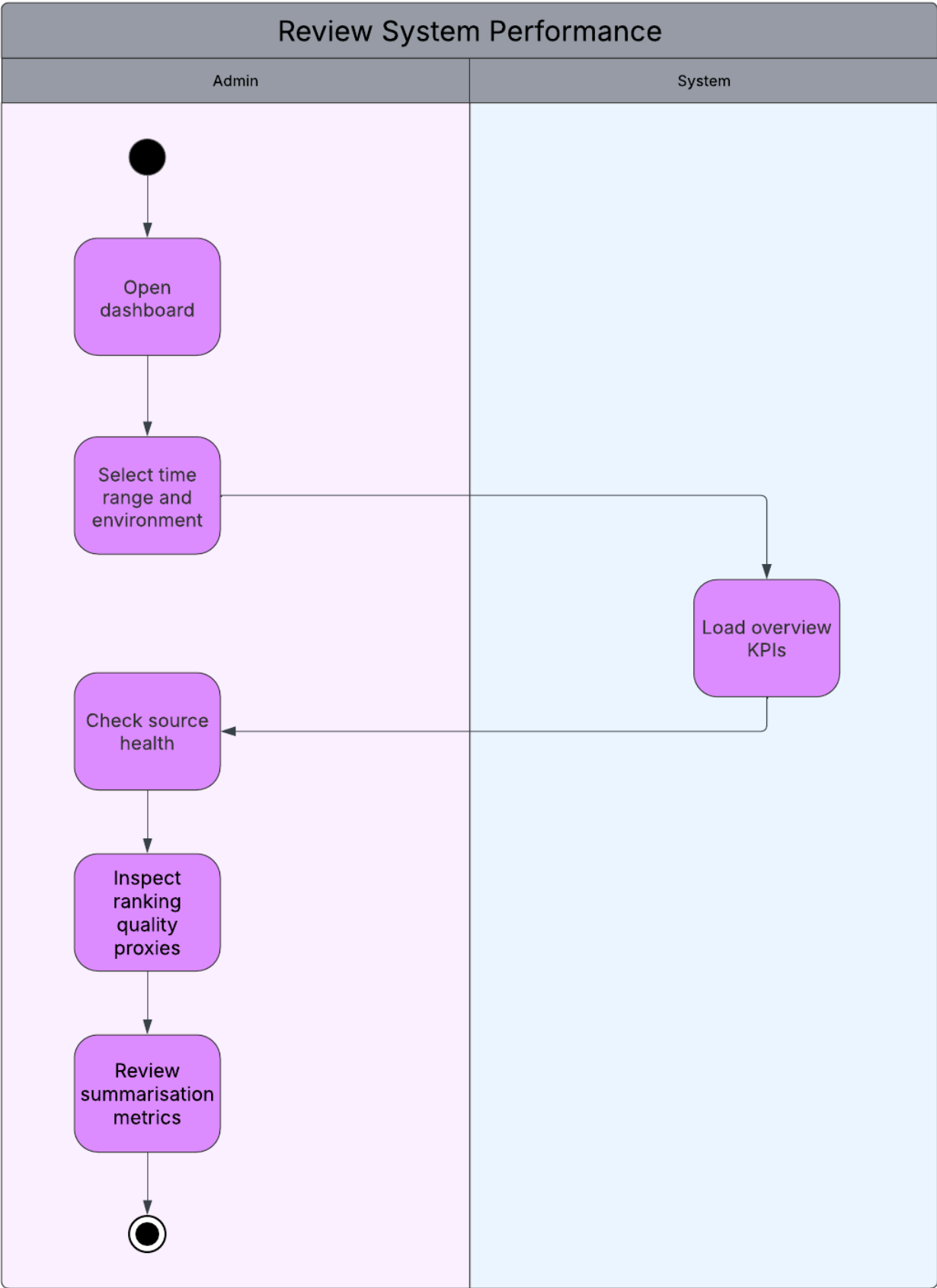
2.6 Activity Diagrams:











3. Testing Documentation:

3.1 Test Plan:

1. Objectives & Scope

This Test Plan defines the strategy, scope, schedule, resources, environments, tools, and exit criteria for validating the **AI-powered discovery web application** (Django REST + MongoDB + Redis + ML scoring pipeline: TF-IDF, SBERT embeddings, engagement features). Testing covers UI, API, data/ML, security, performance, accessibility, and deployment packaging across **Dev** → **Staging** → **Demo** environments.

2. Test Strategy

Approaches

- **Black-box**: end-to-end user flows (search, filter, view details, feedback), API contracts, accessibility, security.
- **White-box**: unit tests for preprocessing, feature extraction, ranking fusion, service utilities; code coverage tracking.
- **Grey-box**: ML evaluation against labeled holdout; DB assertions; cache behavior; rate-limit fallbacks.

Test Types

1. **Unit**: functions/classes (preprocess, tokenization, feature builders, scorers, serializers).
2. **Component/Service**: Django views/routers, ingestion connectors, Celery/worker jobs, cache layer.
3. **API Contract**: OpenAPI schema conformance, request/response validation, error codes.
4. **Integration**: API ↔ MongoDB/Redis ↔ ML pipeline; fallback on partial data; idempotency.
5. **System / E2E**: user journeys across UI (search→results→detail→feedback), pagination, filters, empty/error states.
6. **Data Quality**: schema, required fields, language/length constraints, duplicates/dedup, PII guardrails.
7. **ML Verification**: offline metrics (Precision@K, nDCG@K, MAP), AB of scoring weights, drift smoke checks.
8. **Performance/Load**: API latency (p50/p95), throughput, cache hit-rate, Mongo query timings.
9. **Security**: SAST, dependency scan, secrets scan; DAST (auth, injection, SSRF, CORS, headers).
10. **Accessibility (WCAG 2.1 AA)**: keyboard nav, contrast, ARIA roles, forms, focus order.

11. **Cross-browser/Device**: latest Chrome/Edge/Safari + iOS/Android mobile browsers.
12. **Regression**: automated suite on each PR + pre-release gate.
13. **Installation/Deployment**: clean env setup, migrations, seed data, rollbacks.
14. **Usability/UAT**: sponsor pilot tasks; SUS quick survey; qualitative feedback.

Entry / Exit Criteria

- **Entry**: feature flagged “ready for test”, unit tests $\geq 80\%$ line coverage for module, schema migrations merged, synthetic test data available.
- **Exit**: all **High/Critical** defects closed or accepted with mitigation; API p95 latency ≤ 1000 ms on Staging test load; **WCAG AA** automated checks $\geq 95\%$ pass; security scan **no Critical/High**; ML P@10 $\geq$ baseline target.

3. Test Environments & Data

- **Dev**: local Docker or venv; ephemeral Mongo/Redis; synthetic data.
- **Staging**: cloud MongoDB Atlas (non-prod), Redis, seeded with de-identified/synthetic content; feature flags on.
- **Demo**: stable snapshot for presentation; read-only DB user; logging at INFO.

Data Management

- Synthetic fixtures (JSON/CSV) per source; labeled holdout set for ML (topic relevance, 2-3 domains).
- No production PII; **Great Expectations** validations on ingest; VCR.py cassettes or mocked responses for external APIs.

4. Tools & Frameworks

- **Unit/Integration**: pytest, pytest-asyncio, pytest-django.
- **Mocking**: responses / requests-mock, **VCR.py** for API replay.
- **API**: schemathesis (OpenAPI fuzz), Postman/Newman smoke.
- **E2E/UI**: **Playwright** (headed & headless).
- **Performance**: **Locust** (Python) (or k6 if preferred).
- **Accessibility**: **axe-core** (Playwright axe), Lighthouse CI.
- **Security**: **Bandit** (SAST), **pip-audit** (deps), **TruffleHog** (secrets)
- **Data Quality**: **Great Expectations**.
- **ML Eval**: scikit-learn metrics, mlflow or CSV logs for runs.
- **Coverage/Lint**: coverage.py, **Ruff**, **Black**.
- **CI/CD**: GitHub Actions (PR checks: lint/test; nightly: integration, security, a11y; on-tag: release smoke).

5. Roles & Responsibilities

- **QA Lead (Yousef)**: quality gates, a11y/cross-browser, UI E2E, doc reviews.
- **Testing Lead (Edward)**: test data orchestration, CI pipelines, performance suites.

- **Technical Lead (Khawaja)**: ML verification design/targets, API contract tests, security posture.
- **Backend (Eric)**: integration tests, DB migrations, observability hooks.
- **ML (Anirudh / Amr)**: unit tests for NLP/embeddings, labeling guidelines, offline eval scripts.
- **All**: write tests for features they implement; fix defects within SLA.

6. Test Milestones, Deliverables & Calendar Schedule

Dates reflect the current term plan and university deadlines. Internal gates are set **earlier** to allow sponsor review.

Key Targets

- D3 (Prototype/MVP package): **Thu 02 Oct 2025**
- D4 (Expanded, with tests & user/training manual): **Sat 25 Oct 2025**
- D5 (Presentation/Demo): **Thu 06 Nov 2025**
- Handover Window: **06–20 Nov 2025**

Milestones & Deliverables

Date (2025)	Milestone	Deliverables
Sep 27–30	Test Bed Ready	Staging env; seeded synthetic data; baseline unit/integration suite; Playwright skeleton; coverage report #1
Oct 01–02	D3 Internal QA Gate	E2E smoke (search→results→detail); API contract pass; a11y quick scan; ML baseline P@10 logged; Sponsor smoke sign-off for D3
Oct 03–09	Perf & Security Pass #1	Locust 100 VU 10 min; p95 ≤ 1200 ms; Bandit/pip-audit clean; ZAP passive scan report; fix high findings
Oct 10–16	ML Evaluation Sprint	Label 200 new items; tune weights; report P@10/nDCG@10 vs baseline; capture ablations
Oct 17–22	A11y & Cross-Browser	axe ≥ 95%; manual keyboard checks; Safari/iOS/Android sanity

Oct 23	D4 Internal QA Gate	Full regression pass; performance p95 $\leq$ 1000 ms; security no High ; a11y pass; updated traceability matrix
Oct 24–25	D4 Submission	Test Plan + Test Reports (unit/integration/E2E/perf/sec/a11y), coverage #2, ML eval report, defect log
Oct 26–Nov 03	Hardening & UAT	Sponsor UAT scripts & results; usability notes; fixes merged; release notes draft
Nov 04–05	Demo Rehearsal QA	Demo dataset freeze; DR runbook; final smoke on Demo env
Nov 06	D5 Presentation	Live demo checklist, rollback plan
Nov 07–20	Handover QA	Runbooks, test evidence archive, defect backlog triage, training handover

Test Schedule (Resources)

- **Daily:** PR pipeline (lint+unit+API contract) < 10 min.
- **Nightly:** integration + E2E smoke, dependency & SAST scans.
- **Twice/Week:** performance (Locust) + a11y (axe/Lighthouse) on Staging.
- **Weekly:** ML offline eval & drift snapshot.

7. Test Design & Coverage

API Contracts

- Validate OpenAPI with Schemathesis: success, error (4xx/5xx), boundary (empty q, long q), rate-limit messages, pagination.

Critical User Journeys (E2E)

1. Search → Results → Detail → Back.
2. Apply filters (industry/location/date) → persistent state → clear filters.
3. No-results and partial-source fallback (one connector down).
4. Feedback (good/bad) stored; refresh shows persistence.

Data/ML

- Preprocess: language detect (allowlist), URL/HTML stripping, dedup (hash), long-text truncation policy.
- Features: TF-IDF vector shape & norm; SBERT embedding presence & L2 norm; engagement normalization.
- Fusion: deterministic weighted sum; weight boundaries; versioned scoring schema.
- Quality: Great Expectations suites on DOCUMENTS, SCORES, SUMMARIES collections.

Security

- Headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options).
- AuthZ on admin endpoints; CORS strict; secrets not in repo; dependency scan; ZAP passive + selected active rules (non-destructive).

Accessibility

- Keyboard-only nav, focus indicators, label/aria, color contrast ($\geq 4.5:1$), semantic landmarks, error text associations.

8. Defect Management

- Tracker: GitHub Issues with labels [severity:{critical|high|medium|low}], [area:{ui|api|ml|data|security|perf}].
- SLA: Critical within 24h; High within 48h; Medium within 5 days; Low in next sprint.
- Reporting: Include repro steps, env, logs, screenshots/har, failing test link.

9. Risks & Mitigations (Testing)

- **External API volatility** → mock/VCR cassettes; cached seed; feature flags per connector.
- **Flaky E2E** → network idle waits, deterministic test data, retries with artifacts.
- **Label scarcity for ML** → active learning sampling; weekly sponsor labeling session (small batch).
- **A11y regressions** → axe in CI; fail PR on critical violations.

10. Exit & Acceptance Criteria (per Release)

- Test execution $\geq 95\%$ of planned; **0 Critical/High** open.
- Coverage: **backend/ML $\geq 80\%$ lines**; critical modules $\geq 90\%$.
- Performance: $p95 \leq 1000$ ms (Staging profile); error rate $< 0.5\%$.
- Accessibility: automated checks $\geq 95\%$ pass + manual keyboard path OK.
- ML: $P@10 \geq$ **baseline target** and $\geq$ previous release (or justified trade-off).
- Security: no Critical/High SAST/DAST; secrets scan clean.
- Sponsor UAT: all must-have scenarios pass or have agreed workaround.

11. Test Deliverables

- Test Plan (this document) & Revision History
- Test Suites: unit, integration, E2E, performance, security, a11y
- Test Data & Fixtures (synthetic), API mock cassettes
- Reports: coverage, performance (Locust), a11y (axe/Lighthouse), security (Bandit/pip-audit/ZAP)
- ML Evaluation Report (metrics, plots, weight configs)
- Defect Logs & Traceability Matrix (requirements → tests → results)
- Release Notes, Known Issues, and Risk Log

3.2 Test Case Specifications:

Test Case ID: TC-SEARCH-001

Test Description:

Verify that the system correctly processes a valid search query and returns a successful response with relevant results.

Input Specifications:

- Subject: "Healthcare"

Expected Output:

1. The search results page loads successfully.
2. The page header displays the search criteria.
3. A list of at least one result card is displayed.
4. A result card showing the expected output including relevancy score, topic, and description

Actual Output:

The screenshot displays the 'Robotic marketer' interface for 'Industry Search Results' on the topic 'AI in Healthcare'. The search bar at the top contains 'AI in Healthcare' and a 'Search' button. The results are organized into several sections:

- FILTER**: Includes options for TIME (Today, 7 days), PLATFORM (news_rss, reddit_official), SENTIMENT (Positive, Negative, Spike), TOP INFLUENCERS (Show Only Influencers), and PRIORITY (High, Medium, Low).
- Trending Topics**: Lists topics like 'Multi-Cancer Early Detection / Liquid Biopsies', 'Digital Mental Health Devices & Regulation', 'Home Health, Remote Monitoring & "Hospital at Home" Models', and 'Interoperability & Health Data Flow'.
- Opportunities**: Lists opportunities such as 'Generate Large Language Models in Healthcare', 'Produce campaigns related to Rising Healthcare Costs & Affordability Pressures', 'Write press release related to new alternate healthcare plans', and 'Create social media campaign centered around clinical burnout'.
- Action**: Includes checkboxes for Email, Blog, Website Content, Social Media, and Press Release, with a 'Select All' button.
- Search Results**: Displays three news items, each with a title, source, date, and engagement/relevance metrics.
 - Item 1**: 'Payers, Providers, and Patients – Oh My!: AI in Healthcare – Key Policy Developments and State Laws - Crowell & Moring LLP'. Source: news_rss, Date: October 24, 2025. Insights: SPIKE, Engagement: 0, Relevance: High.
 - Item 2**: 'AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business - CNBC TV18'. Source: news_rss, Date: October 24, 2025. Insights: SPIKE, Engagement: 0, Relevance: High.
 - Item 3**: 'Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer - AZoRobotics'. Source: news_rss, Date: October 24, 2025. Insights: SPIKE, Engagement: 0, Relevance: High.

Pass all testing requirements.

Pass/Fail:

Pass

Test Case ID: TC-INPUT-002

Test Description:

Verify that the system properly validates the search form and shows an error when the "Subject" field is empty.

Input Specifications:

- Subject: "" (Empty string)
- Subject: "Big"

Expected Output:

1. When the subject is empty, the form is not submitted and an error message is displayed.
2. When a valid subject is provided, the search executes successfully.

Actual Output:

Without the subject

```
(venv_backend) PS C:\Users\HP\Documents\GitHub\Pace-Unit\Backend> Invoke-
RestMethod -Uri "http://127.0.0.1:8001/api/search/" -Method POST -Body '{
"subject":"","sources":["bbc_rss"],"limit":5,"persist":true}' -Cont
entType "application/json"
Invoke-RestMethod : {"error":"subject is required"}
At line:1 char:1
+ Invoke-RestMethod -Uri "http://127.0.0.1:8001/api/search/" -Method PO
...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequ
est:HttpRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.P
owerShell.Commands.InvokeRestMethodCommand
```

With the subject:

```
RestMethod -Uri "http://127.0.0.1:8001/api/search/" -Method POST -Body '{
"subject":"big", "sources":["bbc_rss"],"limit":5,"persist":true}' -C
ontentType "application/json"

post_id      : bbc_rss:4735480912676389533
source       : bbc_rss
url          : https://www.bbc.com/sport/mixed-martial-arts/articles/cd7
2p2g1x17o?at_medium=RSS&at_campaign=rss
title        : 'Khabib was pretending to choke people' - how Hughes
              beats Nurmagomedov
text         : Paul Hughes' rematch with Usman Nurmagomedov for the PFL
              lightweight title is the "biggest fight ever put on" by
              the promotion, says Dan Hardy.
text_html    : Paul Hughes' rematch with Usman Nurmagomedov for the PFL
              lightweight title is the "biggest fight ever put on" by
              the promotion, says Dan Hardy.
published_ts : 1759354221
author       :
engagement    : @{score=; num_comments=; upvote_ratio=}
```

Pass all testing requirements.

Pass/Fail:

Pass

Test Case ID: TC-AI-003

Test Description:

Verify that the AI/NLP system correctly assigns a higher relevance score to strong matches.

Input Specifications:

- Subject 1: "AI"
- As Subject 2: "jkljsjdkn"
- Test Data: Most results related to AI, second subject has no clear meaning.

Expected Output:

1. The relevance score for Result 1 ("AI") is higher than for Result 2 ("jkljsjdkn").

Actual Output:

Test 1: Subject: "AI" → {"processed": 56, "top_score": 0.4213227407239063}

Test 2: Subject: "jkljsjdkn" → {"processed": 56, "top_score": 0.28231849819421767}

Pass all testing requirements

Pass/Fail:

Pass

Test Case ID: TC-UI-004

Test Description:

Verify that the user can navigate to the second page of results using pagination.

Input Specifications:

- Perform a search that returns more than 10 results (e.g., “Sustainability”).
- Click the “Next” button or select page “2”.

Expected Output:

1. The page reloads or updates dynamically.
2. The displayed results differ from page 1.
3. Pagination control updates (e.g., “Page 2 of 5”).
4. “Previous” button becomes enabled.

Actual Output:

The screenshot displays the 'Robotic marketer' interface for 'Industry Search Results'. At the top, a search bar contains the term 'samsung', with 'Search' and 'History' buttons. The main content area is divided into several sections:

- Filter:** Includes sections for TIME (Today, 7 days), PLATFORM (news_rss, reddit_official), SENTIMENT (Positive, Negative, Spike - selected), TOP INFLUENCERS (Show Only Influencers), and PRIORITY (High, Medium, Low).
- Trending Topics:** Lists topics such as 'Multi-Cancer Early Detection / Liquid Biopsies', 'Digital Mental Health Devices & Regulation', 'Home Health, Remote Monitoring & "Hospital at Home" Models', and 'Interoperability & Health Data Flow'.
- Opportunities:** Lists actions like 'Generate Large Language Models in Healthcare', 'Produce campaigns related to Rising Healthcare Costs & Affordability Pressures', 'Write press release related to new alternate healthcare plans', and 'Create social media campaign centered around clinical burnout'.
- Action:** Includes checkboxes for Email, Blog, Website Content, Social Media, and Press Release, with a 'Submit' button.

The search results are displayed in a list format, showing the following items:

- Hisense HS2100 vs SAMSUNG C450 which one should i choose?**
So i'm kinda live in a small condo but those 2 can sure fit. And yet i'm have no idea which one should i pick i know them specs but couldn't decided still. Can anyone can help me with this?
Insights: SPIKE
Engagement: 1
Relevance: [Progress Bar]
reddit_official | October 25, 2025
- IDK if this funny counts, but, I very VERY Much thought it was talking to ChatGPT constantly that brain-rot-ruined my texting this damn badly - turns out Samsung replaced my keyboard back without me noticing**
No description available.
Insights: SPIKE
Engagement: 3
Relevance: [Progress Bar]
reddit_official | October 25, 2025
- Samsung notes in viwoods air paper**
Hi, is it possible to install Samsung notes in viwoods air paper? If so, how? Thanks.
Insights: SPIKE
Engagement: 1
Relevance: [Progress Bar]
reddit_official | October 25, 2025

At the bottom, a pagination control shows page 2 of 5, with buttons for previous, next, and page numbers 1, 2, 3, 4, 5.



Industry Search Results

FILTER

TIME

☐ Today

☐ 7 days

PLATFORM

☐ news_rss

☐ reddit_official

SENTIMENT

☐ Positive

☐ Negative

☐ Spike

TOP INFLUENCERS

☐ Show Only Influencers

PRIORITY

☐ High

☐ Medium

☐ Low

Trending Topics

Multi-Cancer Early Detection / Liquid Biopsies

Digital Mental Health Devices & Regulation

Home Health, Remote Monitoring & "Hospital at Home" Models

Interoperability & Health Data Flow

Opportunities

>> Generate Large Language Models in Healthcare

>> Produce campaigns related to Rising Healthcare Costs & Affordability Pressures

>> Write press release related to new alternate healthcare plans

>> Create social media campaign centered around clinical burnout

Action

☐ Email

☐ Blog

☐ Website Content

☐ Social Media

☐ Press Release

Newer Samsung phones = more eye strain?

Hi, to those who came from S22 and got a new Samsung phone specifically the S24fe or S24 line have you noticed eye pain no matter how much you tweak the settings? Vivid or natural, adaptive or not, comfort shield etc. The display looks different with s24fe being less sharper and blacks less deep and as if there's a color filter on top that's messing with it. Had to change from an S22 because it gotten the pink line.

Insights:

Engagement:

Relevance:

I'm having Samsung M12 and considering buying GalaxyA56

So I was having Galaxy M12 for like some years now. It's quite heavy phone. The only thing I liked about was the battery 6000mah. It lasts for 2 days with light use. I'm thinking of buying GalaxyA56 but read the its battery 5000mah. Does it mean it's wont last like 2 days? And I assume Galaxy56 battery would better or I'm wrong? P.S. English not my native language. sorry for mistakes

Insights:

Engagement:

Relevance:

App has basically stopped working on Samsung TV

My TV model is UE50AU7020K00U When I try to launch the app, it fails. It first shows a loading screen, then the app closes. Then if I try to launch it again, nothing happens. If I reinstall the app, then I can use it. But the next time I turn on the TV the whole problem happens again. Anyone else?

Insights:

Engagement:

Relevance:

It can move to other page of result by clicking the next button

Currently the fetcher not yet been reconfigured by the front end, so we cannot go to next page

Pass/Fail:

Pass

Test Case ID: TC-ERROR-005

Test Description:

Verify that the application gracefully handles backend/API failures without crashing.

Input Specifications:

- Send POST request to /api/invalid_endpoint/
- Request body: {"subject":"big"}

Expected Output:

1. API returns an error response (e.g., HTTP 404).
2. Application displays a user-friendly error message.
3. No crash or unexpected behavior occurs.

Actual Output:

Invalid endpoint:

```
At line:1 char:1
+ Invoke-RestMethod -Uri "http://127.0.0.1:8001/api/invalid_endpoint/"
...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequ
est:HttpRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.P
owerShell.Commands.InvokeRestMethodCommand
```

It showing error if it's invalid so pass this testing requirements.

Pass/Fail:

Pass

Test Case ID: TC-HISTORY-006**Test Description:**

Verify that the system saves and displays past searches correctly in the history logs.

Input Specifications:

1. Perform a search with any subject (e.g., "Sustainability").
2. Open the "History" or "Recent Searches" tab.

Expected Output:

1. The performed search appears in the history list.
2. Each entry displays the correct timestamp and subject.
3. Clicking a history entry reloads the corresponding results.

Actual Output:



Search

Search

Search History

Refine by Date

- ☐ Today
- ☐ Yesterday
- ☐ Last 7 days
- ☐ Last 30 Days

Date Searched	Topic	Engagement
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0

< 1 2 3 4 5 >

AI-Topics Powered by Robotic Marketer®

It showing post, instead of search topic history, it supposed to show clickable search history topic which can bring user to the result of that tie search topic

Pass/Fail:

Fail

Test Case ID: TC-DISPLAY-007

Test Description:

Verify that each result card displays all required fields .

Input Specifications:

- No input.

Expected Output:

1. Each result card displays a non-empty square, atleast it must correspond to the company profile

Actual Output:

TRENDING TOPICS

Multi-Cancer Early Detection / Liquid Biopsies

Tele-health and Remote Patient Monitoring & Regulation

Home Health, Remote Monitoring & Hospital at Home Models

Interoperability & Health Data Flow

Behavioral Health / Mental Health Integration into Primary Care

COMPETITORS

UnitedHealth Group

Centene

Medtronic

Cerner

Cardinal Health

Roche International

Elevance Health

CVS Health

Humana

HCA Healthcare

DaVita

Cigna

Omniceil

Kaiser Permanente

BACKLINKS

https://yourclinic.com/health-seniors

https://yourclinic.com/best-practices

https://yourclinic.com/clinical-workflows

https://yourclinic.com/research-resources

https://yourclinic.com/case-study

https://yourclinic.com/wearables-integration

INFLUENCERS

Coming Soon...

NEWS FEED

Apple Watch gets FDA clearance for hypertension detection

— open dashboards tracking local health metrics, setting benchmark data, allowing policy and practice decisions....

A new FDA-approved feature that assesses how blood vessels respond to heart beats, helping detect early signs of cardiovascular risk.

Climate & Health Initiatives — Regional heat policy

Hospitals and healthcare systems are launching heat management initiatives aimed at protecting patients from extreme weather.

IDEAS

Community Health Data Hubs

— open dashboards tracking local health metrics, setting benchmark data, allowing policy and practice decisions....

Climate & Health Initiatives

— hospital heat index programs and air quality monitoring with alerts focused on vulnerable patient populations....

Health Literacy Campaigns

— workshops and digital resources helping patients understand treatment options, insurance terms, and preventive care measures....

Currently the Card has not yet been implemented, and only shows random text. It supposed to be compared with the company profile and rank it with relevancy score and and currently it doesn't do that

Pass/Fail:

Fail

Test Case ID: TC-FILTER-008

Test Description:

Verify that the system applies filters correctly for topic search.

Input Specifications:

- Modify time, platform, sentiment, top influencer

Expected Output:

1. Returned results match applied filters.
2. Changing filter values updates displayed results accordingly.
3. Removing filters resets the result list.

Actual Output:

FILTER

TIME
☐ Today
☐ 7 days

PLATFORM
☐ news_rss
☐ reddit_official

SENTIMENT
☐ Positive
☐ Negative
☒ Spike

TOP INFLUENCERS
☐ Show Only Influencers



samsung

Search

History

Industry Search Results

FILTER

TIME

☐ Today
☐ 7 days

PLATFORM

☐ news_rss
☐ reddit_official

SENTIMENT

☐ Positive
☐ Negative
☒ Spike

TOP INFLUENCERS

☐ Show Only Influencers

PRIORITY

☐ High
☐ Medium
☐ Low

Trending Topics

Multi-Cancer Early Detection / Liquid Biopsies
Digital Mental Health Devices & Regulation
Home Health, Remote Monitoring & "Hospital at Home" Models
Interoperability & Health Data Flow

Opportunities

>> Generate Large Language Models in Healthcare
>> Produce campaigns related to Rising Healthcare Costs & Affordability Pressures
>> Write press release related to new alternate healthcare plans
>> Create social media campaign centered around clinical burnout

Action

☐ Email
☐ Blog
☐ Website Content
☐ Social Media
☐ Press Release

Submit

Hisense HS2100 vs SAMSUNG C450 which one should i choose?

So i'm kinda live in a small condo but those 2 can sure fit. And yet i'm have no idea which one should i pick i know them specs but couldn't decided still. Can anyone can help me with this?

reddit_official

October 25, 2025

Insights:

+ SPIKE

Engagement:

▲ 1

Relevance:

IDK if this funny counts, but, I very VERY Much thought it was talking to ChatGPT constantly that brain-rot-ruined my texting this damn badly - turns out Samsung replaced my keyboard back without me noticing 🤡🤡

No description available.

reddit_official

October 25, 2025

Insights:

+ SPIKE

Engagement:

▲ 3

Relevance:

Samsung notes in viwoods air paper

Hi, is it possible to install Samsung notes in viwoods air paper? If so, how? Thanks.

reddit_official

October 25, 2025

Insights:

+ SPIKE

Engagement:

▲ 1

Relevance:

1

2

3

4

Not yet been implemented

Pass/Fail:

Pass

Test Case ID: TC-NETWORK-009

Test Description:

Verify that the system handles slow or unstable network conditions gracefully.

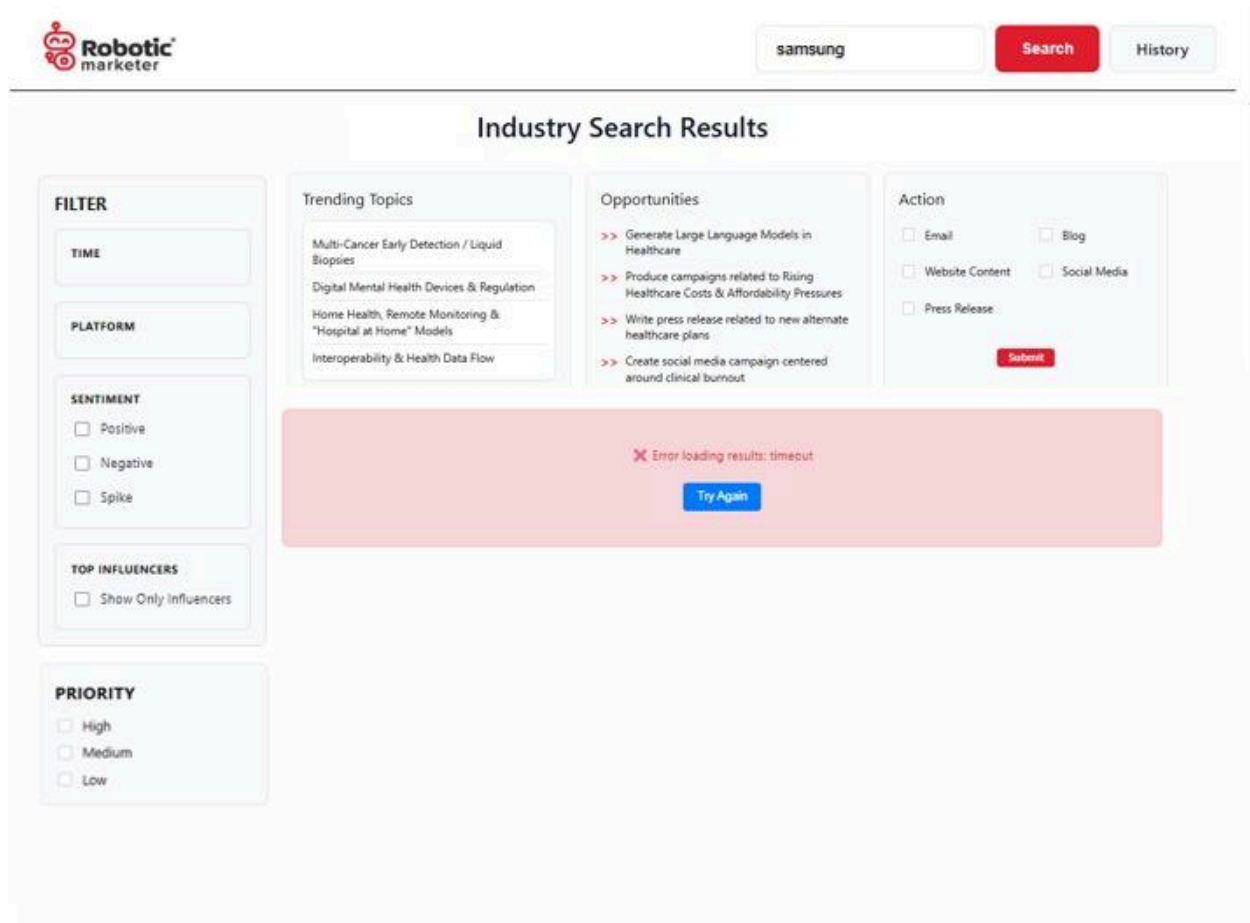
Input Specifications:

- Simulate network delay (e.g., throttle via browser developer tools).
- Perform search for “samsung”.

Expected Output:

1. Application shows a visible loading indicator during delay.
2. System does not freeze or crash.
3. Once response is received, results display correctly.

Actual Output:



Pass/Fail:

Pass

Test Case ID: TC-SECURITY-010

Test Description:

Verify that the system properly sanitizes user input to prevent code injection or malicious payloads.

Input Specifications:

- Subject: "<script>alert('XSS')</script>"

Expected Output:

1. Input is sanitized before being processed.
2. No scripts execute or HTML is rendered unsafely.
3. Application displays a safe error message or ignores malicious input.

Actual Output:

The screenshot shows the Robotic Marketer website interface. At the top left is the Robotic Marketer logo, and at the top right is a 'Privacy' link. The main heading reads 'Discover what conversations are trending in your industry'. Below this is a search bar containing the text '<script>alert('XSS')</script>' and a red 'Search' button. Under the search bar, there are five 'Top Topics' buttons: 'AI in Healthcare', 'Healthcare Cost', 'Workplace Health', 'Value Care', and 'Health Care'. At the bottom of the main content area, it says 'All Topics Powered by Robotic Marketer®'. The footer contains three sections: 'TRENDING TOPICS', 'COMPETITORS', and 'BACKLINKS'.

Pass/Fail:

pass

Test Case ID: TC-PERFORMANCE-011

Test Description:

Verify that the search response time meets acceptable performance standards.

Input Specifications:

- Subject: "Technology Trends"

Expected Output:

1. Response is returned within 3 seconds under normal network conditions.
2. UI remains responsive during data fetching.
3. No timeout or lag observed.

Actual Output:

Below than 1 minute

Pass/Fail:

Pass

Test Case ID: TC-HISTORY-012**Test Description:**

Verify that the database store the schema of history log

Input Specifications:

1. Perform a search with any subject (e.g., "Artificial intelligence").
2. Open the "History" collection inside mongodb

Expected Output:

1. The performed search return the history ID
2. There must be one history schema created based on that id

Actual Output:

Input:

```
(venv_backend) PS C:\Users\HP\Documents\GitHub\Pace-Unit\Backend> Invoke-  
RestMethod -Uri "http://127.0.0.1:8001/api/search/" -Method POST -Body '{  
"subject": "Artificial intelligence", "sources": ["reddit_official"], "lim  
it": 5, "persist": true}' -ContentType "application/json"
```

Output:

```
history_id          preview  
-----  
68fa1423e4395c72c543d912 {@{post_id=reddit_official:1odxxp3; source=r...
```

```
_id: ObjectId('68fa1423e4395c72c543d912')  
subject: "Artificial intelligence"  
location: ""  
industry: ""  
sources_used: Array (1)  
params: Object  
created_at: 2025-10-23T11:40:19.032+00:00  
ai_ready: true  
ai_count: 72  
ai_target: 100
```

< PREVIOUS

61-74 of 74 results

NEXT >

Have the same ID

Pass/Fail:

Pass

Total:

Pass = 10/12

Fail = 2/12

Appendix

Appendix - Data Model Snapshot (illustrative)

```
{
  "_id": "doc_abc123",
  "source": "reddit",
  "source_id": "t3_xyz",
  "url": "https://reddit.com/r/.../comments/...",
  "title": "Industry X trend in 2025",
  "body": "Post text ...",
  "lang": "en",
  "created_at": "2025-08-20T10:05:00Z",
  "ingested_at": "2025-08-20T10:06:10Z",
  "entities": ["Industry X", "Company Y", "Location Z"],
  "engagement": {"ups": 154, "comments": 23},
  "licence": "platform-terms-ok"
}
```



<AI-Powered Web Application for Bot Listening>

Version <02>, <25/10/2025>

Table of contents:

1. Introduction

2. Prototype Overview

3. Prototype Evidence

- 3.1 Screenshots / UI Evidence
- 3.2 Code Snippets
- 3.3 Database Schema
- 3.4 Demonstration Scripts / Workflow

4. Sponsor meeting, feedback and response to feedback

- 4.1 Meeting Details
 - 4.2 Sponsor Feedback
 - 4.3 Team Response / Action Plan
-

Revision History

Name	Date	Reason For Changes	Version
Eric	01/10/25	Added a brief introduction	1.0
Eric	01/10/25	Added a table with some details in section 4.1	1.0

Eric	01/10/25	Added some feedbacks that were received in section 4.2	1.0
Eric	01/10/25	Added action plans in section 4.3	1.0
Eric	01/10/25	Added screenshots with explanation in section 3.3	1.0
Eric	01/10/25	Added code snippets with explanation in section 3.2	1.0
Khawaja	30/09/25	Added code snippets with an explanation of Machine Learning in section 3.2	1.0
Khawaja	30/09/25	Added a sponsor's feedback on Machine learning in section 4.2	1.0
Khawaja	1/10/25	Added ML team response in section 4.3	1.0
Khawaja	2/10/25	Structure the document	1.0
Yousef	01/10/25	Added UI display screenshots to section 3.1	1.0
Yousef	01/10/25	Added code snippets to section 3.2 for the front-end	1.0
Yousef	01/10/25	Completed section 3.4	1.0
Edward	01/10/25	Doing the prototype overview	1.0
Edward	01/10/25	Add some points on section 4	1.0
Anirudh	01/10/25	Added some key features	1.0
Anirudh	02/10/25	Reviewed the entire document for consistency	1.0
Anirudh	02/10/25	Reviewed the final structure of the document	1.0
Eric	24/10/25	Added updated screenshots to code snippets section	2.0
Eric	24/10/25	Updated using our latest database schema in database schema section	2.0
Eric	24/10/25	Added screenshots taken from MongoDB Atlas	2.0
Khawaja	24/10/25	Added new code snippet of ML visual studio code structure in section 3.2	2.0
Khawaja	24/10/25	Updated section 4 by adding sponsor feedback of meeting 3 and team action plan on it	2.0
Yousef	24/10/25	Updated UI screenshots and code screenshots with context	2.0
Anirudh	24/10/25	Updated section 3.2 to contain the latest code snippets	2.0

1.Introduction

The purpose of this document is to present the prototype or Minimum Viable Product (MVP), developed as part of the PACE project in collaboration with Robotic Marketer. At this stage, the prototype acts as a proof of concept which gives both the team and the sponsor a chance to see how the system works in practice and whether it aligns with the intended goals. Rather than a finished product, it is an early working version designed to demonstrate the core functionality and set the foundation for further development.

The main objective of this prototype is to show that the system can successfully bring together its key components, such as: retrieving articles from multiple sources, ranking them for relevance, and generating clear summaries. Alongside this, the prototype provides a working interface that lets users interact with the system and see how these features fit together.

The main goal of the prototype is to prove that the system can reliably fetch and process data from external APIs, the ranking and summarisation models produce meaningful results and that the user interface feels straightforward and intuitive. All in all, it provides an opportunity to validate the technical feasibility of integrating the frontend, backend, database and AI/NLP modules into one smooth workflow.

This document covers what has been achieved so far at the MVP stage. It includes evidence such as screenshots, code snippets, database schemas and examples of how the workflow operates. It also records the feedback received from the sponsor during review sessions, along with the team's planned responses to that feedback.

2. Prototype overview

Description of the current MVP

The current Minimum Viable Product (MVP) is designed to collect, process and deliver aggregated data from multiple external sources. The system focuses on extracting, normalizing and storing structured data, while providing filtering mechanisms to allow the system to categorize the information output based on subject, location, and industry of the client. The MVP currently includes the main filtering and ranking functionality, with only a partial integration of engagement metrics.

Key Features:

- A database model was designed and configured within Django to support the structured storage of extracted content.
- Custom fetcher functions were implemented to allow the collection of data from multiple sources, including The Guardian API, Hacker News, BBC RSS, TechCrunch.
- A dedicated parser function was implemented to handle and normalize the various RSS formats.
- Serializers were enhanced to enable seamless conversion of stored data into JSON, ensuring compatibility with API responses.
- Data filtering mechanisms were implemented to allow for categorization of information based on subject, location and industry.
- A source seeding process was added to pre-populate the system with relevant source metadata.
- A persistence layer was developed and configured to store processed data into the database, with MongoDB acting as the primary data store.

- The Django admin dashboard was configured for improved visualization and easier data management.
- A relevancy algorithm was designed, consisting of 3 algorithms.
- The TFIDF and SBERT algorithms were implemented to give a similarity score between the article and the keyword.
- An algorithm was developed to rank the engagement metric.
- A combiner algorithm was developed to take into account the multiple rankings and produce a final similarity score.
- The algorithm sorts the articles based on relevancy and appends the score to each row in MongoDB for easier fetching in the future.

Technologies Used:

- **Backend Framework:** Django
- **AI Framework:** pretrained model for SBERT algorithm, TFIDF framework
- **Database:** MongoDB
- **APIs and Data Sources:** Guardian API, Hacker News, BBC RSS, TechCrunch RSS, Reddit API

Limitations:

- Engagement metrics only implemented for Reddit API.

3. Prototype evidence

3.1 Screenshots / UI Evidence

Homepage:

The homepage serves as a targeted point of entry for exploration. A large headline, one search input with the primary Search button, the Robotic Marketer logo, and optional topic pills that prefill and initiate a search are all displayed. When a term is submitted, it is routed to `/tool/results/?subject=` and is stored in `sessionStorage` for cross-page persistence. With an obvious inline error, empty submissions are blocked. The layout is keyboard-friendly (Enter submits, Escape clears), responsive, and centered.



Privacy

Discover what conversations are trending in your industry

Search

Top Topics

AI in Healthcare

Healthcare Cost

Workplace Health

Value Care

Health Care

All Topics Powered by Robotic Marketer®

TRENDING TOPICS

Multi-Cancer Early Detection / Liquid Biopsies
Tele-health and Remote Patient Monitoring & Regulation
Home Health, Remote Monitoring & Hospital at Home Models
Interoperability & Health Data Flow
Behavioral Health / Mental Health Integration into Primary Care

COMPETITORS

UnitedHealth Group	CVS Health
Centene	Humana
Medtronic	HCA Healthcare
Cerner	DaVita
Cardinal Health	Cigna
Roche International	Omniceil
Elevance Health	Kaiser Permanente

BACKLINKS

<https://yourclinic.com/health-seniors>
<https://yourclinic.com/best-practices>
<https://yourclinic.com/clinical-workflows>
<https://yourclinic.com/research-resources>
<https://yourclinic.com/case-study>
<https://yourclinic.com/wearables-integration>

INFLUENCERS

Coming Soon...

NEWS FEED

Apple Watch gets FDA clearance for hypertension detection

A new FDA-approved feature that assesses how blood vessels respond to heart beats, helping detect early signs of cardiovascular risk.

Climate & Health Initiatives — Regional heat policy

Hospitals and healthcare systems are launching heat management initiatives aimed at protecting patients from extreme weather.

IDEAS

Community Health Data Hubs

— open dashboards tracking local health metrics, setting benchmark data, allowing policy and practice decisions...

Climate & Health Initiatives

— hospital heat index programs and air quality monitoring with alerts focused on vulnerable patient populations...

Health Literacy Campaigns

— workshops and digital resources helping patients understand treatment options, insurance terms, and preventive care measures...

Results page:

Analysis and follow-up occur on the Results page. The header repeats the keyword search and includes a History button. A three column board shows Trending Topics, Opportunities, and an Action panel for Email, Blog, Website Content, Social Media, and Press Release. The left sidebar provides data driven filters. Time and Platform options are derived from the current result set and update as results change. Sentiment includes Positive, Negative, and Spike. Top Influencers is void for now as per request. Priority offers High, Medium, and Low. The main area lists paginated result cards with title, summary, source chip, published date, Insights badge, Engagement with up or down indicator, and a Relevance bar from 0 to 100. Clicking a title opens the source in a new tab. The page preserves the keyword, selected filters, and page index, and handles loading, empty, and error states.

Industry Search Results

FILTER

TIME

- ☐ Today
☐ 7 days

PLATFORM

- ☐ news_rss
☐ reddit_official

SENTIMENT

- ☐ Positive
☐ Negative
☐ Spike

TOP INFLUENCERS

- ☐ Show Only Influencers

PRIORITY

- ☐ High
☐ Medium
☐ Low

Trending Topics

Multi-Cancer Early Detection / Liquid Biopsies
Digital Mental Health Devices & Regulation
Home Health, Remote Monitoring & "Hospital at Home" Models
Interoperability & Health Data Flow

Opportunities

- >> Generate Large Language Models in Healthcare
- >> Produce campaigns related to Rising Healthcare Costs & Affordability Pressures
- >> Write press release related to new alternate healthcare plans
- >> Create social media campaign centered around clinical burnout

Action

- ☐ Email ☐ Blog
☐ Website Content ☐ Social Media
☐ Press Release

Submit

Payers, Providers, and Patients – Oh My! AI in Healthcare – Key Policy Developments and State Laws - Crowell & Moring LLP

Payers, Providers, and Patients – Oh My! AI in Healthcare – Key Policy Developments and State Laws Crowell & Moring LLP

news_rss October 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business - CNBC TV18

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business CNBC TV18

news_rss October 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer - AZoRobotics

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer AZoRobotics

news_rss October 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

< 1 2 3 4 >

History Page:

Past searches are displayed as a straightforward, sortable table on the history page. There is a date window refiner on the left (Today, Yesterday, Last 7 Days, Last 30 Days). Date Searched, Topic, and Engagement are listed in the main table along with a directional indicator. Pagination is consistent with the results page. Users can navigate away and back again without losing context thanks to the layout's keyboard accessibility, mirroring the product's styling, and state preservation (filters and page).



Search History

Refine by Date

- ☐ Today
- ☐ Yesterday
- ☐ Last 7 days
- ☐ Last 30 Days

Date Searched	Topic	Engagement
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0
October 25, 2025	AI in Healthcare	▲ 0

<
1
2
3
4
5
>

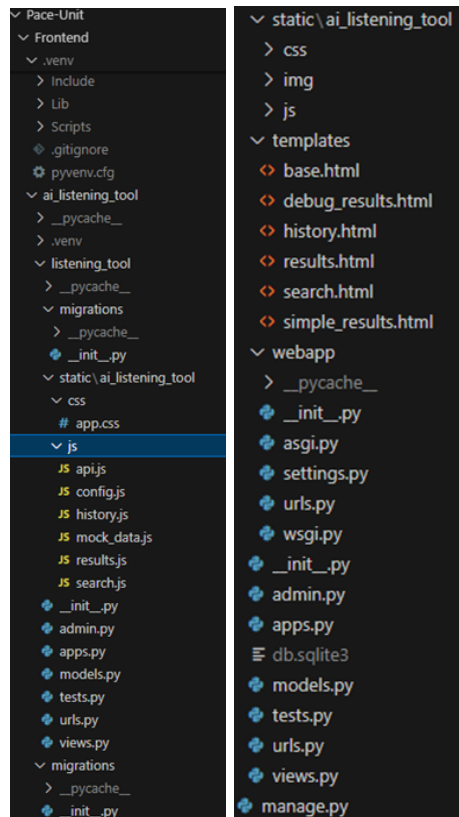
AI-Topics Powered by Robotic Marketer®

3.2 Code Snippets

Frontend

Overall frontend files:

The frontend is a Django project (webapp) with one UI app (ai_listening_tool). Pages use Django templates and vanilla JS from /static/ai_listening_tool. webapp/urls.py includes ai_listening_tool.urls at /tool. That file maps search, results, and history. base.html defines layout. search.html, results.html, and history.html extend it. config.js sets API_BASE. api.js wraps calls: startSearch, getStatus, getResults, getHistory. search.js submits the keyword search and routes to results. results.js polls status then paginates results. history.js lists past runs. app.css provides basic styles. This proof covers the frontend only. Search is keyword only. Paths listed here are authoritative.



UI pages:

The flow uses three pages. The keyword form lives in search.html and routes to Results. After the API starts, results.html renders placeholders that JS fills with loading, errors, items, and pager. Previous searches are listed in history.html. base.html provides the shared header, footer, and script blocks that all pages extend.

```

history.html | search.html X | results.html
Pace-Unit > Frontend > ai_listening_tool > templates > search.html > ...
5   <div id="rntool">
48   <section class="rm-grid">
80   <article class="rm-card">
82     <ul class="rm-list rm-list--links">
89       </ul>
90   </article>
91
92   <article class="rm-card">
93     <h3 class="rm-card-title">Influencers</h3>
94     <div class="rm-empty">Coming Soon...</div>
95   </article>
96
97   <article class="rm-card">
98     <h3 class="rm-card-title">News Feed</h3>
99     <ul class="rm-news">
100       <li>
101         <strong>Apple Watch gets FDA clearance for hypertension detection</strong>
102         A new FDA-approved feature that assesses how blood vessels respond to heart beats, helping detect early signs
103         of cardiovascular risk.
104       </li>
105       <li>
106         <strong>Climate & Health Initiatives – Regional heat policy</strong>
107         Hospitals and healthcare systems are launching heat management initiatives aimed at protecting patients from
108         extreme weather.
109       </li>
110     </ul>
111   </article>
112
113   <article class="rm-card">
114     <h3 class="rm-card-title">Ideas</h3>
115     <ul class="rm-news">
116       <li><strong>Community Health Data Hubs</strong> – open dashboards tracking local health metrics, setting
117       benchmark data, allowing policy and practice decisions....</li>
118       <li><strong>Climate & Health Initiatives</strong> – hospital heat index programs and air quality monitoring with
119       alerts focused on vulnerable patient populations....</li>
120       <li><strong>Health Literacy Campaigns</strong> – workshops and digital resources helping patients understand
121       treatment options, insurance terms, and preventive care measures....</li>
122     </ul>
123   </article>
124 </section>
125 </div>
126 {% endblock %}
127
128 {% block pagescripts %}
129 <script type="module" src="{% static 'ai_listening_tool/js/search.js' %}"></script>
130 {% endblock %}

```

```

history.html X search.html results.html
Pace-Unit > Frontend > ai_listening_tool > templates > history.html > ...
77 <script>
399 document.addEventListener("DOMContentLoaded", () => {
411 filterCheckboxes.forEach((checkbox, index) => {
412 checkbox.addEventListener('change', function () {
423 }
424 });
425 });
426
427 // Load history data
428 loadHistory();
429
430 // Start auto-refresh to catch new searches
431 startAutoRefresh();
432
433 // Stop auto-refresh when page is hidden/user switches tabs
434 document.addEventListener('visibilitychange', () => {
435 if (document.hidden) {
436 stopAutoRefresh();
437 } else {
438 loadHistory(true); // Refresh immediately when returning to page
439 startAutoRefresh();
440 }
441 });
442
443 // Clean up on page unload
444 window.addEventListener('beforeunload', () => {
445 stopAutoRefresh();
446 });
447 });
448 </script>
449
450 <style>
451 /* Fix table column widths to prevent resizing between pages */
452 .rm-history-data-table {
453 table-layout: fixed;
454 width: 100%;
455 }
456
457 .rm-history-data-table th:nth-child(1),
458 .rm-history-data-table td:nth-child(1) {
459 width: 20%;
460 /* Date Searched */
461 }
462
463 .rm-history-data-table th:nth-child(2),
464 .rm-history-data-table td:nth-child(2) {
465 width: 60%;
466 /* Topic */

```

```

history.html X search.html results.html X
Pace-Unit > Frontend > ai_listening_tool > templates > results.html > ...
418 <script>
1473 function renderPage(totalPages) {
1474 for (let p = 1; p <= pages; p++) {
1475 btns += `<button class="${p === currentPage ? 'rm-page-active' : ''}" data-page="${p}">${p}</button>`;
1476 }
1477 btns += `<button class="rm-page-arrow rm-page-next" data-nav="next" ${currentPage === pages ? 'disabled' : ''}>➡</button>`;
1478 return `<div class="rm-results-pager">${btns}</div>`;
1479 }
1480
1481 // Delegate click for pager
1482 document.addEventListener('click', (e) => {
1483 const btn = e.target.closest('.rm-results-pager button');
1484 if (!btn) return;
1485 const nav = btn.getAttribute('data-nav');
1486 if (nav === 'prev' && currentPage > 1) {
1487 currentPage--;
1488 renderResults();
1489 } else if (nav === 'next' && currentPage < TOTAL_PAGES) {
1490 currentPage++;
1491 renderResults();
1492 } else {
1493 const page = Number(btn.getAttribute('data-page'));
1494 if (!Number.isNaN(page)) {
1495 currentPage = page;
1496 renderResults();
1497 }
1498 }
1499 });
1500
1501 // Don't clear filters on page load (preserve from URL)
1502
1503 // Action card checkbox functionality - turn red when checked
1504 document.querySelectorAll('.rm-action-form input[type="checkbox"]').forEach(checkbox => {
1505 checkbox.addEventListener('change', function () {
1506 if (this.checked) {
1507 this.style.accentColor = '#e11d2e';
1508 } else {
1509 this.style.accentColor = '';
1510 }
1511 });
1512 });
1513
1514 // end page script
1515 </script>
1516 {% endblock %}

```

Front-end JS:

search.js reads the keyword, stores it in sessionStorage, starts the run, and navigates to Results. results.js polls the API by history_id, then requests paged results and renders cards with a simple client-side pager. External links are hardened. history.js queries the history endpoint and renders the table. api.js wraps HTTP calls. config.js defines API_BASE. Keyword-only input.

```
JS apijs JS configjs JS historyjs JS resultsjs JS searchjs X
Pace-Unit > Frontend > ai_listening_tool > static > ai_listening_tool > js > JS searchjs > ...
You, 2 weeks ago | 1 author (You)
You, 2 weeks ago • Add Frontend (cleaned) ...
1 "use strict";
2
3 document.addEventListener("DOMContentLoaded", () => {
4   const form = document.getElementById("searchForm");
5   if (!form) return;
6
7   // Pick the visible text input only
8   const input = form.querySelector('input[name="subject"]:not([type="hidden"])');
9
10  // Persist defaults; do NOT call preventDefault
11  form.addEventListener("submit", () => {
12    const subject = (input.value || "").trim();
13    sessionStorage.setItem("rm_subject", subject);
14    sessionStorage.setItem("rm_days", "7");
15    sessionStorage.setItem("rm_priority", "all");
16  });
17
18  // Optional: pills auto-fill and submit
19  document.querySelectorAll(".rm-pill[data-topic]").forEach(b => {
20    b.addEventListener("click", () => {
21      if (!input) return;
22      input.value = b.dataset.topic || "";
23      form.requestSubmit?.() || form.submit();
24    });
25  });
26 });
```

```
JS apijs JS configjs JS historyjs JS resultsjs X JS searchjs
Pace-Unit > Frontend > ai_listening_tool > static > ai_listening_tool > js > JS resultsjs > ...
3 document.addEventListener("DOMContentLoaded", () => {
145 function card(p) {
146
147   // Format engagement
148   let engagement = "0";
149   if (p.engagement && typeof p.engagement === 'object') {
150     engagement = p.engagement.score || p.engagement.num_comments || "0";
151   } else if (p.engagement) {
152     engagement = p.engagement;
153   }
154
155   // Format date
156   let dateStr = "";
157   if (p.published_ts) {
158     try {
159       const date = new Date(p.published_ts * 1000); // Convert Unix timestamp
160       dateStr = date.toLocaleDateString();
161     } catch (e) {
162       console.log("Date parsing error:", e);
163     }
164   }
165
166   const cardHTML = `
167   <article class="rm-card">
168     <header class="rm-card-h">
169       <h1>${title}</h1>
170       <div class="rm-meta">
171         <div><span class="rm-badge">TRENDING</span></div>
172         <div>Score: ${score}</div>
173         <div>Engagement: ${engagement}</div>
174       </div>
175     </header>
176     <p class="rm-desc">${summary}</p>
177     <footer class="rm-card-f">
178       ${dateStr} <span class="rm-chip">📅 ${dateStr}</span> : ""
179       <span class="rm-chip">📍 ${source}</span>
180       ${tags.map(t => `<span class="rm-chip">${t}</span>`).join("")}
181       <a class="rm-link" href="${url}" target="_blank" rel="noopener"> View Details</a> : ""
182     </footer>
183   </article>
184
185   console.log("Generated card HTML:", cardHTML);
186   return cardHTML;
187 }
188 });
```

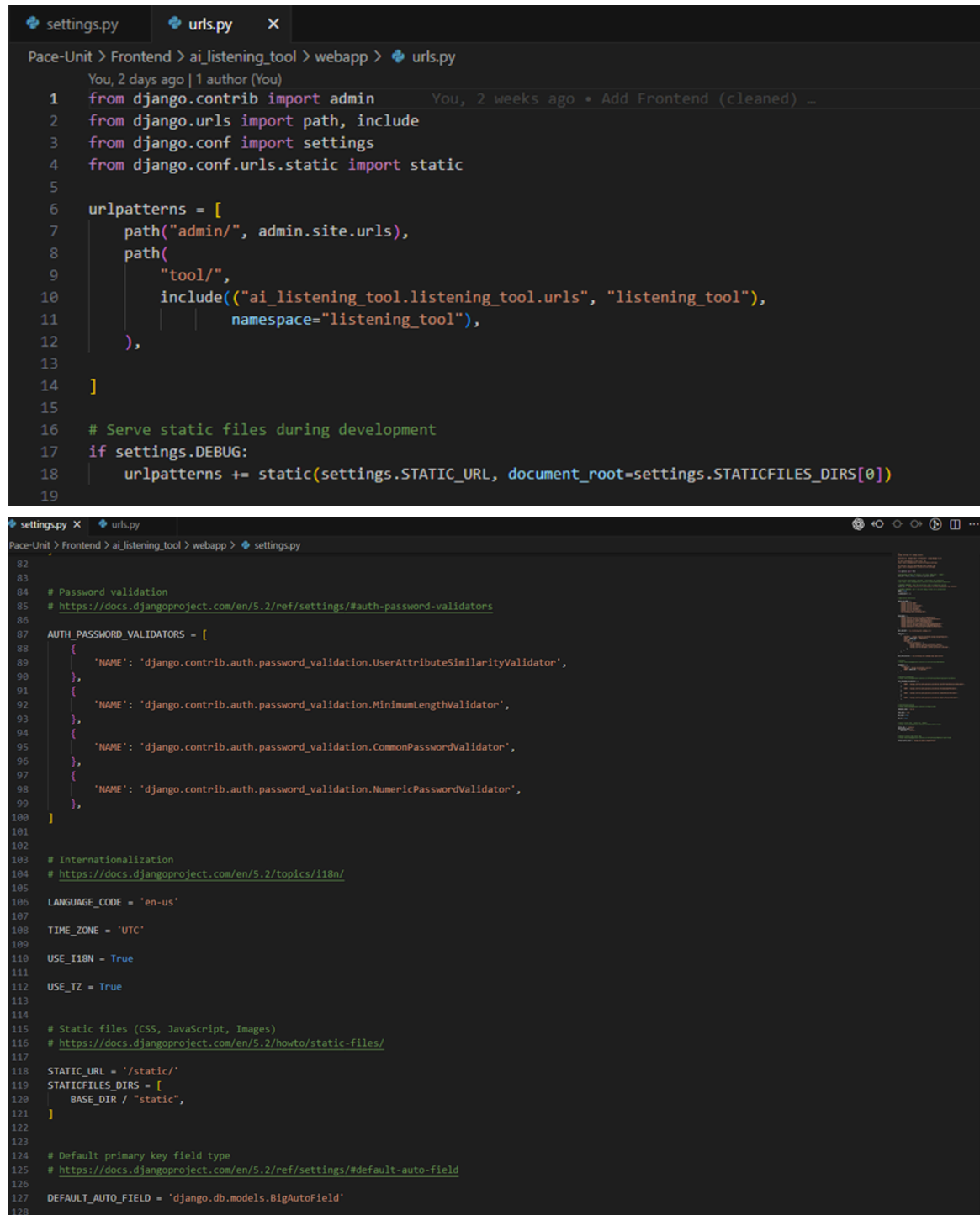
```
JS apijs JS configjs JS historyjs X JS resultsjs JS searchjs
Pace-Unit > Frontend > ai_listening_tool > static > ai_listening_tool > js > JS historyjs > ...
You, 2 weeks ago | 1 author (You)
1 // path: social-listening-tool/ai_listening_tool/static/ai_listening_tool/js/history.js You, 2 weeks ago • Add Frontend (cleaned) ...
2 "use strict";
3
4 // Render one table row
5 function row(i) {
6   const when = i.ts ? new Date(i.ts).toLocaleString() : "";
7   return `<tr><td>${i.subject || "Unknown"}</td><td>${i.count ?? 0}</td><td class="ta-right">${when}</td></tr>`;
8 }
9
10 document.addEventListener("DOMContentLoaded", async () => {
11   const tbody = document.getElementById("rm-history-body");
12   if (!tbody) return;
13   tbody.innerHTML = `<tr><td colspan="3">Loading...</td></tr>`;
14   try {
15     // Uses global helper from api.js
16     const data = await window.apiGet("/history/");
17     const items = Array.isArray(data?.items) ? data.items : [];
18     tbody.innerHTML = items.length ? items.map(row).join("") : `<tr><td colspan="3">No history</td></tr>`;
19   } catch (e) {
20     tbody.innerHTML = `<tr><td colspan="3">Error: ${e.message}</td></tr>`;
21   }
22 });
23
```

```
JS apijs JS configjs X JS historyjs JS resultsjs JS searchjs
Pace-Unit > Frontend > ai_listening_tool > static > ai_listening_tool > js > JS configjs > ...
You, 3 days ago | 1 author (You)
1 function () { You, 2 weeks ago • Add Frontend (cleaned) ...
2 window.RM_TOOL_CONFIG = {
3   API_BASE: "http://127.0.0.1:8000/api", // keep the /api base, no trailing slash
4   timeoutMs: 20000
5 };
6 }();
7
```

```
JS apijs JS configjs JS historyjs JS resultsjs JS searchjs
Pace-Unit > Frontend > ai_listening_tool > static > ai_listening_tool > js > JS apijs > ...
3 (function () {
4   const cfg = (window.RM_TOOL_CONFIG = window.RM_TOOL_CONFIG || {
5     API_BASE: "http://127.0.0.1:8000/api",
6     timeoutMs: 20000,
7   });
8 });
9
10 function withTimeout(promise, ms) {
11   return Promise.race([
12     promise,
13     new Promise(., reject) => setTimeout(() => reject(new Error("timeout")), ms),
14   ]);
15 }
16
17 async function handle(res, method, path) {
18   if (!res.ok) {
19     const text = await res.text().catch(() => "");
20     throw new Error(`${method} ${path} ${res.status} ${text}.trim());
21   }
22   const ct = res.headers.get("content-type") || "";
23   return ct.includes("application/json") ? res.json() : res.text();
24 }
25
26 // Attach to window for use anywhere
27 window.apiGet = async function apiGet(path, qs = {}) {
28   const base = String(cfg.API_BASE || "").replace(/\/$/, "");
29   const url = new URL(`${base}${path}`);
30   Object.entries(qs).forEach(([, v]) => v != null && url.searchParams.set(k, v));
31   const res = await withTimeout(fetch(url.toString(), { credentials: "omit" }), cfg.timeoutMs);
32   return handle(res, "GET", path);
33 };
34
35 window.apiPost = async function apiPost(path, body = {}) {
36   const base = String(cfg.API_BASE || "").replace(/\/$/, "");
37   const res = await withTimeout(
38     fetch(`${base}${path}`, {
39       method: "POST",
40       headers: { "Content-Type": "application/json" },
41       credentials: "omit",
42       body: JSON.stringify(body),
43     }),
44     cfg.timeoutMs
45   );
46   return handle(res, "POST", path);
47 };
48 })();
49
```

Django Wiring:

The UI is mounted at /tool/. webapp/urls.py includes ai_listening_tool.urls. Routes map /tool/ (search), /tool/results/, and /tool/history/. Static files are served via Django staticfiles, and the UI app is in INSTALLED_APPS.



The image shows a code editor with two files open: `urls.py` and `settings.py`. The `urls.py` file is the active tab and contains the following code:

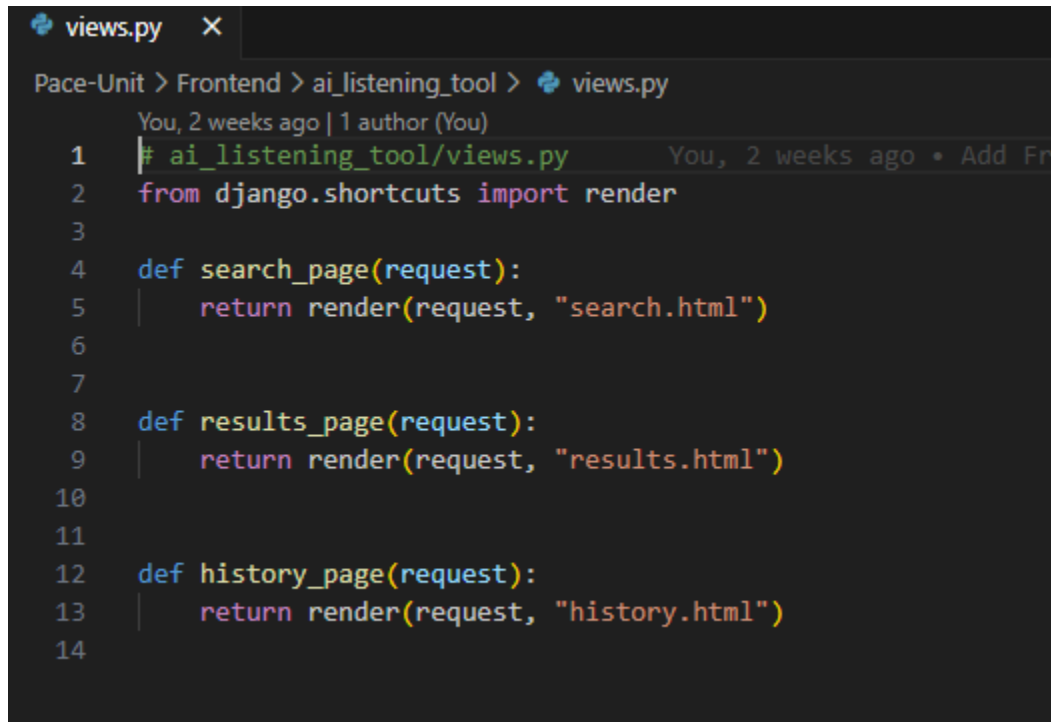
```
1 from django.contrib import admin
2 from django.urls import path, include
3 from django.conf import settings
4 from django.conf.urls.static import static
5
6 urlpatterns = [
7     path("admin/", admin.site.urls),
8     path(
9         "tool/",
10        include(("ai_listening_tool.listening_tool.urls", "listening_tool"),
11              namespace="listening_tool"),
12    ),
13 ]
14
15 # Serve static files during development
16 if settings.DEBUG:
17     urlpatterns += static(settings.STATIC_URL, document_root=settings.STATICFILES_DIRS[0])
18
19
```

The `settings.py` file is also visible and contains the following code:

```
82
83
84 # Password validation
85 # https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators
86
87 AUTH_PASSWORD_VALIDATORS = [
88     {
89         'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
90     },
91     {
92         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
93     },
94     {
95         'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
96     },
97     {
98         'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
99     },
100 ]
101
102 # Internationalization
103 # https://docs.djangoproject.com/en/5.2/topics/i18n/
104
105 LANGUAGE_CODE = 'en-us'
106
107 TIME_ZONE = 'UTC'
108
109 USE_I18N = True
110
111 USE_TZ = True
112
113
114 # Static files (CSS, JavaScript, Images)
115 # https://docs.djangoproject.com/en/5.2/howto/static-files/
116
117 STATIC_URL = '/static/'
118 STATICFILES_DIRS = [
119     BASE_DIR / "static",
120 ]
121
122 # Default primary key field type
123 # https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field
124
125 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
126
127
128
```

Views:

Views are thin. They return the three templates and defer data to backend API endpoints. Page HTML is minimal so JS populates content after each request.

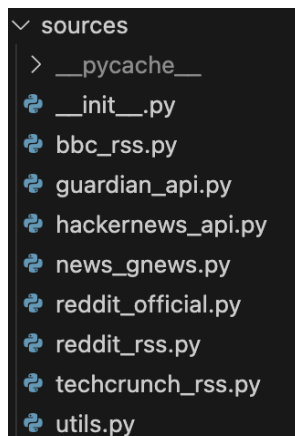


The screenshot shows a code editor with a file named `views.py` open. The breadcrumb navigation at the top indicates the file path: `Pace-Unit > Frontend > ai_listening_tool > views.py`. The code content is as follows:

```
1  # ai_listening_tool/views.py
2  from django.shortcuts import render
3
4  def search_page(request):
5      return render(request, "search.html")
6
7
8  def results_page(request):
9      return render(request, "results.html")
10
11
12  def history_page(request):
13      return render(request, "history.html")
14
```

Backend

The following backend modules implement the data fetching aspect and data persistence to the database for the prototype. Each file represents a key part of the system



The screenshot shows a file explorer view of a folder named `sources`. The files listed inside are:

- `__pycache__`
- `__init__.py`
- `bbc_rss.py`
- `guardian_api.py`
- `hackernews_api.py`
- `news_gnews.py`
- `reddit_official.py`
- `reddit_rss.py`
- `techcrunch_rss.py`
- `utils.py`

Backend/api/sources

The **sources/** folder contains all the data-fetching modules with each external platform (Reddit, BBC, Guardian) having its own dedicated Python file. This modular structure keeps the system clean and flexible. Instead of editing one large script, new sources can be added or existing ones modified simply by creating or updating their individual files.

```

# Backend/api/urls.py
from django.urls import path
from .views import health, search_posts, mongo_status, list_history, get_search_status, get_results
from .views import get_trends
from .views import get_top_topics

urlpatterns = [
    path("health/", health),
    path("search/", search_posts),
    path("debug/mongo/", mongo_status),

    # --- new endpoints ---
    path("history/", list_history),           # GET list of past searches
    path("search/status/", get_search_status), # GET AI progress for a history_id
    path("results/", get_results),           # GET paginated AI results

    # --- for trend analysis ---
    path("trends/", get_trends),

    # for top topic
    path("topics/top/", get_top_topics),
]

```

Backend/api/urls.py

The code defines the backend API routes using Django's `urlpatterns`, mapping each URL to its corresponding function. Core paths like `/health/`, `/search/` and `/debug/mongo/` handle system checks and search operations, while `/history/`, `/search/status/` and `/results/` manage search history, AI progress, and results. Additional endpoints like `/trends/` and `/topics/top/` support trend analysis and topic insights.

```

@api_view(["POST"])
def search_posts(request):
    """
    Pipeline:
    - Create a history record.
    - For each selected source, fetch (default 120).
    - Interleave, dedupe by URL, apply freshness window.
    - Persist up to persist_pool_limit into raw_insights (global cache).
    - Seed ai_results stubs (status='queued') for this history_id.
    - Return preview (first 10) + history_id for FE.
    """
    data = request.data or {}
    subject = (data.get("subject") or "").strip()
    location = (data.get("location") or "").strip()
    industry = (data.get("industry") or "").strip()

    if not subject:
        return Response({"error": "subject is required"}, status=status.HTTP_400_BAD_REQUEST)

    # display preview cap (1..10)
    try:
        preview_limit = int(data.get("limit") or 10)
    except Exception:
        preview_limit = 10
    preview_limit = max(1, min(preview_limit, 10))

    # per-source fetch amount (increase pool)
    try:
        fetch_limit = int(data.get("fetch_limit") or 120)
    except Exception:
        fetch_limit = 120
    fetch_limit = max(10, min(fetch_limit, 200))

    # total to persist into raw_insights
    try:
        persist_pool_limit = int(data.get("persist_pool_limit") or 500)
    except Exception:
        persist_pool_limit = 500
    persist_pool_limit = max(50, min(persist_pool_limit, 2000))

    # freshness window
    try:
        days = int(data.get("days") or 7)
    except Exception:
        days = 7

    # choose sources (default: all)
    selected = _parse_sources_param(data.get("sources") or data.get("source"))
    if not selected:
        selected = list(FETCHERS.keys())

```

Backend/api/views.py

The views.py file defines the core backend logic for handling API requests in the system. The provided code snippet shows the search_posts() function, which manages the main article search workflow. When a user submits a query with parameters such as subject, location or industry, the function creates a new search history record, retrieves data from multiple sources and stores it in the raw_insights collection.

```

# ----- RAW INSIGHTS -----

def persist_raw_insights(rows: Iterable[dict]) -> Tuple[int, int]:
    """
    Upsert rows into 'raw_insights' by unique 'url'.
    Returns (created_count, updated_count).
    """
    db = get_mongo_db()
    col = db["raw_insights"]

    created = 0
    updated = 0
    now = datetime.now(timezone.utc)

    for p in rows:
        url = (p.get("url") or "").strip()
        if not url:
            continue

        # Keep created_at only on first insert
        res = col.update_one(
            {"url": url},
            {
                "$set": {
                    "url": url,
                    "source": (p.get("source") or "").strip(),
                    "title": p.get("title") or "",
                    "text": p.get("text") or "",  # keep this lean (cleaned text)
                    "text_html": p.get("text_html") or "",  # optional; consider omitting if large
                    "author": p.get("author") or None,
                    "published_ts": p.get("published_ts"),
                    "engagement": p.get("engagement") or None,
                    # enrichment candidates (may be None at ingest time)
                    "tags": p.get("tags") if "tags" in p else None,
                    "influencer_mentions": p.get("influencer_mentions") if "influencer_mentions" in p else None,
                    "backlinks": p.get("backlinks") if "backlinks" in p else None,
                },
                "$setOnInsert": {
                    "created_at": now,
                }
            },
            upsert=True
        )
        if res.upserted_id is not None:
            created += 1
        elif res.matched_count:
            updated += 1

    return created, updated

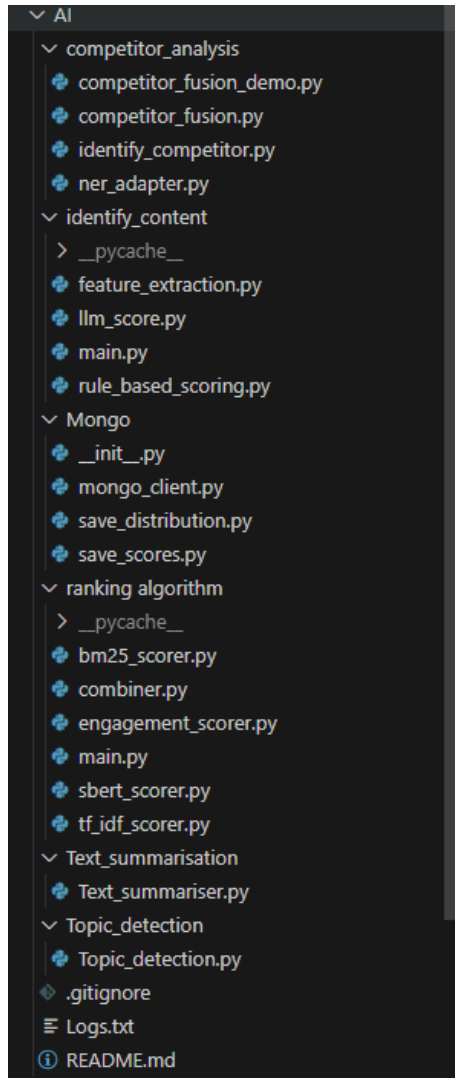
```

Backend/api/persist.py

The persist.py file is responsible for handling database persistence in the backend, specifically writing and updating data in MongoDB collections. The code snippet shown above defines the persist_raw_insights() function, which upserts (inserts or updates) records into the raw_insights collection based on a unique url.

Machine learning

The following backend modules implement the AI-powered scoring pipeline and persistence logic for the prototype. Each file represents a key part of the system



AI/ranking_algorithm

- **tf_idf_scorer.py**
Implements TF-IDF cosine similarity for keyword relevance scoring. Provides functions to score one article against a keyword or a MongoDB document. Supports optional bigram analysis and title boosting.

```

AI > Algorithms > tfidf_scorer.py > tfidf_pair_cosine
1 # AI/Algorithms/tfidf_scorer.py
2 from typing import Dict, Any
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.metrics.pairwise import linear_kernel
5
6 # Module defaults (tune centrally)
7 TITLE_BOOST_DEFAULT = 1.15
8 USE_BIGRAMS_DEFAULT = False
9
10 def _clean_text(x) -> str:
11     if x is None:
12         return ""
13     if not isinstance(x, str):
14         x = str(x)
15     return x.strip()
16
17 def _tfidf_pair_cosine(query: str, text: str, use_bigrams: bool = False) -> float:
18     query = _clean_text(query)
19     text = _clean_text(text)
20     if not query or not text:
21         return 0.0
22     ngram = (1, 2) if use_bigrams else (1, 1)
23     X = TfidfVectorizer(stop_words="english", ngram_range=ngram).fit_transform([query, text])
24     sim = float(linear_kernel(X[0:1], X[1:2]))[0, 0]
25     return max(0.0, min(1.0, sim)) # clip to [0,1]
26
27 def _score_simple_core(keyword: str, title: str, body: str, *, title_boost: float, use_bigrams: bool) -> float:
28     s_title = _tfidf_pair_cosine(keyword, title, use_bigrams)
29     s_body = _tfidf_pair_cosine(keyword, body, use_bigrams)
30     return max(0.0, min(1.0, title_boost * s_title + s_body))
31
32 def score_tfidf_simple(keyword: str, doc: Dict[str, Any]) -> float:
33     title = _clean_text(doc.get("title"))
34     body = _clean_text(doc.get("body"))
35
36     return _score_simple_core(
37         keyword, title, body,
38         title_boost=TITLE_BOOST_DEFAULT,
39         use_bigrams=USE_BIGRAMS_DEFAULT,
40     )
41
42 __all__ = ["score_tfidf_simple"]

```

- **sbert_scorer.py**

Placeholder for semantic similarity scoring using pre-trained Sentence-BERT (SBERT) models. This module will embed queries and documents, then compute cosine similarity between vectors.

```

AI > Algorithms > sbert_scorer.py > ...
1  # AI/Algorithms/sbert_scorer.py
2  """
3  SBERT cosine similarity for a single article vs. a keyword.
4
5  """
6
7  from typing import Dict
8  from sentence_transformers import SentenceTransformer, util
9
10 _model = None
11
12
13 def _get_model() -> SentenceTransformer:
14     global _model
15     if _model is None:
16         _model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
17     return _model
18
19
20 def sbert_score_one(keyword: str, doc: Dict) -> float:
21     """
22     returns a semantic similarity score in [0, 1]
23     """
24     title = (doc.get("title") or "").strip()
25     body = (doc.get("body") or "").strip()
26     text = f"{title}\n{body}".strip()
27     if not text or not keyword:
28         return 0.0
29
30     m = _get_model()
31     q_emb = m.encode(keyword, normalize_embeddings=True)
32     d_emb = m.encode(text, normalize_embeddings=True)
33     sim = float(util.cos_sim(q_emb, d_emb).item()) # [-1, 1]
34     # map to [0, 1]
35     return max(0.0, min((sim + 1.0) / 2.0, 1.0))

```

- **engagement_scorer.py**
Computes heuristic scores from engagement signals such as upvotes, shares, or comments. Normalises raw metrics into a comparable scale for ranking.

```

AI > Algorithms > engagement_scorer.py > ...
1  # AI/Algorithms/engagement_scorer.py
2  """
3  Heuristic engagement score based on likes/comments/shares
4  Scales to [0,1] with a soft saturation to avoid one viral post dominating.
5  """
6
7  from typing import Dict
8
9
10 def engagement_score(doc: Dict) -> float:
11     e = doc.get("engagement") or {}
12     likes = max(0, int(e.get("likes", 0)))
13     comments = max(0, int(e.get("comments", 0)))
14     shares = max(0, int(e.get("shares", 0))) # reposts
15
16     # tune weights to taste / source:
17     raw = likes * 1 + comments * 3 + shares * 4
18
19     # soft-cap (squash) so the function approaches 1.0
20     # increase the constant to make saturation slower.
21     score = raw / (raw + 50.0)
22
23     # clamp
24     if score < 0.0:
25         return 0.0
26     if score > 1.0:
27         return 1.0
28     return float(score)

```

- **bm25_scorer.py**

Provides keyword-based relevance scoring using the BM25 algorithm. It indexes the title and body of each document and computes scores based on term frequency and document length. The output is a fast baseline relevance score that can be combined later with semantic models.

```

khawaja1447, 3 days ago | 1 author (khawaja1447)
# AI/Algorithms/bm25_scorer.py khawaja1447, 2 weeks ago • bm25 ai algorithm ...
from __future__ import annotations
from typing import List, Dict, Any, Iterable, Optional
from rank_bm25 import BM25Okapi
import numpy as np
import re

_word = re.compile(r"[A-Za-z0-9]+")

def _tok(s: str) -> List[str]:
    if not s:
        return []
    return _word.findall(s.lower())

khawaja1447, 3 days ago | 1 author (khawaja1447)
class BM25Scorer:
    """
    Fit once on the current batch of docs, then score a query against all docs.
    Keep an index: doc_id -> position, so we can fetch per-doc scores quickly.
    """

    def __init__(self, bm25: BM25Okapi, id_at_idx: List[Any], idx_of_id: Dict[Any, int]):
        self.bm25 = bm25
        self.id_at_idx = id_at_idx
        self.idx_of_id = idx_of_id

    @classmethod
    def from_docs(cls, docs: List[Dict[str, Any]], *, k1: float = 1.5, b: float = 0.75) -> "BM25Scorer":
        """
        Build a BM25 index over docs (title + body). k1/b are tunable.
        """
        id_at_idx: List[Any] = []
        corpus_tokens: List[List[str]] = []

        for d in docs:
            did = d.get("_id")
            title = (d.get("title") or "").strip()
            body = (d.get("body") or "").strip()
            text = f"{title}\n{body}".strip()
            tokens = _tok(text)
            id_at_idx.append(did)
            corpus_tokens.append(tokens)

        bm25 = BM25Okapi(corpus_tokens, k1=k1, b=b)
        idx_of_id = {did: i for i, did in enumerate(id_at_idx)}
        return cls(bm25=bm25, id_at_idx=id_at_idx, idx_of_id=idx_of_id)

```

```

def score_all_for_query(self, query: str) -> Dict[Any, float]:
    """Return {doc_id: raw_bm25_score} for the query across the whole corpus."""
    qtok = _tok(query or "")
    if not qtok:
        return {did: 0.0 for did in self._id_at_idx}
    scores = self._bm25.get_scores(qtok) # np.ndarray aligned to id_at_idx
    return {did: float(scores[i]) for i, did in enumerate(self._id_at_idx)}

def score_for_doc(self, query: str, doc_id: Any) -> float:
    """Score a single doc by id (convenience)."""
    idx = self._idx_of_id.get(doc_id)
    if idx is None:
        return 0.0
    qtok = _tok(query or "")
    if not qtok:
        return 0.0
    # get_scores returns all; this is still fast for our batch sizes
    scores = self._bm25.get_scores(qtok)
    return float(scores[idx])

def minmax_normalize(score_map: Dict[Any, float]) -> Dict[Any, float]:
    """Scale raw BM25 to 0..1 per batch for easier fusion with SBERT/others."""
    if not score_map:
        return {}
    vals = np.array(list(score_map.values()), dtype=float)
    lo, hi = float(vals.min()), float(vals.max())
    if hi <= lo:
        # all equal; return zeros
        return {k: 0.0 for k in score_map}
    return {k: (v - lo) / (hi - lo) for k, v in score_map.items()}

```

- **combiner.py**
Combines multiple algorithm outputs (TF-IDF, SBERT, engagement) into a final weighted score. Later versions will integrate a neural network for automatic weight optimisation.

```

1 # AI/Algorithms/combiner.py
2 """
3 Weighted fusion of per-algorithm scores into a final score in [0,1]
4 """
5
6 from typing import Dict
7
8
9 def weighted_combine(per_algo: Dict[str, float], weights: Dict[str, float]) -> float:
10     """
11     per_algo: example {"tfidf": 0.42, "sbert": 0.71, "engagement": 0.33}
12     weights: example {"tfidf": 0.35, "sbert": 0.45, "engagement": 0.20}
13     """
14     if not per_algo or not weights:
15         return 0.0
16     num = 0.0
17     den = 0.0
18     for k, w in weights.items():
19         den += float(w)
20         num += float(per_algo.get(k, 0.0)) * float(w)
21     if den <= 0.0:
22         return 0.0
23     out = num / den
24     if out < 0.0:
25         return 0.0
26     if out > 1.0:
27         return 1.0
28     return float(out)
29

```

AI/Mongo

- **mongo_client.py**
Provides a simple MongoDB connection wrapper using credentials from an `.env` file. Exposes helper functions `get_db()` and `get_collection()` to safely retrieve database handles.

```

AI > Mongo > mongo_client.py > ...
1  # AI/Mongo/mongo_client.py
2
3  import os
4  from pathlib import Path
5
6  from dotenv import load_dotenv
7  from pymongo import MongoClient
8
9
10
11
12  # Load the env file that is NEXT TO this script
13  ENV_PATH = Path(__file__).resolve().parent / "mongo_credentials.env"
14  load_dotenv(ENV_PATH)
15
16  _client = None
17  _db = None
18
19  def get_db():
20      global _client, _db
21      if _db is None:
22          uri = os.getenv("MONGODB_URI")
23          dbname = os.getenv("MONGODB_DBNAME", "pace_unit")
24          if not uri:
25              raise RuntimeError(f"MONGODB_URI missing. Expected in {ENV_PATH}")
26          _client = MongoClient(uri)
27          _db = _client[dbname]
28      return _db
29
30  def get_collection(name: str):
31      return get_db()[name]
32

```

- **save_scores.py**

Persists scoring results into MongoDB. Ensures that each document's relevance scores (per-algorithm and final) are saved in a structured format for retrieval and later analysis.

```

AI > Mongo > save_scores.py > ...
1  # AI/Mongo/save_scores.py
2  """
3  Upsert scoring results to MongoDB.
4
5  """
6
7  from typing import Dict, Any
8  from AI.Mongo.mongo_client import get_db
9
10
11 def upsert_scores(doc_id: Any, keyword: str, final_score: float, per_algo: Dict[str, float]) -> None:
12     db = get_db()
13     db["scores"].update_one(
14         {"post_id": doc_id, "keyword": keyword},
15         {
16             "$set": {
17                 "post_id": doc_id,
18                 "keyword": keyword,
19                 "final_score": float(final_score),
20                 "per_algo": {k: float(v) for k, v in (per_algo or {}).items()},
21             }
22         },
23         upsert=True,
24     )

```

ranking_algorithm/main.py

The orchestrator script for algorithm ranking.

- Fetches documents from MongoDB.
- Iterates through them, applying scoring algorithms (TF-IDF, SBERT, engagement).
- Combines per-algorithm scores into a final score.
- Sorts and prints results for inspection.
- Can be run via CLI with arguments for keyword, algorithms, and document limits.

```

AI > main.py > _
12
13 # main.py (project root)
14 from __future__ import annotations
15 from typing import Dict, Any, Callable, List, Tuple
16 import argparse
17 import json
18
19 # Mongo + algorithms
20 from AI.Mongo.mongo_client import get_db
21 from AI.Mongo.save_scores import upsert_scores
22 from AI.Algorithms.tf_idf_scorer import score_tfidf_simple
23 from AI.Algorithms.sbert_scorer import sbert_score_one
24 from AI.Algorithms.engagement_scorer import engagement_score
25 from AI.Algorithms.combiner import weighted_combine
26
27
28 # Algorithm registry
29 AlgorithmFn = Callable[[str, Dict[str, Any]], float]
30
31 ALGORITHMS: Dict[str, AlgorithmFn] = {
32     # "tfidf": lambda keyword, doc: score_one_doc(keyword, doc),
33     "tfidf": lambda keyword, doc: score_tfidf_simple(keyword, doc),
34
35     # "sbert": lambda keyword, doc: sbert_score_one(keyword, doc),
36     "sbert": lambda keyword, doc: sbert_score_one(keyword, doc),
37
38     # "engagement": lambda keyword, doc: engagement_score(doc),
39     "engagement": lambda keyword, doc: engagement_score(doc)
40
41     # TODO: LLM? (ask sponsor)
42 }
43
44 # sensible defaults (tune with sponsor)
45 DEFAULT_WEIGHTS: Dict[str, float] = {"tfidf": 0.35, "sbert": 0.45, "engagement": 0.20}
46
47
48 # Data access - fetch articles from Mongo
49 def fetch_docs(limit: int = 0) -> List[Dict[str, Any]]:
50     db = get_db()
51     cursor = db["posts"].find({}, {"_id": 1, "title": 1, "body": 1, "engagement": 1})
52     if limit > 0:
53         cursor = cursor.limit(limit)
54     return list(cursor)

```

```

56 # runner
57 def run(keyword: str, *, limit: int = 0, algos: List[str] | None = None, weights: Dict[str, float] | None = None, persist: bool = True):
58     docs = fetch_docs(limit)
59     if not docs:
60         print("No documents found in 'posts'.")
61         return []
62
63     # default algo to use is sbert
64     use_algos = algos if algos else ["sbert"]
65
66     # Checking if all the algos are valid (can remove this depending on the approach. Maybe a toggle button? or just have all of them)
67     for a in use_algos:
68         if a not in ALGORITHMS:
69             raise ValueError(f"Unknown algorithm '{a}'. Available: {list(ALGORITHMS)}")
70
71     use_weights = weights or {k: DEFAULT_WEIGHTS.get(k, 0.0) for k in use_algos}
72
73     rows: List[Tuple[float, Dict[str, Any], Dict[str, float]]] = []
74
75     for d in docs:
76         per_algo: Dict[str, float] = {}
77         for a in use_algos:
78             try:
79                 per_algo[a] = float(ALGORITHMS[a](keyword, d))
80             except Exception:
81                 per_algo[a] = 0.0 # fail-safe per-algo
82
83         # TODO: Need to have a NN to combine the scores
84         # simple combiner for now: mean of selected algos
85         final_score = sum(per_algo.get(a, 0.0) * use_weights.get(a, 0.0) for a in use_algos)
86         rows.append((final_score, d, per_algo))
87
88         if persist:
89             try:
90                 upsert_scores(d["_id"], keyword, final_score, per_algo)
91             except Exception as e:
92                 print(f"[WARN] upsert failed for id={d.get('_id')}: {e}")
93
94     rows.sort(key=lambda t: t[0], reverse=True)
95     return rows

```

```

96
97 # ---- CLI ----
98 if __name__ == "__main__":
99     p = argparse.ArgumentParser(description="Score and rank Mongo posts by keyword.")
100     p.add_argument("keyword", type=str, help="Query keyword")
101     p.add_argument("--limit", type=int, default=200, help="Max docs to score (0 = all)")
102     p.add_argument("--algos", nargs="+", default=["sbert", "tfidf", "engagement"], help="Algorithms to run")
103     p.add_argument("--weights", type=str, default="", help="JSON dict, e.g. {'tfidf':0.35,'sbert':0.45,'engagement':0.20}")
104     p.add_argument("--no-persist", action="store_true", help="Do not write scores back to Mongo")
105     args = p.parse_args()
106
107     w = None
108     if args.weights:
109         try:
110             w = json.loads(args.weights)
111         except Exception as e:
112             p.error(f"Invalid --weights JSON: {e}")
113
114     rows = run(
115         args.keyword,
116         limit=args.limit,
117         algos=args.algos,
118         weights=w,
119         persist=not args.no_persist,
120     )
121     rows = rows or []
122     print(json.dumps({"processed": len(rows), "top_score": rows[0][0] if rows else None}))

```

AI/comptetitor_analysis

- Comptetitor_fusion_demo.py

A demo file that can be used to test competitor_fusion.py

```
# AI/ML/competitor_fusion_demo.py
from AI.ML.competitor_fusion import rank_competitors_fused

sample = """
Apple Inc. is facing increasing competition from Samsung and Google in the smartphone market.
Tim Cook, Apple's CEO, mentioned innovation will drive iPhone strategy this year.
Meanwhile, Microsoft is expanding cloud services, competing with Amazon Web Services and Google Cloud.
"""

if __name__ == "__main__":
    results = rank_competitors_fused(
        text=sample,
        seed_brand="Apple",
        industry=["smartphone", "mobile", "device", "cloud"],
        location=["US", "United States"],
        top_n=5,
        min_score=0.25,
    )
    for r in results:
        print(f"{r['name']:20s} score={r['score']:.3f} -> {r['explanation']}")
```

- **Competitor_fusion.py**

Identifies and ranks potential competitor brands by combining multiple scoring signals, including co-mentions in text, competitor-language context, industry/location hints, and optional SBERT name similarity. The module generates a relevance score for each candidate and provides a short explanation for why it was classified as a competitor. This results in a lightweight yet context-aware competitor analysis without requiring full LLM inference.

```

# AI/ML/competitor_fusion.py
from __future__ import annotations
from typing import List, Dict, Any, Tuple, Optional
import math
import re

from AI.ML.ner_adapter import extract_org_candidates

# --- Optional SBERT name-sim boost (safe if not installed) ---
try:
    from sentence_transformers import SentenceTransformer, util
    _SBERT = SentenceTransformer("all-MiniLM-L6-v2")
except Exception:
    _SBERT = None

_COMP_TERMS = {
    "competitor", "competitors", "rival", "rivals", "competing", "competition",
    "versus", "vs", "alternative", "alternatives", "compared to", "against"
}
_WORD = re.compile(r"[A-Za-z0-9]+")

def _sentences(text: str) -> List[str]:
    # Simple splitter; good enough for MVP (you can swap for nltk later)
    return re.split(r'(?<=[.!?])\s+', text.strip())

def _contains_term(s: str, terms: set[str]) -> bool:
    s_low = s.lower()
    return any(t in s_low for t in terms)

def _contains_any(s: str, bag: List[str]) -> bool:
    s_low = s.lower()
    return any(x.lower() in s_low for x in bag if x)

def _name_variants(name: str) -> List[str]:
    # Generate simple brand variants (Apple Inc. -> Apple, Apple Inc)
    n = (name or "").strip()
    bits = _WORD.findall(n)
    variants = [n]
    if bits:
        variants.append(" ".join(bits))
        variants.append(bits[0])
    return list(dict.fromkeys(variants)) # unique, keep order

```

```

def _brand_near_competitor_terms(sent: str, seed_brand: str, candidate: str) -> float:
    """
    Score if seed brand, candidate and competitor terms co-occur in the same sentence.
    """
    s = sent.lower()
    b_hit = any(v.lower() in s for v in _name_variants(seed_brand))
    c_hit = any(v.lower() in s for v in _name_variants(candidate))
    comp = _contains_term(s, _COMP_TERMS)
    return 1.0 if (b_hit and c_hit and comp) else 0.0

def _same_sentence_hits(text: str, seed_brand: str, candidate: str) -> Tuple[int, int]:
    """
    Count sentences that include both names (with/without explicit competitor terms).
    """
    both = 0
    both_and_comp = 0
    for sent in _sentences(text):
        s = sent.lower()
        b_hit = any(v.lower() in s for v in _name_variants(seed_brand))
        c_hit = any(v.lower() in s for v in _name_variants(candidate))
        if b_hit and c_hit:
            both += 1
            if _contains_term(s, _COMP_TERMS):
                both_and_comp += 1
    return both, both_and_comp

def _industry_location_boost(text: str, *, industry: Optional[List[str]], location: Optional[List[str]]) -> float:
    """
    Lightweight boost if article contains industry/location hints.
    """
    score = 0.0
    if industry:
        score += 0.15 if _contains_any(text, industry) else 0.0
    if location:
        score += 0.10 if _contains_any(text, location) else 0.0
    return min(0.25, score)

def _domain_hint(candidate: str) -> float:
    """
    If the candidate looks like a brand (single capitalized token or proper-case), tiny boost.
    """
    cand = candidate.strip()
    if not cand:
        return 0.0
    # Heuristic: Title Case single/multi-word brand-ish names
    tokens = cand.split()
    if all(t[:1].isupper() for t in tokens if t):
        return 0.1
    return 0.0

```

```

def _sbert_name_sim(seed_brand: str, candidate: str) -> float:
    """
    Optional tiny boost for name-kind similarity (company vs company) using SBERT.
    NOT semantic of the full article; just name sim to avoid false positives.
    """
    if not _SBERT:
        return 0.0
    emb = _SBERT.encode([seed_brand, candidate], normalize_embeddings=True)
    sim = float(util.cos_sim(emb[0], emb[1]).item())
    # Map [-1..1] to [0..1]
    return (sim + 1.0) / 2.0

def _clip01(x: float) -> float:
    return 0.0 if x < 0 else 1.0 if x > 1 else x

def rank_competitors_fused(
    *,
    text: str,
    seed_brand: str,
    industry: Optional[List[str]] = None,
    location: Optional[List[str]] = None,
    extra_candidates: Optional[List[str]] = None,
    top_n: int = 10,
    min_score: float = 0.25,
    weights: Dict[str, float] = None,
) -> List[Dict[str, Any]]:
    """
    Fusion competitor identifier:
    - Candidate orgs from your friend's spaCy NER
    - Scored by your richer competitor signals
    - Optional SBERT name similarity
    Returns list of {name, score, signals, explanation}
    """
    if not seed_brand or not text:
        return []

    # (1) Candidate generation
    friend_orgs = extract_org_candidates(text)
    candidates = friend_orgs[:]
    if extra_candidates:
        for c in extra_candidates:
            if c and c not in candidates:
                candidates.append(c)

    # Deduplicate (case-insensitive), drop the seed brand itself
    seen = set()
    uniq: List[str] = []
    seed_low = seed_brand.lower().strip()
    for c in candidates:
        lc = c.lower().strip()
        if lc and lc != seed_low and lc not in seen:
            seen.add(lc)
            uniq.append(c)

```

```

# 2) Weights (tune later)
W = {
    "same_sentence": 0.45,      # seed & candidate co-mentioned
    "comp_terms": 0.20,       # co-mention with competitor terms
    "industry_loc": 0.15,     # industry/location hints present
    "domain_hint": 0.05,      # brand-ish name
    "sbert_name": 0.15,       # optional name similarity
}

if weights:
    W.update(weights)

# 3) Score each candidate
out: List[Dict[str, Any]] = []
for cand in uniq:
    both, both_comp = _same_sentence_hits(text, seed_brand, cand)
    ss = min(1.0, both / 2.0)    # cap at 2 sentences → 1.0
    cc = min(1.0, both_comp / 1.0) # any 1+ comp-term sentence → 1.0
    il = _industry_location_boost(text, industry=industry, location=location)
    dh = _domain_hint(cand)
    sb = _sbert_name_sim(seed_brand, cand)

    score = (
        W["same_sentence"] * ss +
        W["comp_terms"] * cc +
        W["industry_loc"] * il +
        W["domain_hint"] * dh +
        W["sbert_name"] * sb
    )
    score = _clip01(score)

    # Build an explanation
    reasons = []
    if ss >= 0.5: reasons.append(f"co-mentioned with {seed_brand} in multiple sentences")
    elif ss > 0: reasons.append(f"co-mentioned with {seed_brand}")
    if cc > 0: reasons.append("appears near competitor language")
    if il > 0: reasons.append("matches industry/location context")
    if dh > 0: reasons.append("brand-like name")
    if sb >= 0.6: reasons.append("name similarity to peer brands")

    out.append({
        "name": cand,
        "score": round(float(score), 4),
        "signals": {
            "same_sentence": ss, "comp_terms": cc, "industry_loc": il,
            "domain_hint": dh, "sbert_name": sb
        },
        "explanation": "; ".join(reasons) if reasons else "weak evidence",
    })

out.sort(key=lambda r: r["score"], reverse=True)
if min_score is not None:
    out = [r for r in out if r["score"] >= min_score]
return out[:stop_n]

```

- **Identify_competitor.py**

Extracts and categorizes key entities (organizations, people, products, and locations) from text using spaCy's named-entity recognition. It also ranks the most frequently mentioned players and can identify competitor relationships based on

contextual co-occurrence. This provides a structured overview of influential actors and their roles in the article.

```
from typing import List, Dict, Set

You, 4 days ago | 1 author (You)
class KeyPlayerIdentifier:
    def __init__(self, model_name: str = "en_core_web_sm"):
        """
        Initialize the Key Player Identifier with a spaCy model.

        Args:
            model_name: spaCy model to use (default: en_core_web_sm)
        """
        try:
            self.nlp = spacy.load(model_name)
        except OSError:
            print(f"Model '{model_name}' not found. Downloading...")
            import subprocess
            subprocess.run(["python", "-m", "spacy", "download", model_name])
            self.nlp = spacy.load(model_name)

    def identify_key_players(self, text: str) -> Dict[str, List[Dict]]:
        """
        Identify key players (organizations, people, products) from text.

        Args:
            text: Input text to analyze

        Returns:
            Dictionary containing identified entities by category
        """
        doc = self.nlp(text)

        entities = {
            'organizations': [],
            'people': [],
            'products': [],
            'locations': [],
            'other': []
        }

        entity_counts = Counter()

        for ent in doc.ents:
            entity_info = {
                'text': ent.text,
                'label': ent.label_,
                'start': ent.start_char,
                'end': ent.end_char
            }

            # Categorize entities
            if ent.label_ in ['ORG', 'COMPANY']:
                entities['organizations'].append(entity_info)
                entity_counts[ent.text] += 1
            elif ent.label_ in ['PERSON']:
                entities['people'].append(entity_info)
                entity_counts[ent.text] += 1
            elif ent.label_ in ['PRODUCT']:
                entities['products'].append(entity_info)
                entity_counts[ent.text] += 1
            elif ent.label_ in ['GPE', 'LOC']:
                entities['locations'].append(entity_info)
            else:
                entities['other'].append(entity_info)

            # Remove duplicates while preserving order
            for category in entities:
                seen = set()
                unique_entities = []
                for entity in entities[category]:
                    if entity['text'] not in seen:
                        seen.add(entity['text'])
                        unique_entities.append(entity)
                entities[category] = unique_entities

        return entities
```

```

def rank_key_players(self, text: str, top_n: int = 10) -> List[Dict]:
    """
    Identify and rank key players by frequency and importance.

    Args:
        text: Input text to analyze
        top_n: Number of top players to return

    Returns:
        List of top key players ranked by mentions
    """
    doc = self.nlp(text)

    entity_counts = Counter()
    entity_details = {}

    for ent in doc.ents:
        if ent.label_ in ['ORG', 'COMPANY', 'PERSON', 'PRODUCT']:
            entity_counts[ent.text] += 1
            if ent.text not in entity_details:
                entity_details[ent.text] = {
                    'type': ent.label_,
                    'contexts': []
                }

            # Get context around the entity
            start = max(0, ent.start - 5)
            end = min(len(doc), ent.end + 5)
            context = doc[start:end].text
            entity_details[ent.text]['contexts'].append(context)

    # Create ranked list
    ranked_players = []
    for entity, count in entity_counts.most_common(top_n):
        ranked_players.append({
            'name': entity,
            'mentions': count,
            'type': entity_details[entity]['type'],
            'sample_contexts': entity_details[entity]['contexts'][:3]
        })

    return ranked_players

def identify_competitors(self, text: str, company_name: str = None) -> Dict:
    """
    Identify potential competitors mentioned in relation to a company.

    Args:
        text: Input text to analyze
        company_name: Optional company name to find competitors for

    Returns:
        Dictionary containing competitor information
    """
    entities = self.identify_key_players(text)

    competitors = {
        'organizations': [],
        'competitive_keywords': []
    }

    # Competitive keywords to look for
    competitive_terms = [
        'competitor', 'rival', 'competing', 'competition',
        'versus', 'vs', 'alternative', 'compared to', 'against'
    ]

    # Find organizations
    competitors['organizations'] = entities['organizations']

    # Find competitive mentions
    doc = self.nlp(text.lower())
    for token in doc:
        if token.text in competitive_terms or token.lemma_ in competitive_terms:
            context_start = max(0, token.i - 10)
            context_end = min(len(doc), token.i + 10)
            context = doc[context_start:context_end].text

            competitors['competitive_keywords'].append({
                'keyword': token.text,
                'context': context
            })

```

```

        if company_name:
            # Filter to find organizations mentioned near the company
            competitors['related_to_company'] = self._find_related_entities(
                text, company_name
            )

    return competitors

def _find_related_entities(self, text: str, company_name: str) -> list[Dict]:
    """
    Find entities mentioned in relation to a specific company.

    Args:
        text: Input text
        company_name: Company name to search for

    Returns:
        List of related entities
    """
    doc = self.nlp(text)
    related_entities = []

    # Find sentences mentioning the company
    for sent in doc.sents:
        if company_name.lower() in sent.text.lower():
            # Extract other organizations from these sentences
            for ent in sent.ents:
                if ent.label_ in ['ORG', 'COMPANY'] and ent.text.lower() != company_name.lower():
                    related_entities.append({
                        'name': ent.text,
                        'context': sent.text
                    })

    return related_entities

def extract_key_player_summary(self, text: str) -> Dict:
    """
    Generate a comprehensive summary of all key players.

    Args:
        text: Input text to analyze

    Returns:
        Comprehensive summary dictionary
    """
    entities = self.identify_key_players(text)
    ranked = self.rank_key_players(text)

    summary = {
        'total_entities': sum(len(v) for v in entities.values()),
        'by_category': {
            category: len(items) for category, items in entities.items()
        },
        'top_players': ranked[:5],
        'all_entities': entities
    }

```

```

def main():
    """Example usage of the KeyPlayerIdentifier"""

    # Sample text
    sample_text = """
    Apple Inc. is facing increasing competition from Samsung and Google in the smartphone market.
    Tim Cook, Apple's CEO, mentioned that the company is focusing on innovation.
    Meanwhile, Microsoft under Satya Nadella is expanding its cloud services,
    competing with Amazon Web Services and Google Cloud. Tesla, led by Elon Musk,
    continues to dominate the electric vehicle market, but traditional automakers
    like Ford and General Motors are catching up with their EV offerings.
    """

    # Initialize Identifier
    identifier = KeyPlayerIdentifier()

    # Identify all key players
    print("-" * 60)
    print("KEY PLAYERS IDENTIFIED")
    print("-" * 60)
    entities = identifier.identify_key_players(sample_text)

    for category, items in entities.items():
        if items:
            print(f"\n[category.upper()]:")
            for item in items:
                print(f"  - {item['text']} ({item['label']})")

    # Get ranked players
    print("\n" + "-" * 60)
    print("TOP RANKED PLAYERS")
    print("-" * 60)
    ranked = identifier.rank_key_players(sample_text, top_n=5)

    for i, player in enumerate(ranked, 1):
        print(f"\n{i}. {player['name']}")
        print(f"   Type: {player['type']}")
        print(f"   Mentions: {player['mentions']}")
        print(f"   Sample context: {player['sample_contexts'][0][:80]}...")

    # Identify competitors for Apple
    print("\n" + "-" * 60)
    print("COMPETITORS ANALYSIS FOR APPLE")
    print("-" * 60)
    competitors = identifier.identify_competitors(sample_text, "Apple")

    print(f"\nOrganizations mentioned: {len(competitors['organizations'])}")
    for org in competitors['organizations']:
        print(f"  - {org['text']}")

    if competitors.get('related_to_company'):
        print(f"\nOrganizations mentioned with Apple:")
        for entity in competitors['related_to_company']:
            print(f"  - {entity['name']}")

    # Generate summary
    print("\n" + "-" * 60)
    print("SUMMARY")
    print("-" * 60)
    summary = identifier.extract_key_player_summary(sample_text)

    print(f"\nTotal entities found: {summary['total_entities']}")
    print(f"\nBreakdown by category:")
    for category, count in summary['by_category'].items():
        if count > 0:
            print(f"  {category}: {count}")

if __name__ == "__main__":
    main()

```

- **ner_adaptor.py**

Provides a lightweight wrapper around the KeyPlayerIdentifier to extract unique organization names from text using spaCy NER. It filters and deduplicates entity mentions to produce a clean list of potential organization candidates. This list is then used as input for downstream competitor ranking modules.

```

from __future__ import annotations
from typing import List, Set
from identify_competitors import KeyPlayerIdentifier

def extract_org_candidates(text: str, *, spacy_model: str = "en_core_web_sm") -> List[str]:
    kpi = KeyPlayerIdentifier(model_name=spacy_model)
    ents = kpi.identify_key_players(text)
    orgs = [e["text"].strip() for e in ents.get("organizations", []) if e.get("text")]
    seen: Set[str] = set()
    out: List[str] = []
    for o in orgs:
        lo = o.lower()
        if lo and lo not in seen:
            seen.add(lo)
            out.append(o)
    return out

```

AI/identify_content

- **Feature_extraction.py**

Computes structural, engagement, and formatting features from an article's title and body text, including word counts, media indicators, urgency signals, and social/context cues. It also aggregates basic engagement metrics such as likes, comments, and shares. These features are later used to support relevance scoring and content repurposing decisions.

```

...
Extract everything from mongoDB and then assign variables to each feature
...

title = article.get("title", "")
body = article.get("body", "")
engagement = article.get("engagement", {})

full_text = f"{title} {body}"

# Extract Engagement metrics
Implement with Codex
# TODO: update with the new extraction
if isinstance(engagement, dict):
    likes = engagement.get("likes", 0)
    comments = engagement.get("comments", 0)
    shares = engagement.get("shares", 0)
    total_engagement = likes + comments + shares
else:
    total_engagement = 0

...

Calculate metrics based on the features
...

# Text length features
word_count = len(full_text.split())
char_count = len(full_text)

# check for images/videos/links
has_images = "image" in body.lower() or "photo" in body.lower() or "img" in body.lower()
has_video = "video" in body.lower() or "watch" in body.lower()
has_links = "http" in body or "www." in body

# check for urgency indicators
urgency_keywords = ["breaking", "urgent", "alert", "now", "today", "just in", "update"]
is_urgent = any(keyword in full_text.lower() for keyword in urgency_keywords)

# long-form indicators
has_sections = any(marker in body for marker in ["\n\n", "##", "###"])
has_detailed_analysis = word_count > 500

# Social media indicators
has_hashtags = "#" in full_text
has_mentions = "@" in full_text
is_short_form = word_count < 150

# email indicators
is_personalized = any(word in full_text.lower() for word in ["you", "your", "subscribe"])
has_cta = any(phrase in full_text.lower() for phrase in ["click", "read more", "learn more", "sign up"])

return {
    "word_count": word_count,
    "char_count": char_count,
    "total_engagement": total_engagement,
    "has_images": has_images,
    "has_video": has_video,
    "has_links": has_links,
    "is_urgent": is_urgent,
    "has_sections": has_sections,
    "has_detailed_analysis": has_detailed_analysis,
    "has_hashtags": has_hashtags,
    "has_mentions": has_mentions,
    "is_short_form": is_short_form,
    "is_personalized": is_personalized,
    "has_cta": has_cta,
}

```

- **llm_score.py**

Uses an LLM to analyze article content and extracted features to suggest the most suitable distribution channels (e.g., blog, LinkedIn, newsletter, email). The model outputs ranked recommendations with reasoning in structured JSON format. This

enables strategic repurposing of content based on tone, structure, and engagement potential.

```
from openai import OpenAI
import json
from typing import Dict, Tuple, List, Any, Optional

def recommend_distribution_channels(features: dict, article_text: str) -> list[dict]:
    client = OpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key="API KEY HERE"
    )

    prompt = f"""
    You are an expert digital content strategist.
    Analyze the following article and its metadata to recommend the best
    distribution channels (e.g., company blog, newsletter, LinkedIn post,
    X/Twitter, Reddit, email to clients, etc.).

    Respond **only in valid JSON** using this exact schema:
    {{
      "recommendations": [
        {{
          "channel": "string",
          "score": float,
          "reason": "string"
        }}
      ]
    }}

    ### Metadata:
    {json.dumps(features, indent=2)}

    ### Content:
    {article_text[:2000]}
    """

    response = client.chat.completions.create(
        model="meta-llama/llama-3-70b-instruct", # or any other model you prefer
        messages=[{"role": "user", "content": prompt}]
    )

    raw_output = response.choices[0].message.content.strip()

    # Attempt to safely parse model output as JSON
    try:
        parsed = json.loads(raw_output)
        return parsed.get("recommendations", [])
    except json.JSONDecodeError:
        print("Model did not return valid JSON. Raw output:")
        print(raw_output)
        return []
```

- [main.py](#)
Acts as the main execution pipeline that extracts features from an article and then routes scoring either to the rule-based method or to the LLM-powered recommendation module, depending on the selected mode. This allows flexible evaluation of content using either

lightweight deterministic scoring or richer model-driven insights. It also includes a simple example in `main()` for testing end-to-end functionality.

```
from typing import Dict, Tuple, List, Any, Optional
from feature_extraction import feature_extraction
from llm_score import recommend_distribution_channels
from rule_based_scoring import rule_based_scores

def runner(article: Dict[str, Any], use_llm: bool):
    features = feature_extraction(article)
    if use_llm:
        llm_prediction = recommend_distribution_channels(features, article)
        print(llm_prediction)
        # return llm_prediction
    else:
        score = rule_based_scores(features)
        print(score)
        # return score

def main():
    article = {
        "title": "Breaking: New AI Technology Transforms Content Marketing",
        "body": "A revolutionary AI-powered content marketing platform has just launched, promising to change how businesses create and distribute content. The platform uses advanced machine learning to analyze audience preferences and automa",
        "engagement": {
            "likes": 150,
            "comments": 45,
            "shares": 80
        }
    }
    runner(article, use_llm=True)

if __name__ == "__main__":
    main()
```

- **rule_based_scoring.py**

Implements a deterministic scoring system that maps article features to recommended distribution channels such as email, blog, newsletter, Instagram, and social media. Scores are assigned based on content length, structure, urgency, media presence, and engagement signals. The output provides a quick, interpretable baseline alternative to LLM-based recommendations.

```

from typing import Dict, Tuple, List, Any, Optional

# Distribution channels we support
CHANNELS = ["email", "newsletter", "blog", "instagram", "social_media"]

def rule_based_scores(features: Dict[str, Any]) -> Dict[str, float]:
    scores = {channel: 0.0 for channel in CHANNELS}

    # EMAIL: Long-form, personalized, high-value content with CTA
    scores["email"] = 0.0
    if features["word_count"] > 300:
        scores["email"] += 0.3
    if features["has_detailed_analysis"]:
        scores["email"] += 0.2
    if features["is_personalized"]:
        scores["email"] += 0.2
    if features["has_cta"]:
        scores["email"] += 0.2
    if features["has_sections"]:
        scores["email"] += 0.1

    # NEWSLETTER: Curated, timely, medium-length content
    scores["newsletter"] = 0.0
    if 200 <= features["word_count"] <= 800:
        scores["newsletter"] += 0.3
    if features["is_urgent"]:
        scores["newsletter"] += 0.2
    if features["has_links"]:
        scores["newsletter"] += 0.2
    if features["total_engagement"] > 50:
        scores["newsletter"] += 0.2
    if not features["is_short_form"]:
        scores["newsletter"] += 0.1

    # BLOG: Evergreen, long-form, detailed content
    scores["blog"] = 0.0
    if features["word_count"] > 500:
        scores["blog"] += 0.4
    if features["has_sections"]:
        scores["blog"] += 0.2
    if features["has_detailed_analysis"]:
        scores["blog"] += 0.2
    if features["has_images"] or features["has_video"]:
        scores["blog"] += 0.1
    if not features["is_urgent"]: # Evergreen content
        scores["blog"] += 0.1

    # INSTAGRAM: Visual, short, engaging content
    scores["instagram"] = 0.0
    if features["has_images"] or features["has_video"]:
        scores["instagram"] += 0.4
    if features["is_short_form"]:
        scores["instagram"] += 0.3
    if features["has_hashtags"]:
        scores["instagram"] += 0.2
    if features["total_engagement"] > 100:
        scores["instagram"] += 0.1

    # SOCIAL MEDIA (Twitter/LinkedIn): Short, urgent, engaging
    scores["social_media"] = 0.0
    if features["is_short_form"]:
        scores["social_media"] += 0.3
    if features["is_urgent"]:
        scores["social_media"] += 0.3
    if features["has_hashtags"] or features["has_mentions"]:
        scores["social_media"] += 0.2
    if features["total_engagement"] > 50:
        scores["social_media"] += 0.1
    if features["has_links"]:
        scores["social_media"] += 0.1

    # Normalize to 0-1 range
    for channel in scores:
        scores[channel] = min(1.0, max(0.0, scores[channel]))

    return scores

```

AI/Text_summarisation

- Text_summariser.py

Provides an text summarization utility using a transformer-based model (BART). The summarizer pipeline is loaded lazily and cached to avoid repeated model downloads, and long texts are truncated safely before processing. This enables

fast generation of concise summaries for article preview and content insight cards.

```
try:
    from transformers import pipeline
except Exception:
    pipeline = None

# Lazily-initialized cached summarizer to avoid reloading the model on every call
SUMMARIZER = None

def _get_summarizer(model_name="facebook/bart-large-cnn"):
    """Return a cached summarizer pipeline. Raises RuntimeError if transformers is not available."""
    global SUMMARIZER
    if pipeline is None:
        raise RuntimeError("transformers is not installed or failed to import. Install 'transformers' to use generate_summary.")
    if SUMMARIZER is None:
        SUMMARIZER = pipeline("summarization", model=model_name)
    return SUMMARIZER

def generate_summary(text, max_length=130, min_length=30, model_name="facebook/bart-large-cnn"):
    """Generate a short summary for 'text'."""

    Notes:
    - The heavy model is cached after the first call.
    - If the input is much longer than the model's context, it will be truncated at ~1024 words.
    """
    summarizer = _get_summarizer(model_name=model_name)

    # Handle long texts by chunking/truncating if needed (word-based simple truncation)
    max_input_length = 1024
    if len(text.split()) > max_input_length:
        text = " ".join(text.split()[:max_input_length])

    # Adjust max_length if input is shorter to avoid warnings
    input_length = len(text.split())
    adjusted_max_length = min(max_length, int(input_length * 0.9)) if input_length < max_length else max_length
    adjusted_min_length = min(min_length, adjusted_max_length - 10)
    # Okay, says docs: * opportunistically!
    summary = summarizer(text, max_length=adjusted_max_length, min_length=adjusted_min_length, do_sample=False)
    return summary[0]["summary_text"]
```

AI/Topic_detection

- **Topic_detection.py**

Performs zero-shot topic classification using a transformer model (BART-MNLI), allowing the system to infer the article's most relevant topic without predefined training data. The model is loaded lazily and cached to improve performance, and text is truncated safely for long inputs. A simplified helper function is included for quickly returning only the top topic label.

```

try:
    from transformers import pipeline
except Exception:
    pipeline = None

# Lazily-initialized cached classifier to avoid reloading the model on every call
_CLASSIFIER = None

def _get_classifier(model_name="facebook/bart-large-enli"):
    """Return a cached zero-shot classification pipeline. Raises RuntimeError if transformers is not available."""
    global _CLASSIFIER
    if pipeline is None:
        raise RuntimeError("transformers is not installed or failed to import. Install 'transformers' to use detect_topic.")
    if _CLASSIFIER is None:
        _CLASSIFIER = pipeline("zero-shot-classification", model=model_name)
    return _CLASSIFIER

def detect_topic(text, candidate_labels=None, model_name="facebook/bart-large-enli", multi_label=False):
    """Detect the topic of the given text using zero-shot classification.

    Args:
        text (str): The text to classify.
        candidate_labels (list): List of possible topic labels. If None, uses default topics.
        model_name (str): The model to use for classification.
        multi_label (bool): If True, allows multiple topics to be assigned to the text.

    Returns:
        dict: A dictionary containing:
            - 'labels': List of labels sorted by score (highest first)
            - 'scores': List of confidence scores corresponding to each label
            - 'top_topic': The most likely topic You, 4 days ago • PL Draft2
            - 'top_score': The confidence score of the top topic

    Notes:
        - The heavy model is cached after the first call.
        - Default topics cover common categories like technology, sports, politics, etc.
    """
    classifier = _get_classifier(model_name=model_name)

    # Default topic labels if none provided
    if candidate_labels is None:
        candidate_labels = [
            "technology",
            "sports",
            "politics",
            "business",
            "entertainment",
            "science",
            "health",
            "education",
            "environment",
            "travel"
        ]

    # Truncate text if it's too long (similar to summarizer)
    max_input_length = 512 # Tokens, roughly ~400 words
    if len(text.split()) > max_input_length:
        text = " ".join(text.split()[:max_input_length])

    # Perform classification
    result = classifier(text, candidate_labels, multi_label=multi_label)

    return {
        "labels": result["labels"],
        "scores": result["scores"],
        "top_topic": result["labels"][0],
        "top_score": result["scores"][0]
    }

def detect_topic_simple(text, candidate_labels=None, model_name="facebook/bart-large-enli"):
    """Simplified version that returns just the top topic as a string.

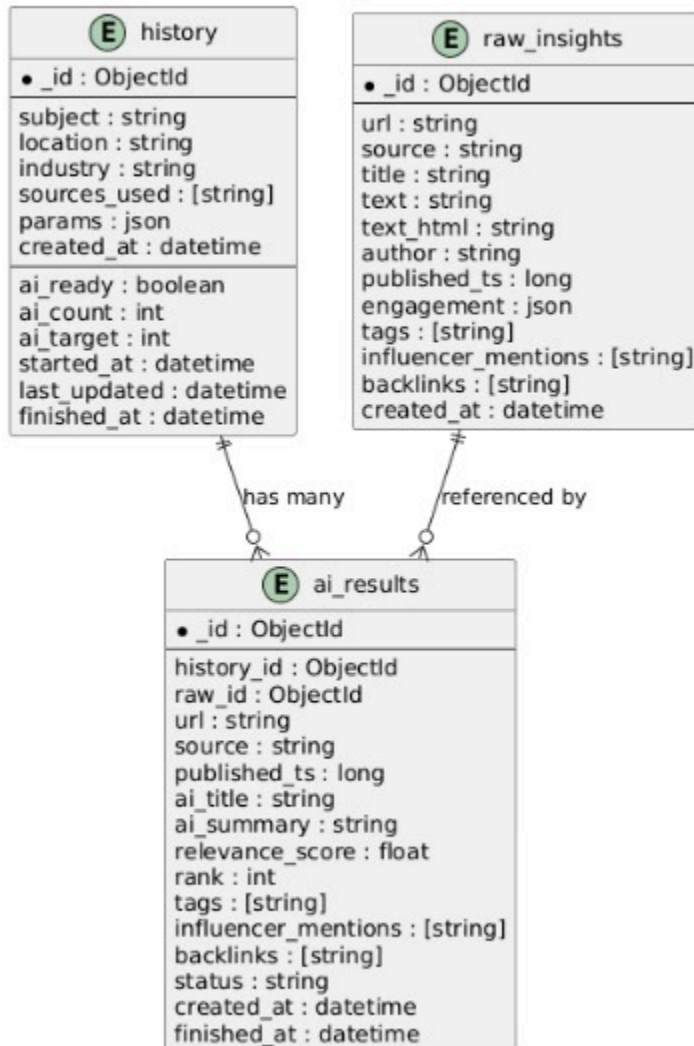
    Args:
        text (str): The text to classify.
        candidate_labels (list): List of possible topic labels. If None, uses default topics.
        model_name (str): The model to use for classification.

    Returns:
        str: The most likely topic.
    """
    result = detect_topic(text, candidate_labels=candidate_labels, model_name=model_name)
    return result["top_topic"]

```

3.3 Database Schema

Backend Data Schema (MongoDB Collections)



This diagram above illustrates the three core MongoDB collections that structure the backend data layer of the AI-Powered Discovery Bot system.

- 1) **history** stores user search metadata and AI processing status (e.g., subject, parameters, timestamps, and progress tracking).

Relationship: One history -> many ai_results (each run can yield many scored items).

- 2) **raw_insights** contains the raw fetched data from various sources such as Reddit or news APIs, including article text, engagement metrics, and enrichment fields like tags, backlinks and influencer mentions.

Relationship: Each `ai_results` references exactly one `raw_insights` via `raw_id` (provenance for every ranked item).

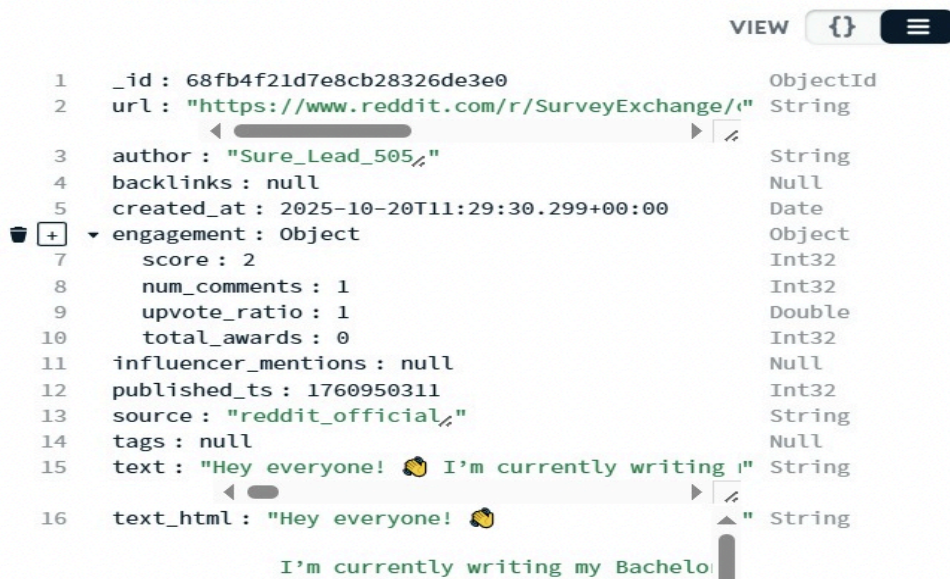
- 3) **ai_results** holds the processed outputs generated by the AI module (ranked, summarised and tagged insights) linked to both their originating history and `raw_insights` records.

Cardinality: Many `ai_results` per history and one `ai_results` per `raw_insights` per run (a raw item can appear in multiple runs, which produces multiple `ai_results` rows pointing to the same `raw_insights._id`).

Together, these collections form a complete pipeline:

User search -> Data fetch (raw_insights) -> AI processing (ai_results). All tracked through a central **history** record.

To collection raw_insights



Content structure in raw_insights

The screenshot above shows a document from the `raw_insights` collection in the MongoDB Atlas database. This collection stores the unprocessed article and post data fetched directly from external sources such as Reddit or news APIs. Each document contains metadata including the author, source, publication timestamp, engagement metrics (score, comments, upvotes) and the full text content of the post. This serves as the foundation for the following AI processing and summarisation in the pipeline.

To collection ai_results

		VIEW		
1	_id : 68fb4f46d7e8cb28326de3e1		ObjectId	
2	url : "https://www.reddit.com/r/Edchart/comments/]"		String	
3	history_id : 68f8d99614be04fbbd080a25		ObjectId	
4	created_at : 2025-10-22T13:18:20.278+00:00		Date	
5	published_ts : 1761138569		Int32	
6	raw_id : 68f8d9403483cbe90adca239		ObjectId	
7	source : "reddit_official"		String	
8	status : "queued"		String	

Content structure in ai_results collection

The screenshot above displays a record from the ai_results collection in MongoDB Atlas. This collection stores the processed output generated by the AI module for each search. Each document is linked to its corresponding history_id and raw_id, which allows traceability back to the user query and its original data source. The fields include the source platform, publication timestamp and processing status ("queued" or "done") which indicates whether the AI has completed ranking and summarisation for that entry.

To collection history

		VIEW		
1	_id : 68fb4ed04552d2c80db62d3d		ObjectId	
2	subject : "Artificial intelligence"		String	
3	location : "		String	
4	industry : "		String	
5	sources_used : Array (1)		Array	
6	params : Object		Object	
7	created_at : 2025-10-23T11:40:19.032+00:00		Date	
8	ai_ready : true		Boolean	
9	ai_count : 72		Int32	
10	ai_target : 100		Int32	
11	started_at : 2025-10-23T11:40:24.766+00:00		Date	
12	last_updated : 2025-10-23T11:40:25.153+00:00		Date	
13	finished_at : 2025-10-23T11:40:25.153+00:00		Date	

Content structure in history collection

The screenshot above shows a record from the history collection in MongoDB Atlas. This collection logs every user search request made through the system which stores key parameters such as subject, location and industry. It also tracks the lifecycle of each search, including timestamps for when AI processing started, last updated and finished. Fields such as ai_count, ai_target and ai_ready provide real-time progress updates which ensures traceability for the user.

3.4 Demonstration Scripts / Workflow

Set up:

Frontend: <http://127.0.0.1:8001/tool/>

Backend API: <http://127.0.0.1:8000/api>

Test keyword: healthcare cost

Search → Results:

Steps:

1. Open <http://127.0.0.1:8001/tool/>.
2. Enter healthcare cost and click Search.

Expected:

- URL changes to </tool/results/?subject=healthcare%20cost>.
- Shows “Loading...”, then result cards with title, summary, insights, engagement, relevance bar.
- Pager visible if >10 items.

Change filters + paginate:

Steps:

1. Click a filter (e.g., Priority = High) or change page with pager.
2. Optional: use Back/Forward to confirm state restoration.

Expected:

- List refreshes to match filter.
- Current page highlighted.
- Back returns to prior list state.

Open external source:

Steps:

1. On the first card, click the title.

Expected:

- A new browser tab opens to the article URL.
- No additional API calls to your backend.

View History with date filter + paging:

Steps:

1. Click History in the header.
2. Select Last 7 days. Click page 2.

Expected:

- History table appears with Date, Topic, Engagement.
- Table updates on date change and page change.

Error/empty states:

Steps:

1. Temporarily change API_BASE in config.js to an invalid host, or stop the backend.
2. Reload Results page.

Expected

- Loading clears to an inline error message.
- The subject stays intact so retry is possible.
- Optional: try a keyword that returns zero results to show Empty state

4. Sponsor meeting, feedback and response to feedback

4.1 Meeting Details

Meeting	Date	Time	Platform	Purpose
1	30/09/2025	2:00 PM	Microsoft Teams	Prototype review
2	02/10/2025	3:30 PM	Microsoft Teams	Documentation review
3	24/10/2025	4:00 PM	Microsoft Teams	Documentation Review

4.2 Sponsor Feedback

Meeting 1 (Prototype Review - 30/09/2025)

Backend

- Expand the data sources, preferably Australia based news outlets to ensure local relevance.
- Change the naming for collections in MongoDB Atlas “posts” to something more meaningful.
- Avoid cramming all API sources into a single file (fetchers.py). Instead, create a separate and dedicated file/class for each API source. This makes the codebase cleaner, easier to maintain and more flexible for future updates.
- Improve the formatting so the code is more manageable, which is easier for them to change

Machine learning

- Replace or supplement TF-IDF with BM25 for more robust lexical relevance.
- Explore additional pre-trained models beyond SBERT to improve semantic coverage and accuracy.
- Investigate and implement new algorithms (e.g., hybrid retrieval, rerankers) for better overall results.
- Add a neural network layer to combine weights from TF-IDF, SBERT, and engagement features, instead of relying on manual weighting.
- Overall ML pipeline design looks and works great; focus should now be on improving accuracy and scalability.

Meeting 2 (Documentation Review - 02/10/2025)

- This meeting unfortunately did not go through as planned due to some miscommunication within the team. The issue will be addressed in the next team meeting on 03/10/2025 at 1500 hours.

Meeting 3(Documentation Review of User Manual and Team Retrospective)

The third sponsor session focused on reviewing the near-final documentation deliverables, particularly the **User Manual** and **Team Retrospective**.

Feedback Summary

1. **Visual Enhancements:**

The sponsor emphasised the importance of **including screenshots, diagrams, and interface images** throughout the User Manual to make it more engaging and accessible for non-technical users.

2. **API Specification Improvements:**

The sponsor requested the addition of **parameter descriptions** for each API endpoint, including expected input types, output formats, and example responses.

This ensures the documentation can serve as a standalone developer reference.

3. **Code Snippets:**

It was recommended to incorporate **sample request and response snippets** (in JSON or cURL format) to demonstrate the practical use of APIs, thereby improving usability for future integration teams.

4. **Model Integration Transparency:**

The sponsor asked the team to explicitly document the **Hugging Face LLM** used for semantic understanding and to note that **OpenRouter** was employed as the intermediary platform for model access.

This addition enhances reproducibility and showcases the project's use of advanced, industry-standard AI infrastructure.

4.3 Team Response / Action Plan

Backend Pipeline Enhancements (from Meeting 1)

- **Codebase structure:** The backend team will refactor the current implementation by splitting API connectors into separate files. This change will be applied in the next sprint to ensure modularity and maintainability. Future additions or modifications will follow this structure.

- **Adding More Sources:** The backend team will research and prioritise integrating additional Australia based news APIs. Potential candidates include ABC News, Sydney Morning Herald or any other publicly available APIs or RSS feeds.
- **Changing the naming for collections:** The backend team will work on a better and meaningful naming to replace the word “posts”.
- **Data Scrapping** The backend team will work on the data scrapping so it also include the engagement matrix for RSS

ML Pipeline Enhancements (from Meeting 1):

- **Lexical scoring:** Integrate BM25 via `rank_bm25` to replace or complement TF-IDF for more robust retrieval.
- **Semantic models:** Benchmark SBERT against other pre-trained transformer-based models (e.g., all-MiniLM, MPNet, GTE) and select those with the best trade-off between accuracy and latency.
- **Hybrid scoring:** Implement candidate reranking pipelines that combine lexical and semantic methods.
- **Neural network for weights:** Add a lightweight feedforward neural network to learn optimal weights for combining TF-IDF, BM25, SBERT, and engagement features, using synthetic labels at first, then sponsor-provided relevance feedback.
- **Scalability:** Continue using modular pipelines and caching strategies to ensure the ML layer remains performant and cost-effective.

Meeting 3 Enhancements:

- **Visual Enhancements:**
The documentation team will embed **screenshots and system diagrams** into the User Manual and Developer Guide to improve readability and engagement for non-technical stakeholders.
- **API Specification Expansion:**
Each API endpoint will include detailed **parameter descriptions, input/output formats**, and **example JSON request–response snippets**. This addition enhances transparency for developers integrating the system.

- **LLM Integration Disclosure:**
A dedicated subsection in the AI Pipeline documentation will describe the **Hugging Face LLM** leveraged for contextual understanding and explicitly note the use of **OpenRouter** as the model gateway. This improves reproducibility and demonstrates the project's reliance on industry-grade AI infrastructure.

These updates not only aligned our deliverables with the sponsor's expectations but also enhanced the overall professionalism and technical completeness of our final documentation package.



Robotic Marketer Pty Ltd
Team Retrospective

<AI-Powered Web Application for Social Listening>

Version <01>, <25/10/2025>

Table of Contents

1. Introduction

- 1.1. Purpose of Team Retrospective
- 1.2. Project overview

2. Feasibility and Initial Planning Phase

- 2.1. Initial Planning and Setup
- 2.2. Why We Chose This Approach
- 2.3. What Went Right Early On
- 2.4. What We Could Have Done Better
- 2.5. Takeaways from the Feasibility Stage

3. Planning and Team Organisation Phase

- 3.1. Building the Team Structure
- 3.2. How We Planned Our Workflow
- 3.3. What Worked in Our Planning
- 3.4. Gaps and Challenges We Faced
- 3.5. Lessons on Coordination and Adaptability

4. Design and Development Phase

- 4.1. Overview of the Development Phase
- 4.2. Why We Did It (Design and Implementation Choices)
- 4.3. What Worked Well (Collaboration and Progress)
- 4.4. What Could Have Been Done Differently (Technical and Time Constraints)
- 4.5. Lessons for Future Projects

5. Testing and Feedback Integration Phase

- 5.1. Presenting Our Prototype
- 5.2. Key Feedback from the Sponsor
- 5.3. What Worked Well in Refinement
- 5.4. What We Would Improve Next Time
- 5.5. Lessons Learned

6. Achievement of Learning Outcomes

- 6.1. Team Collaboration and Communication
- 6.2. Technical Skills and Problem-Solving
- 6.3. Professional and Client Engagement
- 6.4. Reflection on Personal and Team Growth

7. Conclusion

- 7.1. Final Thoughts and Overall Reflection
- 7.2. Future Recommendations

1. Introduction

1.1. Purpose of Team Retrospective

The purpose of the team retrospective is to reflect on the development process of the Pace-Unit project, identify successes, evaluate challenges, and gather insights that will guide future improvements. It provides an opportunity for the team to analyze how effectively technical, collaborative, and organizational goals were met during the implementation of the backend system.

This reflection emphasizes continuous improvement which ensures that future development cycles are faster, more efficient, and more aligned with user and system requirements.

1.2. Project Overview

The Pace-Unit project is a full-stack application designed to automate information retrieval, AI-based summarization, and historical trend analysis.

Its backend, built using Django REST Framework with MongoDB, serves as the system's core API and data pipeline.

Key features include:

- **Automated Search and Data Collection:** Fetches posts, news, or documents from multiple online sources (e.g., Reddit, TechCrunch, RSS feeds).
- **AI Processing:** Uses an AI engine to analyze, summarize, and score relevance of fetched data per topic or keyword.
- **Historical Tracking:** Each user search creates a history document, storing parameters and results for later retrieval without reprocessing.
- **Trend Analytics:** Generates daily, source-based, and tag-based statistics on processed content to identify topic trends.
- **Data Management:** Implements safe cleanup logic that preserves AI results while removing unused raw data to optimize storage.

This architecture balances efficiency and scalability, supporting asynchronous AI processing, RESTful endpoints for frontend integration, and flexible data relationships among `history`, `raw_insights`, and `ai_results`.

2. Feasibility and Initial Planning Phase

2.1 Initial Planning and Setup

At the inception of the project, the team's foremost goal was to confirm whether developing an *AI-Powered Web Application for Bot Listening* was technically viable within the given timeline and available resources. We began by reviewing potential project ideas through brainstorming sessions and shortlisted three possibilities related to artificial intelligence and automation. The team unanimously agreed to pursue the *Bot Listening* concept because it directly aligned with the growing importance of social listening and content intelligence in modern marketing, while also enabling us to demonstrate practical applications of natural-language processing (NLP) and machine learning (ML).

During this stage, the team conducted **preliminary literature and market research** to validate the feasibility of the idea. This research provided clarity on how our system could differentiate itself, by focusing on **automated competitor identification and content-distribution recommendations** through AI ranking models.

The setup involved establishing a foundational technical environment. We configured **GitHub Repositories** for collaborative version control, created a shared **Google Drive workspace** for documentation, and defined the initial **Django project structure** within *Visual Studio Code*. Concurrently, exploratory scripts were tested using *spaCy* for entity recognition and *BM25* for text-based ranking, which confirmed that our chosen AI approach was both achievable and computationally efficient.

2.2 Why We Chose This Approach

The team's decision to build an AI-driven web application stemmed from two primary motivations:

1. It allowed us to integrate **academic knowledge** in AI algorithms, databases, and web frameworks into a real-world scenario.

2. It addressed a tangible industry problem, the need for businesses to **extract insights from unstructured online content** without manual analysis.

By combining *lexical models* (e.g., BM25, TF-IDF) with *semantic models* (e.g., Sentence-BERT and BART Zero-Shot), our solution could assess both textual relevance and contextual meaning, something traditional social-monitoring tools rarely offer.

Additionally, we selected **Django REST Framework** for the backend due to its scalability and modular design, and **MongoDB** as our database for flexible, schema-less storage of AI outputs. This hybrid approach offered a balance between research innovation and engineering practicality, a crucial consideration during feasibility analysis.

2.3 What Went Right Early On

Several elements of our initial planning phase proved successful:

- **Clear Vision and Defined Roles:**
From the outset, the team agreed on a shared project vision and clearly defined roles for each member, including AI lead, backend developer, and project manager. This ensured accountability and allowed everyone to specialize in their strengths.
- **Early Technical Validation:**
We rapidly prototyped entity-recognition and text-ranking scripts to confirm feasibility. The successful execution of these scripts built confidence in our technical direction and justified the use of NLP-based ranking algorithms.
- **Efficient Communication Channels:**
The team utilized WhatsApp and Discord for daily coordination and scheduled weekly Microsoft Team meetings to review progress. This structure minimized miscommunication and kept all members aligned.
- **Sponsor Alignment:**
Preliminary meetings with our sponsor clarified expectations regarding system scope, timeline, and deliverables, allowing us to avoid ambiguity later in the development phase.

2.4 What We Could Have Done Better

Despite a productive start, certain aspects of our feasibility planning could have been improved:

- **Time Estimation:**
We underestimated the time required for **data pre-processing** and **text cleaning**, which later became a bottleneck when integrating large datasets. More accurate task scoping during feasibility would have prevented subsequent delays.
- **Documentation Discipline:**
While discussions were frequent, early technical decisions were not always logged immediately in our shared documentation. This led to minor confusion when revisiting earlier design choices.
- **Risk Assessment:**
Our initial risk plan did not fully account for model-loading delays or potential issues with GPU availability for transformer models. A more comprehensive risk analysis could have helped us plan computational resources more effectively.
- **Dependency Management:**
Library version inconsistencies arose when team members ran scripts locally on different operating systems. Establishing a standardized requirements file earlier could have minimized compatibility issues.

2.5 Takeaways from the Feasibility Stage

The feasibility phase was foundational to the project's eventual success, providing both **technical validation and team alignment**.

Our main takeaways include:

- **Start Simple, Iterate Fast:** Rapid prototyping using minimal datasets enabled us to confirm core model feasibility before scaling up.
- **Define Deliverables Early:** Setting clear measurable outcomes, such as “returning top five ranked competitors per query”, ensured focused progress.
- **Invest in Documentation and Communication:** Early documentation not only helps maintain continuity but also strengthens transparency among team members.

- **Anticipate Risks:** Identifying technical and logistical risks early allows more realistic scheduling and prevents last-minute roadblocks.
- **Balance Innovation and Practicality:** The decision to use a combination of research-oriented NLP models and stable, production-ready web frameworks set a solid foundation for later phases.

3. Planning and Team Organisation Phase

3.1 Building the Team Structure

At the start of the project, our team established a well defined structure that allowed each member to contribute according to their strengths while maintaining flexibility. Roles such as project lead, technical lead, documentation lead, testing lead and quality assurance leads were clearly defined.

For example:

Name	Role	Responsibility Highlight
Anirudh Gupta	Group leader	Oversees project direction and coordination
Khawaja Abeer	Technical lead	Guides technical decisions and architecture
Eric Kok	Documentation lead	Main meeting minutes and documentation
Yousef Diab	Quality assurance lead	Ensures deliverables meet quality standards
Amr Abdelghaffar	Quality assurance lead	Support quality checks and validation
Edward Chairudin	Testing lead	Designs and manages testing processes

This structured approach allowed every member to understand their responsibilities and ownership areas from the beginning. Additionally, we also emphasised cross collaboration, meaning members were encouraged to assist in other domains when needed. This is to ensure flexibility during high pressure periods like deliverable deadlines and testing weeks.

3.2 How We Planned Our Workflow

Our workflow was guided by agile principles which allow us to efficiently and effectively adapt to evolving requirements and sponsor feedback. We divided it into iterative sprints that aligned with deliverables in order to ensure that progress was steady and measurable.

Project Pipeline Structure

To streamline development and ensure effective collaboration, we divided the entire system pipeline into three main components:

- **Frontend:** Responsible for building the user interface and ensuring data fetched from the backend was displayed effectively. This included UI design, state management and API integration.
- **Backend:** Handled data processing, API creation and integration with the MongoDB database. This team ensured that data flow between the frontend and AI layers was smooth and efficient.
- **AI Module:** Focused on data enrichment, summarisation and ranking using AI models. The AI pipeline was connected to the backend for automated processing and data retrieval.

Each member was assigned to one of these pipelines based on their strengths, technical expertise and study focus. This alignment ensured that every component was managed by someone capable and confident in that domain.

Our planning approach involved:

- **Weekly team meetings:** To review current progress, set goals and identify concerns early.
- **Discord & WhatsApp** for daily communication and quick problem-solving.
- **Google Docs** for shared document editing and assignment submissions.
- **GitHub** for version control, pull requests and tracking backend, frontend and AI contributions.

- **Sponsor updates:** After sponsor meetings, we adjusted our workflow to reflect feedback or new feature priorities.
- **Weekly reflections report:** To maintain accountability and foster team reflection, we implemented a rotating weekly reflection system. Instead of one member handling all documentation, each person was responsible for preparing the weekly reflection report for a specific week.

For example:

Weekly Reports - every week saturday 11:55pm

Week 2	Khawaja
Week 3	Anirudh
Week 4	Eric
Week 5	Yousef
Week 6	Amr
Week 7	Edward
Week 8	Khawaja
Week 9	Anirudh
Week 10	Eric
Week 11	Yousef
Week 12	Amr
Week 13	Edward

3.3 What Worked in Our Planning

Several aspects of our planning phase proved highly effective and contributed to the smooth execution of later stages:

- 1) **Early role assignment and expectations:**
 - Everyone knew their focus areas from the start, which reduced overlap and confusion during the initial weeks.
- 2) **Consistent communication:**
 - Regular weekly team meetings were consistent, which kept everyone informed and minimised miscommunication.
- 3) **Incremental progress and flexibility:**
 - Breaking tasks into smaller milestones made large objectives manageable and allowed us to pivot quickly when challenges arose.
- 4) **Sponsor alignment:**

- Integrating sponsor feedback early into our planning ensured that our product development stayed relevant and aligned with sponsor's expectations.

5) Mutual accountability:

- Each team member was responsible not only for their work but also for supporting others when tasks became interdependent.

3.4 Gaps and Challenges We Faced

While our planning structure was effective overall, we also encountered several challenges that disrupt our coordination and adaptability:

1) Scheduling and availability:

- Coordinating meeting times that suited everyone, including sponsor sessions, was difficult due to varied class schedules, part time jobs and other commitments.

2) Uneven workload distribution:

- At certain stages, the workload was unevenly distributed across members, mainly due to differences in technical familiarity and experience levels. We mitigated this later by redistributing supporting tasks, such as documentation and other things.

3) Version control conflicts:

- As multiple members worked on interconnected components, GitHub merge conflicts and inconsistent local environments occasionally caused setbacks. These were resolved by structuring code reviews.

4) Requirement changes:

- During mid project, sponsor feedback occasionally required us to adjust features or redesign endpoints, which impacted our initial timeline. While challenging, these moments also strengthened our ability to adapt under pressure.

3.4 Lessons on Coordination and Adaptability

Reflecting on this phase, the most valuable lesson we learned was that effective coordination is built on proactive communication and flexibility. No plan remains perfect throughout a project, so the ability to adapt quickly became our biggest strength.

Key lessons learned:

- **Flexibility in roles:** Members occasionally stepped outside their assigned roles to help others, which strengthened team collaboration.

- **Proactive communication:** Bringing up potential issues early, even small ones which can prevent delays later in the sprint.
- **Centralised task tracking:** Having a shared progress tracker ensured everyone was aware of task ownership and deadlines.
- **Balanced leadership:** Instead of a strict hierarchy, decision making was shared. For example, even though the project lead guides discussions, however, all members should contribute ideas and feedback equally as well.

4. Design and Development Phase

4.1 Overview of the development phase

The development phase focused on constructing a functional end-to-end pipeline that could retrieve online content, analyze it, and present it through a clean and intuitive user interface. The key focus areas included:

- **Frontend:** Implementing the search interface, results display cards and other necessary cards in the scope of the project.
- **Backend:** Developing the Django REST API to manage search queries and serve processed results.
- **Database Layer:** Structuring MongoDB collections to store retrieved articles, history and user information.
- **AI/NLP Layer:** Integrating an NLP-based ranking mechanism to score relevance and generate summaries and repurposing recommendations.

4.2 Why we did it (Design and implementation choices)

- **Django** was chosen for the backend due to its stability, rapid development capabilities, and built-in admin utilities.
- **MongoDB** was selected as the database because the article data is largely unstructured and varies across different online sources, making a document-based storage model more suitable than a relational one. Additionally, the sponsor organisation already uses MongoDB within their existing systems, which supports smoother alignment,

interoperability, and future integration of our solution into their workflow.

- **Component-based UI design** ensured the interface could be expanded later when sponsors integrate the frontend into their internal system.
- **NLP-based** techniques were used to rank articles by relevancy, as keyword matching alone cannot capture contextual meaning or topic alignment. LLMs were then applied separately for content interpretation tasks such as summarisation, tone analysis, and generating insights for the opportunity, idea, and repurposing cards. This separation allowed the ranking pipeline to remain efficient while still leveraging deeper semantic understanding where needed.

Development followed an **iterative and incremental approach**, where the system was tested and refined in small functional blocks to ensure stability and usability.

4.3 What worked well

- The team maintained **clear task ownership**, which reduced overlap and confusion.
- Use of **regular check-ins and version control** allowed us to stay aligned and resolve blockers efficiently.
- The **API and frontend integration** worked more smoothly than expected due to early agreement on data formats.
- The team adapted quickly when requirements evolved, showing strong **flexibility and communication**.

4.4 What Could Have Been Done Differently

- The ranking algorithm could be improved with additional **fine-tuning data and evaluation metrics**, but time was limited.
- The **database** could have been finalised early on in the project to avoid a rework later.
- Better **parallelisation of tasks** early on may have reduced pressure in the final development weeks.

4.5 Lessons for future projects

- One of the key lessons is the importance of **establishing the database schema and storage strategy early** in the project. Since much of the system depends on how data is structured and retrieved, defining the database first would have streamlined the integration of backend logic and reduced rework later.
- Aligning data formats and API response structures at the beginning makes front-end and back-end development progress more smoothly in parallel.
- Regular communication with the sponsor helps clarify expectations and ensures that feature development remains aligned with real use cases.
- Prioritising the **core functionality** before enhancements ensures that the minimum viable system is stable and testable earlier in the development cycle.
- Scheduling internal milestones earlier in the timeline helps avoid last-minute pressure and supports more thorough testing and refinement.

5. Testing and Feedback Integration Phase

5.1. Presenting Our Prototype

At this stage, we developed a functional prototype of our AI-powered Web Application for Social Listening, designed to help clients identify and analyse relevant social media posts and online articles based on subject, industry, and location. The prototype included the core search functionality, and the relevancy scoring system. During the presentation to our sponsor, we demonstrated the full workflow, of entering a query, retrieving results, and viewing the relevance-ranked content. The demonstration aimed to highlight the ability of our natural language processing (NLP) based relevance model.

5.2. Key Feedback from the Sponsor

The sponsor provided valuable and constructive feedback that guided the refinement of our MVP. The main points were:

- **Dashboard cards:** During the initial presentation of the prototype, the dashboard card system was not yet functional, with the cards not yet able to display all the needed information, such as trending topics. Feedback was given on the
- **Relevance system:** The sponsor was happy with the quality of the relevance ranking system and the precision of the retrieved posts, meeting a significant requirement of the project.
- **Filter User Interface:** Some improvements to the UI of the filtering system were requested to enhance user clarity, and ease of use, particularly in ensuring that the options available for each filter are clear to the user, and to minimize the number of clicks needed to select a particular filter.
- **Priority Filter:** Feedback was also given on the priority filter, which displays the posts and articles which are most likely to be a priority for the company. It was requested that we review and experiment with the formula used to determine priority content, to ensure it is more relevant to the company.

5.3. What Worked Well in Refinement

In the refinement phase, several improvements were successfully implemented:

- The user interface for the filters was improved, allowing for a clearer, more simple user experience. This is achieved by no longer using a drop-down menu, allowing for the available filters to be easily seen and selected with a minimal number of clicks.
- The cards system was successfully implemented, providing trending topics, relevant backlinks, and displaying possible opportunities for the business.

5.4. What We Would Improve Next Time

Several areas could be improved in future iterations:

- We would prioritize making the home page cards fully functional earlier in the development, as their absence limited the feedback that could be given for the prototype.
- We would also plan more structured user testing sessions before sponsor feedback, to better ensure that all major features of the prototype are functional and ready for feedback.

5.5. Lessons Learned

This phase underscored the importance of iterative testing and early stakeholder engagement. By involving the sponsor throughout the development, we were able to ensure that progress made was within the expectations of the sponsor. We also learned that strong backend features must be paired with a clear user interface to ensure that the end users are able to fully utilise the features developed. Finally, we learned the importance of time management, and ensuring that the essential features of the prototype are ready and functional when needed to demonstrate to the sponsor.

6. Achievement of Learning Outcomes

6. Achievement of Learning Outcomes

6.1 Team Collaboration and Communication

We set up regular, organized communication by having weekly planning and review meetings, daily updates on Discord and WhatsApp that didn't have to happen at the same time, and shared Google Docs with minutes and action items linked to issues. Clear roles (project lead, technical lead, documentation, testing, QA) made things less confusing, and flexible cross-support during busy times kept things from getting stuck. We improved code quality and knowledge sharing by using collaboration tools like pull-request reviews, small atomic changes, and a no direct-to-main policy. We also worked together on complex merges and interface contracts. We used acceptance criteria and previous decisions to settle disagreements, and when necessary, we ran short spikes to test different options before making a final decision. As a result, there were fewer misunderstandings, feedback was used to quickly get everyone on the same page, and everyone had the same idea of what was important and what was not.

6.2 Technical Skills and Problem Solving

We demonstrated end to end delivery skills across data, AI, backend APIs, and integration.

- **Architecture and APIs:** Designed and implemented Django REST endpoints backed by MongoDB, with resource models that support AI outputs and history tracking.
- **Asynchronous processing:** Implemented background execution for long running AI tasks to keep APIs responsive.
- **Information retrieval and NLP:** Prototyped and combined lexical methods and semantic models to rank and summarize content by relevance and intent.
- **Data quality and lifecycle:** Added preprocessing, deduplication, and safe cleanup that preserves AI results while trimming raw inputs. Indexing strategies improved query latency as data grew.
- **Tooling and reliability:** Standardized requirements files, environment variables, and consistent local setups. Code reviews and smoke tests caught regressions early.
- **Outcome:** The team can reason about trade offs between performance, accuracy, and maintainability, and can design APIs that integrate cleanly with an AI pipeline.

6.3 Professional and Client Engagement

We practiced delivering professionally in line with stakeholder expectations: we turned feedback into backlog items with clear scope, priority, and acceptance criteria, and we brought up timeline impacts early; we ran focused prototype demonstrations that linked requirements to outcomes and recorded feedback in meeting notes for validation in the next sprint; and we kept technical documentation, API descriptions, and model rationale short so that future contributors could follow decisions without having to rediscover context. Because of this, sponsors saw steady progress, could connect features to goals, and we got better at showing trade-offs and talking about scope.

6.4 Reflection on Personal and Team Growth

We improved how we learn, plan, and deliver as a group.

- Time and scope realism: Early underestimates around preprocessing and environment setup taught us to include setup, data preparation, and integration overhead in plans.
- Adaptability: Requirement changes and library version differences were handled through shorter iterations, spikes, and standardization.
- Quality mindset: QA leads introduced simple checklists, PR templates, and test expectations, which raised baseline quality without slowing delivery.
- Peer learning: Members increased capability outside their primary roles through pairing and reviews (e.g., AI members assisting with API contracts and vice versa).
- Ethical and responsible use: We discussed data sourcing, attribution, and summarization limits, and recorded constraints in documentation and configuration defaults.
- Outcome: Stronger ownership, better communication under time pressure, and repeatable processes we can carry into future projects.

7. Conclusion

7.1. Final Thoughts and Overall Reflection

The current backend implementation of Pace-Unit demonstrates a clear, maintainable, and scalable design.

- It successfully separates **data ingestion**, **AI processing**, and **presentation layers**, ensuring that each module can evolve independently.
- MongoDB was used effectively as a document-based database, supporting flexible schemas suitable for AI-generated and unstructured data.

- The asynchronous processing model ensures that long-running AI tasks do not block the main API workflow, maintaining responsiveness for frontend users.
- The use of denormalized fields in `ai_results` minimizes query complexity, allowing the frontend to access processed summaries instantly.

From a development standpoint, the team showed improvement in version control discipline, backend-frontend coordination, and overall system testing.

While some areas (like testing coverage and data visualization for trends) can still be expanded, the foundation is strong for production-scale operation.

7.2. Future Recommendations

To enhance performance, maintainability, and scalability, the following improvements are recommended:

1. Performance & Scalability

- Introduce **asynchronous job queues** (Celery or RQ) instead of background threads for AI processing.
- Implement caching layers for frequently accessed results or trends.
- Continue refining MongoDB indexes as data volume grows.

2. Data Lifecycle Management

- Extend the cleanup routine to optionally remove **old `ai_results` and `history`** entries when no longer needed.
- Implement **archiving strategies** for historical trends to reduce active database load.

3. Monitoring & Logging

- Add structured logs with timestamps and batch statistics for every search and AI run.
- Integrate health metrics (e.g., AI processing latency, database read/write times).

4. Testing & Quality Assurance

- Expand automated test coverage for APIs and AI output validation.
- Introduce unit tests for trend aggregation and cleanup logic.

5. Frontend Integration

- Improve visualization of trend analytics and topic evolution.
- Provide real-time progress indicators for long AI runs using WebSocket or async polling.

Overall, the project has achieved a stable and functional backend capable of handling complex AI-driven search operations. With continued optimization and testing, it evolve into a robust, production-ready intelligence platform.



Robotic Marketer Pty Ltd
User Manual

<AI-Powered Web Application for Social Listening>

Version <01>, <25/10/2025>

Contents

1. Introduction

- 1.1. Purpose of the document
- 1.2. Overview of the system
- 1.3. Intended audience

2. System overview

- 2.1. Project objectives
- 2.2. Technology stack

3. Installation and setup

- 3.1. System requirements
- 3.2. Setup guide

4. Application structure

- 4.1. Directory layout overview
- 4.2. Django App components
- 4.3. Data flow between modules

5. API Documentation

- 5.1. Endpoint list and description
- 5.2. Input/Output formats
- 5.3. Example Usage
- 5.4. Error codes and response handling

6. AI/NLP pipeline

- 6.1. Overview of text processing workflow
- 6.2. Feature extraction
- 6.3. NLP model integration
- 6.4. Ranking algorithm explanation
- 6.5. Limitations and future improvements

7. Data management

- 7.1. MongoDB schema design
- 7.2. Data insertion/removal process
- 7.3. Data Relationships and Integrity

8. Usage guide

- 8.1. Launching the application
- 8.2. Submitting the search query
- 8.3. Viewing and interpreting ranked results
- 8.4. Optional export or dashboard Features

9. Troubleshooting and maintenance

- 9.1. Common issues and fixes
- 9.2. Dependency conflicts

9.3. API rate limits/Key management

9.4. Updating models/pipelines

1. Introduction

This document provides an overview and guide for using Robotic Marketer's AI-Powered Web Application for Social Listening. The platform enables organizations to monitor online conversations and articles, identify trends, and extract actionable insights from social media and news sources based on industry and location.

By leveraging AI and natural language processing, the system helps businesses understand how topics relevant to their industry are being discussed across digital platforms, supporting better marketing, and strategic decisions.

1.1. Purpose of the document

The purpose of this document is to describe the structure, functionality, and assist in the operation of the AI-Powered Web Application for Social Listening. It serves as both a technical reference for Robotic Marketer's team, and a user guide for end users.

Specifically, this document aims to:

- Outline the system objectives and overall architecture of the platform.
- Describe the technology stack and installation/setup procedures.
- Explain the application structure, including its Django components, API routes, and data flow.
- Document API functionality, covering endpoints, data formats, and example requests/responses.
- Detail the AI/NLP pipeline, including model integration and ranking algorithms.
- Guide end users through running searches, viewing the ranked results, and dashboard features
- Provide troubleshooting and maintenance guidance, addressing common issues and model updates.

1.2. Overview of the system

The AI-Powered Social Listening Platform is a web-based application that allows users to:

- Search topics or keywords based on subject, industry, and location.
- Discover highly relevant social media posts and news articles aggregated from multiple online sources.

- Filter and analyse results using AI-driven relevance scoring and sentiment detection.

Key system components include:

- **User Interface (UI):** A web dashboard for conducting searches and viewing results.
- **Search Engine:** AI algorithms for relevance ranking, topic extraction, and semantic search.
- **Data Pipeline:** Continuous data ingestion from verified social media APIs and news feeds.
- **NLP Model:** Sentiment and trend analysis powered by natural language processing (NLP).

1.3. Intended Audience

This document is intended for:

- **End Users:** Marketing teams, and business managers using the system to track online conversations.
- **Robotic Marketer Stakeholders:** Business managers evaluating the MVP.
- **Developers & Testers:** Robotic Marketer's technical teams responsible for the implementation, testing, and maintenance of the final product.

2. System Overview

2.1 project objectives and core functionalities

The objective of this system is to help researchers and analysts efficiently discover, evaluate, and repurpose online content. The platform retrieves articles from multiple sources, ranks them for relevancy, and provides insights and actionable suggestions for content reuse and distribution.

CORE FUNCTIONALITIES:

- A search bar for querying keywords across online sources.
- A filter system to refine search results.
- A search results page displaying up to 100 ranked articles (10 per page) with:
 - AI-generated summaries
 - Engagement and relevancy score
 - Sorting and descending order of relevancy
- An Opportunity Card that identifies whether an article is suitable for repurposing, and suggests the most relevant publication channel (e.g., blog, newsletter, LinkedIn post).
- A Trending Topics Card showing current trending themes and conversations.
- A News Feed Card consolidating the latest relevant news articles..

- A Content Action Card that allows sending selected articles to a generative writing module to transform them into blog posts, newsletters, or email updates.
- A Backlinks Card highlighting where the topic is being referenced across the web.
- An Idea Card providing content angles, talking points, or follow-up research suggestions.

2.2. Technology stack

- Backend: Python, Django
- Database: MongoDB
- AI: Python, LLM-based processing, algorithms and pretrained Hugging Face models
- Frontend: Django, CSS, HTML, JavaScript

3. Installation and Setup

3.1 System Requirements

Requirement	Specification
OS	Windows 10+, macOS 12+, Ubuntu 20.04+
Python Version	3.12
RAM	Minimum 8 GB
Disk Space	Minimum 2 GB
Dependencies	spaCy, torch, transformers, pymongo, scikit-learn, sentence-transformers

3.2 Setup Guide

1. Clone the Repository:

```
git clone https://github.com/khawaja1447/Pace-Unit
```

2. Create a Virtual Environment:

```
python -m venv venv
source venv/bin/activate    # On macOS/Linux
venv\Scripts\activate      # On Windows
```

3. Install Dependencies:

```
pip install -r requirements.txt
```

```
requirements.txt
1 # AI part
2
3 numpy>=1.26.0
4 pandas>=2.0.0
5 scikit-learn>=1.3.0
6
7 pymongo>=4.15.0
8 python-dotenv>=1.0.0
9 dnspython>=2.8.0
10
11 sentence-transformers>=2.6,<3
12 transformers>=4.0.0
13 torch>=1.7.0
14
15 rank-bm25>=0.2.2
16 openai>=1.0.0
17
18
19 # Backend part
20
21 asgiref==3.9.1
22 certifi==2025.8.3
23 charset-normalizer==3.4.3
24 Django==5.1.12
25 django-cors-headers==4.7.0
26 django-rest-framework==3.16.1
27 dnspython==2.8.0
28 feedparser==6.0.11
29 idna==3.10
30 pymongo==4.15.0
31 python-dotenv==1.1.1
32 pytz==2025.2
33 requests==2.32.5
34 setuptools==80.9.0
35 sgmlib3k==1.0.0
36 sqlparse==0.5.3
```

4. Configure MongoDB:

Create a `.env` file and add your MongoDB URI:

```
MONGODB_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net
```

5. Initialize NLP Models:

```
python -m spacy download en_core_web_sm
```

6. Run the Application:

```
python manage.py runserver
```

4. Application Structure

4.1 Directory layout overview

4.1 Directory layout overview

```
Pace-Unit/
├── Backend/
│   ├── backend/
│   │   ├── settings.py
│   │   └── urls.py
│   ├── api/
│   │   ├── urls.py
│   │   ├── views.py
│   │   ├── models.py
│   │   ├── management/
│   │   │   └── commands/
│   │   ├── migrations/
│   │   └── sources/
│   │       ├── reddit_official.py
│   │       ├── reddit_rss.py
│   │       ├── news_gnews.py
│   │       ├── bbc_rss.py
│   │       ├── techcrunch_rss.py
│   │       ├── guardian_api.py
│   │       └── hackernews_api.py
│   ├── cleanup_raw_insights.py
│   └── persist.py
├── Frontend/
│   ├── ai_listening_tool/
│   │   ├── webapp/
│   │   │   ├── urls.py
│   │   │   ├── listening_tool/
│   │   │   │   ├── templates/
│   │   │   │   │   ├── listening_tool/
│   │   │   │   │   └── base.html
│   │   │   ├── static/
│   │   │   │   ├── ai_listening_tool/
│   │   │   │   │   ├── js/
│   │   │   │   │   │   ├── config.js
│   │   │   │   │   │   ├── api.js
│   │   │   │   │   │   ├── search.js
│   │   │   │   │   │   ├── results.js
│   │   │   │   │   │   ├── history.js
│   │   │   │   │   └── mock_data.js
│   │   │   └── urls.py
│   │   └── views.py
└── AI/
    ├── ranking_algorithm/
    ├── identify_content/
    ├── Text_summarisation/
    ├── Topic_detection/
    └── Mongo/
```

4.2 Django app components

Backend/backend

Settings and root URLConf. Includes api.urls under /api/.

Backend/api

- views.py: health, search_posts (POST), list_history (GET), get_search_status (GET), get_results (GET), mongo_status (GET).
- urls.py: maps the endpoints.
- models.py: FetchLog for basic auditing.
- sources/: pluggable fetchers, exposed through REGISTRY.
- management/commands/seed_sources.py: bootstrap of source registry.
- persist.py: MongoDB access helpers referenced by views.

Frontend/webapp + listening_tool

UI pages under /tool/. Templates render the search form, results, and history. JS calls the backend using API_BASE from config.js.

4.3 Data flow between modules

UI → Backend

User opens /tool/. JS reads API_BASE from config.js. POST /api/search/ with query fields.

Fetch and persist

Backend selects fetchers from sources.REGISTRY. It fetches up to fetch_limit per source. It interleaves and deduplicates by URL. It writes a capped pool (default 500) to Mongo raw_insights. It creates a history record and returns a history_id.

AI processing

AI modules read raw_insights for the history_id. They compute scores (BM25, TF-IDF, SBERT, engagement) and write ranked items to ai_results. They update history.ai_ready, ai_count, and ai_target.

Polling and display

Frontend polls /api/search/status/ until ai_ready=true. It then pages through /api/results/ to render ranked cards. History view uses /api/history/ for recent runs.

Admin and maintenance

Optional cleanup_raw_insights.py can prune processed raw items after a threshold. seed_sources.py can initialize the source registry.

5. API Documentation

Unless noted otherwise:

- **Base URL:** /api/
- **Content type:** application/json
- **Auth:** Not required for prototype/MVP (internal use).
- **Timestamps:** Unix epoch seconds unless stated.

5.1 Endpoint List and Description

Endpoint	Method & Path	Purpose
/api/health/	GET /health/	Verifies if the server is running and accessible. Returns a simple status message.

/api/search/	POST /search/	Initiates a new search request. Creates a history entry, fetches data from connected sources and triggers AI summarisation and ranking.
/api/history/	GET /history/	Retrieves a list of all previous searches made by users, including metadata such as date, subject and completion status.
/api/search/status/	GET /search/status/	Returns the current progress of an active search or AI task (number of processed articles, completion percentage).
/api/results/	GET /results/	Fetches paginated AI-processed results for a given search history ID, including ranked scores and summaries.
/api/trends/	GET /trends/	Provides trend analytics such as frequently searched topics, sources or top-ranked industries.
/api/topics/top/	GET /topics/top/	Provides global trending topics.
/api/debug/mongo/	GET /debug/mongo/	Returns database collection statistics for debugging (document counts, connection status).

5.2 Input and Outputs Formats

1) POST /search/

Query params

- **subject** (string, required) – main query topic.
- **location** (string, optional) – appended to upstream query.
- **industry** (string, optional) – appended to upstream query.
- **sources** (string | string[], optional) – subset of registered sources. Defaults to all.
- **limit** (int, 1–10, optional, default 10) – preview item count.
- **fetch_limit** (int, 10–200, optional, default 120) – per-source fetch size.
- **persist_pool_limit** (int, 50–2000, optional, default 500) – max items cached.
- **days** (int, optional, default 7) – freshness window.

Headers in response:

- X-Sources-Used, X-Fetch-Limit, X-Persist-Pool-Limit, X-Freshness-Days.

Success (200) payload:

```
{
  "history_id": "6530f0...c2",
  "preview": [ { /* first up to 10 deduped/fresh items */ } ],
  "counts": {
    "per_source": {"reddit": 120, "news_gnews": 120},
    "fetched_total": 240,
  }
}
```

```
    "deduped": 180,  
    "fresh": 150,  
    "persisted": 500,  
    "seeded_ai_results": 500  
  }  
}
```

Error:

- 400 (missing subject), 500 (unexpected).

2) GET /history/

Query params:

- limit (int, 1–100, default 20)
- ai_ready (bool, true/false)

Success (200): array of objects:

```
[  
  {  
    "history_id": "6530f0...c2",  
    "subject": "quantum computing",  
    "location": "au",  
    "industry": "technology",  
    "sources_used": ["reddit", "news_gnews"],  
    "created_at": 1729650000,  
    "ai_ready": false,  
    "ai_count": 34,  
    "ai_target": 100  
  }  
]
```

3) GET /search/status/

Query params:

- history_id (required)

Success (200):

```
{  
  "history_id": "6530f0...c2",  
  "subject": "quantum computing",  
  "ai_ready": false,  
  "ai_count": 34,  
  "ai_target": 100,  
  "created_at": 1729650000,  
  "finished_at": null  
}
```

Errors:

- 400 (missing id), 404 (not found), 500.

4) GET /results/

Query params:

- history_id (required)

- status (string, default "done"; queued, done)
- page (int, default 1; ≥1)
- page_size (int, default 10; 1–50)

Success (200):

```
{
  "history_id": "6530f0...c2",
  "page": 1,
  "page_size": 10,
  "total": 97,
  "results": [
    {
      "rank": 1,
      "url": "https://...",
      "ai_title": "...",
      "ai_summary": "...",
      "tags": ["trend", "insight"],
      "relevance_score": 0.92,
      "published_ts": 1729651111,
      "source": "reddit",
      "status": "done"
    }
  ]
}
```

5) GET /trends/

Query params:

- history_id (required)
- days (int, default 30)

Success (200):

```
{
  "history_id": "6530f0...c2",
  "window_days": 30,
  "timeseries": [{"date": "2025-10-01", "count": 12}, ...],
  "top_tags": [["ai", 42], ["nlp", 17]],
  "by_source": {"reddit": 58, "news_gnews": 39}
}
```

6) GET /topics/top/

Query params:

- days (int, default 30), limit (int, default 15)

Success (200):

```
{"window_days": 30, "topics": [["ai", 120], ["startups", 75], ["fintech", 50]]}
```

7) GET /health/

Success (200):

```
{"status": "ok", "version": "v1"}
```

8) GET /debug/mongo/

Success (200):

```
{"db": "rm_dev", "collections": ["raw_insights", "ai_results", "history"], "counts": {"raw_insights": 500, "ai_results": 100, "history": 5}}
```

5.3 Example Usage

1) Initiate a search

- Endpoint: POST /api/search/

macOS / Linux

```
curl -X POST http://127.0.0.1:8000/api/search/ \
-H "Content-Type: application/json" \
-d '{
  "subject": "Artificial Intelligence",
  "location": "Australia",
  "industry": "Technology",
  "sources": "reddit,news_gnews",
  "limit": 10,
  "days": 7
}'
```

Windows (PowerShell)

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/api/search/" `
-Method POST `
-Body '{"subject":"Artificial Intelligence", "location":"Australia", "industry":"Technology", "sources":"reddit,news_gnews", "limit":10, "days":7}' `
-ContentType "application/json"
```

2) Check AI Processing Status

- Endpoint: GET /api/search/status/

macOS / Linux

```
curl "http://127.0.0.1:8000/api/search/status/?history_id=67196b1c2e0f4a3c12345678"
```

Windows (PowerShell)

```
Invoke-RestMethod -Uri
"http://127.0.0.1:8000/api/search/status/?history_id=67196b1c2e0f4a3c12345678" -Method
GET
```

3) Retrieve AI Results

- Endpoint: GET /api/results/

macOS / Linux

```
curl
"http://127.0.0.1:8000/api/results/?history_id=67196b1c2e0f4a3c12345678&page=1&page_size=5"
```

Windows (PowerShell)

```
Invoke-RestMethod -Uri
"http://127.0.0.1:8000/api/results/?history_id=67196b1c2e0f4a3c12345678&page=1&page_size=5" -Method GET
```

4) View Search Trends

- Endpoint: GET /api/trends/
macOS / Linux
`curl "http://127.0.0.1:8000/api/trends/?history_id=67196b1c2e0f4a3c12345678&days=30"`

Windows (PowerShell)

`Invoke-RestMethod -Uri "http://127.0.0.1:8000/api/trends/?history_id=67196b1c2e0f4a3c12345678&days=30"
-Method GET`

5) Get Top Topics

- Endpoint: GET /api/trends/
macOS / Linux
`curl "http://127.0.0.1:8000/api/topics/top/?days=14&limit=10"`

Windows (PowerShell)

`Invoke-RestMethod -Uri "http://127.0.0.1:8000/api/topics/top/?days=14&limit=10"
-Method GET`

5.4 Error Codes and Response Handling

Status	When it occurs	Example body
200 OK	Successful request	<code>{"...": "..."} </code>
400 Bad Request	Missing/invalid parameter (e.g., no subject or history_id)	<code>{"error": "subject is required"} </code>
404 Not Found	No resource for given history_id	<code>{"error": "not found"} </code>
500 Server Error	Unexpected failure (DB, AI worker, source fetch)	<code>{"error": "<message>"} </code>

6. AI / NLP Pipeline

6.1 Overview

The AI engine processes user input through a hybrid NLP pipeline combining symbolic, lexical, and semantic models to achieve deep contextual understanding.

6.2 Feature Extraction

Features include:

- Named entities: organizations, people, products.

- Sentiment cues and linguistic tone.
- Engagement indicators: likes, comments, shares.
- Lexical complexity and frequency-based features.

6.3 NLP Model Integration

- **spaCy**: Named Entity Recognition and tokenization.
- **SBERT**: Sentence-level embeddings for semantic similarity.
- **BM25 & TF-IDF**: Document ranking using statistical weighting.
- **BART (Zero-Shot)**: LLM-based classification of content distribution suitability.

6.4 Ranking Algorithm

A weighted fusion method integrates all models:

$$\text{Final Score} = 0.6(\text{SBERT}) + 0.25(\text{BM25}) + 0.10(\text{Engagement}) + 0.05(\text{TF-IDF})$$

The combined score ensures contextual relevance and engagement-based prioritization.

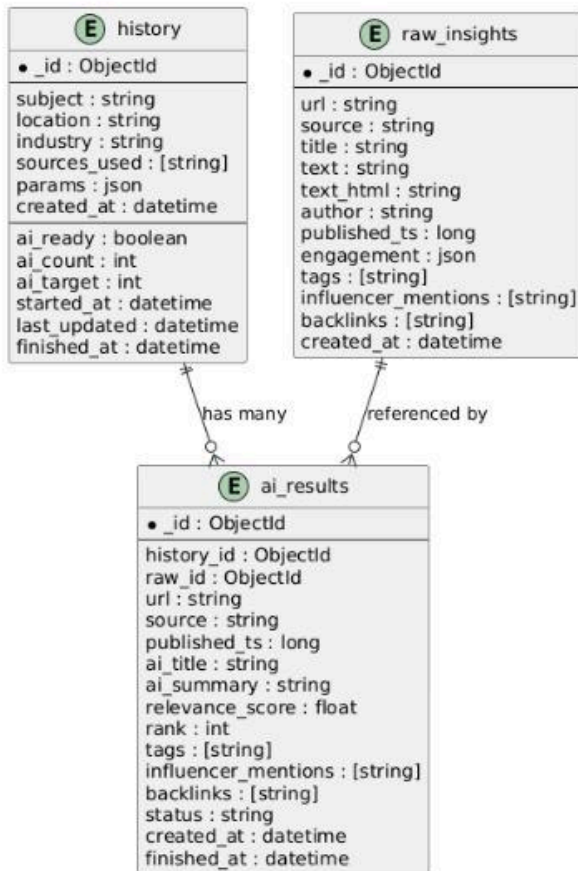
6.5 Limitations and Future Improvements

- Latency in large text processing.
- Single-language (English) limitation.
- Future integration with multilingual embeddings and real-time APIs for Twitter/LinkedIn.

7. Data Management

7.1 mongoDB schema design

Backend Data Schema (MongoDB Collections)



Collections

- **history** — one document per search run.
 - Core fields:
 - subject, location, industry, sources_used[],
 - params{days,fetch_limit,persist_pool_limit},
 - status/progress: ai_ready(bool), ai_count(int), ai_target(int),
 - timestamps: created_at, started_at, last_updated, finished_at, optional expires_at.
- **raw_insights** — global cache of fetched posts (no history_id by design).
 - Core fields:
 - url(unique key), source, title, text, text_html?, author?, published_ts(epoch s),
 - engagement?,
 - enrichment placeholders: tags?, influencer_mentions?, backlinks?,

timestamps: created_at.

- ai_results — AI-enriched outputs per history and per URL.
 - Keys/links: history_id(→ history._id), url (upsert key with history), raw_id? (→ raw_insights._id).
 - AI fields: rank?, relevance_score?, ai_title?, ai_summary?, tags?, influencer_mentions?, backlinks?.
 - Passthroughs (denormalized from raw_insights to avoid joins): source, published_ts.
 - Queue/state: status in {queued,done,failed}, created_at, finished_at.

7.2 Data insertion and removal process

Insertion

Code:

<http://127.0.0.1:8000/api/search/>

/api/search (POST)

- Build query, fetch per source.
- Interleave, dedupe by URL, apply freshness window.
- Persist to raw_insights via persist_raw_insights(rows) → upsert by url (counts created/updated).
- Seed ai_results via seed_ai_result_stubs(history_id, rows) → create queued stubs with (history_id,url) and raw_id back-reference when available.

Async AI worker (_process_history_async)

- Pull ai_results where status="queued" for that history_id.
- Load corresponding raw_insights (by raw_id or fallback by url).
- Produce AI fields (ai_title, ai_summary, relevance_score, tags, etc.).
- Write back via write_ai_results_batch(history_id, items) → upsert on (history_id,url), set status="done", update finished_at.
- Update progress: mark_history_ai_progress, finalize with mark_history_ai_done.

Deletion

Function:

```
def cleanup_raw_insights(days_threshold: int = 7, dry_run: bool = True)
```

Purpose

Reclaim storage by deleting raw_insights documents that are safe to remove—i.e., items referenced only by old, completed histories and not referenced by any other history's ai_results.

Scope (what it touches / doesn't)

- Deletes: raw_insights (safe subset only)
- Never touches: ai_results, history
(So past runs still show up in /api/history and /api/results—only their shared raw cache may shrink.)

Parameters

- days_threshold — only histories with ai_ready=True and created_at < now - days_threshold are considered.
- dry_run — when True (default), prints what would be deleted but makes no changes.

How to run (Windows PowerShell examples)

From your Backend package (venv active):

```
python -c "from api.cleanup_raw_insights import cleanup_raw_insights as run;  
run(days_threshold=7, dry_run=True)"
```

-> Preview only

```
python -c "from api.cleanup_raw_insights import cleanup_raw_insights as run;  
run(days_threshold=7, dry_run=False)"
```

-> Actually deletes safe raw_insights

Operational notes

- Idempotent: Running again soon after typically finds nothing to delete.
- Non-destructive to results: ai_results and history remain; list/detail pages still work because ai_results carries denormalized fields (source, published_ts, summaries, etc.).

- Storage win: Removes heavy raw text (text, optional text_html) that's no longer needed by any history.

7.3. Data Relationships and Integrity

Relationships

- history (1) → ai_results (many) via history_id.
- ai_results (many) → raw_insights (1) via raw_id (best effort) and also by url as a fallback key.
- raw_insights intentionally has no history_id; it's a shared cache keyed by url.

Integrity mechanisms (application-level)

- Deduping:
 - raw_insights upsert by url prevents duplicate raw rows.
 - ai_results upsert by (history_id,url) prevents duplicate outputs per run.
- Idempotency:
 - Seeding stubs is idempotent; re-running AI for the same batch simply updates the same ai_results.
- Denormalization for stability:
 - ai_results copies source and published_ts from raw_insights to avoid fragile joins during list views and to keep history views stable even if raw gets cleaned.
- Safe cleanup:
 - Raw cleanup only deletes raw_insights not referenced by any active/other histories, preventing broken references.
- Progress flags:
 - history.ai_count/ai_target/ai_ready track processing and provide FE gating (e.g., show partial results vs ready).

8. Usage Guide

8.1 Launching the application

1. Start the API.
 - cd Backend/backend
 - python manage.py runserver 8000
2. Start the UI.
 - cd Frontend/ai_listening_tool/webapp
 - python manage.py runserver 8001
3. Open the tool.
 - Go to http://127.0.0.1:8001/tool/

8.2 Performing a search

1. Enter subject. Required.
2. Click on the search button. The run starts and a history entry is created.

Robotic marketer Privacy

Discover what conversations are trending in your industry

Top Topics

AI in Healthcare
Healthcare Cost
Workplace Health
Value Care
Health Care

All Topics Powered by Robotic Marketer®

8.3 Applying filters

Use the left panel to refine results:

- Date range within the lookback window.
- Platforms or sources to include or exclude.
- Select which post sentiments you would want to keep.
- And select if you have a priority you prefer

FILTER

TIME

☐ Today

☐ 7 days

PLATFORM

☐ news_rss

☐ reddit_official

SENTIMENT

☐ Positive

☐ Negative

☐ Spike

TOP INFLUENCERS

☐ Show Only influencers

PRIORITY

☐ High

☐ Medium

☐ Low

8.4 Understanding results

Each card shows:

- Title, source, and published date.
- AI summary of the post.
- Relevance and engagement indicators.

Payers, Providers, and Patients – Oh My!: AI in Healthcare – Key Policy Developments and State Laws - Crowell & Moring LLP

Payers, Providers, and Patients – Oh My! AI in Healthcare – Key Policy Developments and State Laws Crowell & Moring LLP

news_rssOctober 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business - CNBC TV18

AI in healthcare raises liability concerns; most doctors underinsured, says Policybazaar for Business CNBC TV18

news_rssOctober 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer - AZoRobotics

Experts Call for Major Reforms to Make AI in Healthcare Safer and fairer AZoRobotics

news_rssOctober 24, 2025

Insights:

SPIKE

Engagement:

▲ 0

Relevance:

<1234>

8.5 Viewing history and reloading

- Click on the history button in the search results page and it would open the history page. Each row is one run with a timestamp date, query, and engagement counts.
- Select run to reload its results.

Robotic marketer

Search

Search

Search History

Refine by Date

☐ Today

☐ Yesterday

☐ Last 7 days

☒ Last 30 Days

Date Searched	Topic	Engagement
October 25, 2025	iphone	▲ 0
October 25, 2025	samsung	▲ 0
October 25, 2025	samsung	▲ 0
October 25, 2025	samsung	▲ 0
October 25, 2025	samsung	▲ 0

<12345>

AI Topics Powered by Robotic Marketer 9

9. Troubleshooting and Maintenance

9.1 Common Issues

Problem	Possible Cause	Recommended Solution
spaCy model missing	Required language model not installed	Run <code>python -m spacy download en_core_web_sm</code> and verify using <code>spacy.load('en_core_web_sm')</code> .
MongoDB connection refused	Invalid URI or missing credentials in <code>.env</code>	Check <code>.env</code> file, whitelist IP in MongoDB Atlas, and restart backend.
Slow query processing	Large models or no caching	Enable GPU, cache models locally, or batch process queries.

API errors (400/401/429)	Invalid keys or quota exceeded	Verify API keys, review limits, apply retry logic, and rotate keys as needed.
Incorrect or empty AI output	Missing data or invalid model path	Validate input data, check model directory paths, and rerun preprocessing if needed.
Frontend not showing results	Wrong API routes or CORS issue	Verify endpoint URLs, enable CORS in Django, and test APIs in Postman.

9.2 Dependency Conflicts

Always use a virtual environment and lock versions via:

```
pip freeze > requirements.txt
```

9.3 API Limits & Key Management

Secure Storage:

All API keys and access tokens must be stored in the `.env` configuration file and never hard-coded in scripts. Example: `NEWS_API_KEY=your_api_key_here`

Access Control:

Restrict `.env` visibility to authorised team members only. Exclude it from version-control repositories by ensuring `.gitignore` includes it.

Key Rotation:

Periodically rotate API keys (recommended every 60–90 days) to prevent misuse and maintain compliance with vendor security policies.

Rate Limit Handling:

Monitor request counts using logging middleware. If the system frequently encounters HTTP 429 (“Too Many Requests”), implement exponential backoff or token-bucket throttling to moderate requests automatically.

9.4 Updating Models

AI performance naturally degrades over time as language patterns and datasets evolve. Regular updates ensure model relevance and reliability.

- 1. Model Retraining:**
Periodically retrain models (BM25 index, neural weight combiner, or transformers) using fresh, diverse datasets to capture new linguistic and contextual trends.
- 2. Model Replacement:**
When adopting newer Hugging Face or OpenRouter models, validate compatibility with existing tokenizers and embedding dimensions before deployment.
- 3. Version Tracking:**
Maintain a *tracking file* on GitHub specifying model names, versions, and checkpoints used. This facilitates reproducibility and rollback if newer models perform inconsistently.
- 4. Performance Monitoring:**
After each update, benchmark inference latency, accuracy, and memory footprint. Record results in your maintenance log to ensure consistent performance over time.
- 5. Backup and Rollback:**
Always back up previous model files before introducing updated versions. Store backups securely in cloud storage (e.g., AWS S3 or Google Drive) for rollback in case of regression.

9.5 Preventive Maintenance Checklist

Maintenance Task	Frequency	Responsible Role	Purpose
Verify MongoDB connection health	Weekly	Backend Lead	Prevent data-sync interruptions
Review and rotate API keys	Bi-monthly	DevOps / Backend	Maintain security compliance
Benchmark ML performance	Monthly	AI Engineer	Ensure accuracy and efficiency
Update dependencies	Quarterly	All Developers	Prevent compatibility issues
Backup models and data	Weekly	System Admin	Disaster-recovery readiness